e6ai-archiver 2.0.0: reviewer document

Prepared 2026-11-04. Every code excerpt below is copied verbatim, by script, from the files in this package (`e6ai-archiver-2.0.zip`, files under `app/`); the line numbers refer to those exact files.

> **Site status:** on 2026-11-03 e6ai announced it is shutting down; the site is read-only and closes completely by the end of November 2026. Its Terms of Service forbid downloading/storing content except as the site allows. The owner should confirm the intended use is permitted before running.

1. What it does, in plain language

- It makes a local copy of **every post on e6ai.net**: the original image or video file plus all of the post's metadata (tags, rating, score, sources, description, dates, md5 ...). Optionally pools, tags, wiki pages, artists.
- **New in 2.0 (distribution release):** a zip with a friendly folder (`START HERE - README.txt`, `Start e6ai Archiver.bat`, `Check Python.bat`, `app/`, `docs/`); the **Whole archive dashboard fix** (it now agrees with Progress and shows an ETA); "no checksum in listing" for the artists export is INFO; window polish; **safe upgrades from 1.6 - 1.9.1** (refuse a newer DB with exit 11, timestamped backup, transactional migration, old ini read as is, Import settings, old instance still running = exit 5). No live requests.
- **1.9.1 (fix):** the Windows launcher no longer calls `py` blindly. It finds a working Python 3.11+ (existing `.venv`, `py -3`, `python`, `python3`, the usual install folders), skips the Microsoft Store shortcut by actually running each candidate, and if there is none prints install help and exits **9**.
- **New in 1.9:** a **dark mode** (Light / Dark / Follow Windows, instant, saved in `[gui]`), a **whole-archive dashboard**, a **desktop notification** when a run ends, a **low-disk pause** instead of a failure (`--on-low-disk`), a **resume banner** for interrupted runs, **failure-log trimming** (`--failure-log-max-mb`) and **Open failure log** / **Open latest report** buttons. No live requests.
- **1.8:** the **tags export is pulled automatically** (with every posts export, and before every report), so director folders and per-director counts work; **every failed file download is logged** to `failed-downloads.csv` / `.log` in the archive folder; and **`retry-failed`** re-downloads only the failed files (by reason, or from `missing.csv`): CLI, panel mode + **Retry failed files** button, `run.ps1 -Mode retry-failed`.
- **1.7: a completeness report** (`report`, panel mode "Completeness report", and automatically at the end of `full`): the archive vs the latest posts export - expected, saved and verified, missing, failed with the last reason, skipped by filters, extra files, per director - as `completeness-report-<time>.txt` / `.html` plus `missing.csv`, and an **Open report** button.
- It uses **only official interfaces**: the JSON API and the official daily database exports. No HTML scraping, no copy of the website itself.
- **1.6 fast mode (default):** files come from the static host over **16 parallel connections** (1-64) with no per-request limit; API requests keep their own limiter (default 1/s, user-set, no cap any more). A descriptive User-Agent with your username (browser UAs refused); **20 file 403s in a row, or any API 403, stop the run** (exit 3). Requested by the owner on the basis of a Discord message attributed to e6ai's chief architect; **that authorization is not verified**.

- **Safe to stop and restart** at any point; every file is size-checked and written atomically, and `verify` md5-checks the whole archive (bad files deleted or quarantined and downloaded again).
- **New in 1.5:** optional **sign-in** with your e6ai login name + an **API key** (never your password, which e6ai does not accept from tools), so posts hidden from anonymous visitors can be archived. A **Test sign-in** button, optional "remember the key" in the Windows Credential Manager, and automatic re-checking of posts that were hidden on earlier runs.
- **1.4:** a **control panel** is the main window: every option as a checkbox / list / number box in four tabs, a **Browse** folder chooser, live name preview, validation, Start / Pause / Stop. It builds the normal command line and runs it as a hidden background process, so CLI and panel behave the same. Politeness and safety rules are unchanged.
- **1.3:** a **download order** (`--order popular`, `favorites`, `newest`, `oldest`, `smallest-first`, `balanced`) with filters and tiers, driven by the official daily exports (`sync-exports`), an offline `preview` of the top of the queue with size/time per tier, and an order/filter panel in the window.
- 1.2 added the progress window, file names from metadata, folders with one subfolder per director, `preview` / `reorganize` / `repair-db`, and fixes for many crash cases.
- Target platform: **Windows** (also runs on Linux/macOS). No scheduled task is installed (optional, not used).

2. Architecture (short)

```
cli.py            commands: full | run | download | meta | recheck | exports | sync-exports | seed-from-export |
                  stats | verify | report | retry-failed | preview | reorganize | repair-db | test-login ; maps every
failure to an exit code
  config.py       command line > E6AI_* env vars > e6ai-archiver.ini > defaults
  client.py       the ONLY network code: UA policy, global rate limiter, outage-tolerant backoff, 403 stop,
                  Basic auth to the API host only, 401 -> AuthError
  auth.py         1.5: test_login (one request), re-check / re-queue of posts hidden from anonymous visitors
  redact.py       1.5: masks the API key in logs, tracebacks and panel text
  archiver.py     crawl (b<id>/a<id> cursors), download queue, md5 + atomic writes, edge-case handling
  naming.py       templates, presets, source_name, Windows-safe sanitizer, path-length budget
  layout.py       preview and reorganize (dry run first, never deletes)
  exports.py      export bundle sync (sha256, manifest, previous/ fallback), header mapping, incremental SQL merge
  report.py       1.7: completeness report (archive + disk vs posts export), missing.csv
  failures.py     1.8: failed-downloads.csv / .log, reason groups, retry-failed selection; 1.9 trimming
  theme.py        1.9: light / dark palettes, Windows setting, contrast check, ttk styling
  notify.py       1.9: end-of-run desktop notification (toast / balloon / osascript / notify-send)
  priority.py     download order + filters + tiers: the SQLite priority queue
  store.py        SQLite (WAL) + JSONL journals, crash recovery / rebuild
  events.py       progress events + Pause/Stop control (thread-safe, GUI-independent)
  gui.py          1.3 progress window for `--gui <command>` (worker thread + queue)
control_panel.py  1.4 panel logic, no Tk: option/mode tables, build_command, validation, settings, CliRunner
  panel_ui.py     1.4 panel window (tkinter): tabs, Browse, preview, progress/log, Start/Pause/Stop;
                  1.5 sign-in group
  platform_utils.py  long paths, lock-tolerant rename, disk/drive checks, keep-awake, instance lock
run.ps1 / run.bat / run_gui.bat   Windows launchers (in app/ in the 2.0 zip)
Start e6ai Archiver.bat / Check Python.bat / START HERE - README.txt   2.0 zip top level (bundle/ in the repo)
tools/build_dist.py   2.0: builds the user zip (and an optional dev zip with tests)
```

Archive folder: `e6ai.sqlite3`, `posts.jsonl` / `files.jsonl` (journals), `files/<director>/<rating>/<name>`, `db_export/`, `logs/` (run logs and reorganize audit CSVs).

2a. What changed in 2.0 (full list in CHANGELOG.md)

- **Why:** the owner asked to fix the dashboard problem called out earlier, give the UI a once-over and polish, add an "idiot proof" read me, and bundle it as a zip with a friendly folder

structure. The owner has a live 1.9.1 run (~10 % of 526 GiB), so 2.0 must continue it without re-downloading anything.

- **Dashboard root cause and fix:** the `archive` snapshot was emitted once, at the start of `full`, before the posts export was merged, so "Whole archive" showed the old database's handful of rows (e.g. 12 / 55) all run, disagreed with Progress, and "ETA (all)" was blank; a later snapshot also reset the run counters. Now: a fresh snapshot right before every download queue; the panel adds only files saved after it. Expected = non-deleted posts with a file; saved = files present. Regression tests replay the exact event order of a `full` run and check the totals against `archive_counts` and the Progress box.
- **Checksum note:** `artists` is in `NEVER_CHECKSUMMED`; a missing checksum there logs at INFO (not "Last problem"); the posts export still warns.
- **Window polish (nothing removed):** default 1280x860 (DPI-scaled, capped to the screen), minimum 980x640, the log keeps >= 8 lines (status boxes scroll), Cascadia Mono / Consolas log font, a big Start button with Pause / Stop beside it, buttons in two rows, tabs without number prefixes, the Whole archive figures wrap / go to one column, banners wrap in the room the buttons leave, tooltips everywhere.
- **Upgrades:** see section 4.22. Tested against the real 1.6, 1.7, 1.8 and 1.9.1 code (kept in `tests/compat/old_versions.zip`): each old version builds a partial archive against the fake server and is stopped mid-run; 2.0 then resumes it. Checked: no saved file is requested again (md5s, hashes and mtimes unchanged), the DB is backed up and upgraded once, the dashboard equals `archive_counts`, `retry-failed` re-requests exactly the failed files, the report agrees (40 expected, 35 saved, 0 missing, 5 without a file URL), a stray `.part` is reported and not deleted, the old ini loads, and **the old version can still open the upgraded database** (rollback). Also: a running 1.9.1 holding its lock -> 2.0 exits 5 and touches nothing; a DB from a newer version -> exit 11, untouched, no backup; a failed migration rolls back to the old DB; no disk space for the backup -> "nothing was changed".
- **Distribution:** `tools/build_dist.py` builds the zip from the repo (no tests, caches, DB, ini, journals or .venv); `run.ps1` / `install_daily_task.ps1` detect `app\` and use the install folder for data.
- **Live requests for 2.0: none.**

2b. What changed in 1.9.1 (full list in CHANGELOG.md)

- **Why:** on the owner's PC the first run stopped with `py : The term 'py' is not recognized ...` at `run.ps1:131` (`py -3 -m venv .venv`). 1.9.0 assumed the python.org launcher `py` was installed.
- **New `find_python.ps1`** (dot-sourced by `run.ps1`; `run.bat`, `run_gui.bat` and the daily task all go through `run.ps1`, and none of them called `py` themselves). The search order is the existing `.venv`, then `py -3`, `python`, `python3` (each found with `Get-Command`, never called by a bare name), then `%LOCALAPPDATA%\Programs\Python\Python3*`, `%ProgramFiles%\Python3*`, `%ProgramFiles(x86)%\Python3*` and `C:\Python3*`. Every candidate is run with `-c "import sys; ..."`: the Store shortcut and anything older than 3.11 (from `requires-python`) are skipped and listed.
- **No suitable Python:** the tool prints a friendly message and exits **9**. The message covers python.org, ticking "Add python.exe to PATH", reopening PowerShell and rerunning, plus the `winget install -e --id Python.Python.3.12` hint (the tool never runs winget). A missing `venv` / `ensurepip` module is explained and also exits 9.
- **Also fixed:**
- venv and pip exit codes are checked; a pip failure exits **10**.

- pip's console output was being returned from `Initialize-Venv` in 1.9.0 and would have corrupted the Python path.
- A broken `.venv` is moved aside and rebuilt; a half-finished install is completed.
- **New:**
- `-Mode check-python` shows what would be used, without installing anything.
- `install_daily_task.ps1` warns if `.venv` is not set up yet.
- `run.ps1` has a message for exit 8.
- **How it was checked:**
- PowerShell 7.6 was installed on the build box from the official GitHub release tarball.
- All three `.ps1` files parse with no errors.
- PSScriptAnalyzer 1.25 reported 0 findings for PSUseCompatibleSyntax (target 5.1) and for PSUseCompatibleCommands / PSUseCompatibleTypes (the Windows 10 PowerShell 5.1 profile). Its default error/warning rules also reported 0 findings (Write-Host / ShouldProcess style rules excluded).
- `tests/test_v191.py` runs the real scripts with fake python executables: a Store stub, a too-old Python, the py launcher, install folders, no Python (exit 9) and a missing venv module.
- **Not run on Windows PowerShell 5.1 itself, nor against the real Store shortcut.**
- **Live requests for 1.9.1: none.**

3. What changed in 1.9 (full list in CHANGELOG.md)

- **Why:** the owner asked "Any polish or ease of use or quality of life features you would add or improvements you would make? Can you also add a dark mode?"
- **Dark mode.** Header buttons Light / Dark / Follow Windows (default: follows `HKCU\...\Personalize\AppsUseLightTheme`, re-checked every 3 s). Applies instantly, saved at once to `[gui] theme` (the CLI ignores `[gui]`), Ctrl+Shift+D toggles. Every widget themed (tabs, entries, checkboxes, combos, spinboxes, tooltips, log colours, command preview, buttons, bars, scroll bars); all text/log colours at least 4.5:1 contrast (tested); dark title bar on Windows 10/11; no white flash. The HTML report gets `prefers-color-scheme: dark` CSS. Light mode now uses the flat "clam" style on Windows too.
- **Dashboard** (whole archive): saved / expected files with bar and percent, GiB saved, this run, MB/s and files/s, failures now / to retry, ETA for the rest, low-disk line; idle it reads the DB read-only.
- **Notification** at the end of a run (toast -> balloon -> in-panel window), not for Stop / settings errors / short local runs; Basics checkbox, `[gui] notify`.
- **Disk guard.** `--on-low-disk pause|stop` (default pause; `E6AI_ON_LOW_DISK`, ini `on_low_disk`, panel, `run.ps1 -OnLowDisk`): workers wait below `--min-free-gb` and continue by themselves (2 s re-check) or on **Resume**; Stop still works. Pre-Start estimate asks first when the rest will not fit. The daily scheduled task uses `stop` so an unattended run never waits forever.
- **Resume banner** from the `last_run` record in the archive DB (API key never stored).
- **Failure-log trimming** `--failure-log-max-mb` (default 10, 0 = never; env, ini, panel, `run.ps1 -FailureLogMaxMB`): still-failed rows kept, the rest archived to `failed-downloads-archive.*.gz` first.
- **Buttons:** Open archive folder, Open log, **Open failure log**, **Open latest report** (own row).
- **Live requests for 1.9: none.** No API key, no authenticated request, no file download.
- **Screenshots** (simulated data, Linux/Xvfb): `docs/control-panel-dark-basics.png`, `docs/control-panel-light-basics.png`, `docs/control-panel-dark-naming.png`, `docs/control-panel-light-naming.png`, `docs/dashboard-dark.png`, `docs/dashboard-light.png`.

3a. What changed in 1.8

- **Why:** the owner asked to "make the tool pull the tags export automatically, also make it so that it logs each file that fails to download, and there is an option to rerun only failed files".
- **Tags export, automatic.** `full`, `sync-exports` and `exports --download posts` always include `tags`; a missing tags export is fetched even inside the once-a-day window. `report` (and the report at the end of `full` / `retry-failed`) first calls `ensure_tags`: if the tags export is missing or more than 6 h older than the posts export it downloads it **anonymously** (one listing request + one ~6.4 MB file; sha256 verify + cache-bust retry as for every export), loads it and categorizes seeded posts. Unavailable = the report is still written, with a clear note (`_no_director` / `_unknown_director` fallback). `--no-fetch-tags` / `fetch_tags = no` / `E6AI_FETCH_TAGS=0` keep `report` fully offline.
- **Failure log.** `failed-downloads.csv` (time with offset, run, event, post_id, md5, ext, director, status, reason, http_status, tries_this_run, failures_total, url) and `failed-downloads.log` (readable, run start/end lines, summary) in the archive folder, appended every run; `recovered` rows when a file that failed before is saved. URLs without user-info/query/fragment and redacted. Panel log: one warning per failure plus a summary. DB keeps the last reason and now an attempt count (`file_errors.attempts`, migrated).
- **`retry-failed`.** Downloads only posts whose file is `failed` / `missing` / `md5_mismatch`; `--reason` (403, 404, 429, 503, http, network, md5, size, verify, unknown, exact reasons, lists), `--from-csv` (missing.csv, failed-downloads.csv, any `post_id` list), `--dry-run` (no network), then failure log + report. Stopped part-way: not-reached files keep their failed status. Same parallel downloader, so the 20-consecutive-403 stop, back-off and Pause/Stop all apply. Panel mode "Retry failed files only" + a **Retry failed files** button; `run.ps1 -Mode retry-failed -Reason -FromCsv -DryRun`; `run.bat ... retry-failed`. The older "retry" mode is relabelled "Re-queue failed files + download all pending".
- **Live requests for 1.8:** anonymous only - the export listing and the tags export file, descriptive UA `e6ai-archiver/1.8 (by <owner username> on e6ai)`, no API key. No file downloads, no fast mode.
- **Live check:** `report` on a copy of the simulated test archive fetched the tags export (6,394,700 bytes, sha256 OK), loaded 321,143 tag categories (**5,173 director tags**), categorized 193,537 seeded posts, and split the 162,295 expected posts into **4,130 directors** with no `_unknown_director` (`unknown_director` / `anonymous_director` are real site tags). `retry-failed` and the failure log were only run offline.

3b. What changed in 1.7

- **Why:** the owner asked for suggestion #1 (a completeness report) after seeing that the export lists about 160,000 files. Real figures, counted from every row of the **2026-11-03 posts export** (listing https://e6ai.net/db_exports.json, `updated_at` 2026-11-03 23:01 EST; file https://static1.e6ai.net/data/db_export/posts.csv.gz, sha256 checked): 193,772 rows, 31,477 deleted, **162,295 posts with a file = 156,961 images** (png 113,098, jpg 42,859, webp 588, gif 416) **+ 5,334 videos** (webm 4,311, mp4 1,023), 565,031,687,943 bytes = **526.23 GiB**; 114 still pending approval.
- **`report`** (offline) and **`full --report-after`** (default on): buckets expected / verified / unverified / missing / failed / skipped, with reasons (`not_downloaded_yet`, `not_in_archive`, `file_missing_on_disk`, `replaced_on_site`, `http_403`, `http_404`, `http_429`, `network_error`, `md5_mismatch`, `wrong_size`, `verify_*`, `login_only`, `marked_deleted`, `filter`, `tier_limit`), extra /

`.part` / quarantined files, per-director table; txt + html + `missing.csv` (+ `extra-files.csv`). The exit code of `full` changes only if the report fails (1).
- **`file_errors` table**: the parallel downloader and `verify` record why a file failed.
- **Panel**: mode "Completeness report (no network)", "Completeness report when finished" (Basics > Checksums), Advanced > Report options, **Open report** button. `run.ps1 -Mode report -NoReportAfter -ReportExport`.
- **Fix: stale export from the CDN.** A plain download of posts.csv.gz on 2026-11-04 returned the 2026-11-02 file (Cloudflare `max-age` 31 days); its sha256 did not match the listing, so the old code refused it (correct, but it could never get the new one). A mismatch now gets one retry with a cache-busting query, which returned the current file in the live check.
- **Live requests for 1.7:** anonymous only - the export listing and the posts export (the stale and the cache-busted copy), with the descriptive UA. No API key, no file downloads, no fast mode.
- **Tried on a simulated archive** (seeded from the 2026-11-02 export, with simulated saved / failed / deleted files, compared with the 2026-11-03 export, `--rating s,q`): 162,295 expected, every reason counted as planned (233 `not_in_archive` = posts added between the two exports), 3.7 s for the whole pass.

3c. What changed in 1.6

- **Why:** the owner asked to "remove all safety checks and incorporate the checksum check" suggested in a Discord screenshot from a user (DRLa) presented as e6ai's chief architect. **Not verified**: nothing here confirms the identity or that the site authorizes unthrottled downloads. Kept anyway (judgment call): the descriptive UA, browser-UA refusal, the API-host 403 stop, a run-level stop after 20 consecutive file 403s, back-off on 429/503, disk checks, resume, Stop/Pause, single DB writer. A non-blocking note is logged when the API rate is above ~2/s, which is what the e621-family API docs ask for.
- **Parallel downloads** (`fast.py`): `--workers` (default 16, 1-64), one keep-alive session per worker, no per-request limit for `static1.e6ai.net`; `.part` + atomic replace; duplicate md5 fetched once.
- **API rate**: the 2/s hard cap and the panel's "above polite rate" warning/error are removed; default 1.
- **Checksums**: `--check-md5` (inline, default off); `verify` rewritten (parallel hashing, resumable, progress, delete or quarantine, re-queue, `--redownload`); panel mode "Verify checksums (re-download bad files)"; "Verify when finished" after `full` (default on).
- **Panel**: Basics groups Speed (parallel downloads, API requests per second), Checksums, Disk and power; the progress area shows aggregate speed and "N of W parallel downloads active".
- **Estimates**: bandwidth-bound. 525.5 GiB is about **25 h at 50 Mbit/s, 12.5 h at 100, 2.5 h at 500**.
- **Measured locally only** (test server on 127.0.0.1, see section 5): 16 vs 1 workers.
- **No live requests were made for 1.6** (no fast-mode or bulk download against e6ai).

3d. What changed in 1.5

- **Sign in = login name + API key over HTTP Basic auth.** The request asked for username + password; e6ai (e621ng) only accepts the API key from tools, so the password is never

asked for, sent or stored, and the website's login form is not used. The key is created on https://e6ai.net/users/home ("Manage API Keys"; "Manage API Access" on older versions).

- **Panel, Basics tab, "Sign in to e6ai (optional)":** login name (blank = username), masked API key with Show, "Where do I get this?", **Test sign-in** (one request in a background child process with the same UA / rate limit / 403 stop; inline result; 401 = wrong login or key; 403 = stopped, asks before testing again; 3 s cool-down), "Remember API key on this PC" + "Forget saved key" (only when the optional `keyring` package has a working backend; otherwise a one-line note), the re-check checkbox and the number of posts in the archive waiting for sign-in. Login / key removed from Advanced.
- **Key handling:** only in the child's `E6AI_API_KEY`; never in argv, the ini (Save settings skips it), the command preview or the logs (`***`). Basic auth only to `https://e6ai.net`, not to the file server (it used to be on the whole session).
- **Signed-in runs** (`full`, `run`, `download`): non-deleted posts stored as `no_url` are re-fetched 100 per request; posts whose file is now visible go to `pending` and are downloaded in the normal order. Skipped for 24 h when the same account already checked the same remaining set. `--no-login-retry` / `login_retry = false` / checkbox. `recheck` forces it (deleted posts included). `stats.login_required`.
- **CLI:** `test-login` (exit 0 ok / 8 rejected / 3 = 403 / 2 no key); exit **8** for 401 in every command; `auth: signed in as NAME (API key from E6AI_API_KEY, not shown)` or `auth: no (anonymous)` in every log.
- **How hidden posts are detected (from the e621ng source, not checked live):** for a post the visitor may not see, the API omits the file URLs and blanks md5 / ext; the archiver already stored those as `no_url`. Export-seeded posts keep md5 and are never `no_url`; the existing archives here (`/tmp/real13`, `/tmp/live13`) have 0 such rows, so on a real archive the count may be small or 0.
- **Unchanged:** UA with username (`e6ai-archiver/1.5 (by NAME on e6ai)`), 1 req/s default, 2/s hard cap, stop on 403.
- **No live requests were made for 1.5**; none signed in.

3e. What changed in 1.4

- **Control panel** (`run_gui.bat` with no arguments, `run.ps1 -Mode panel`, `e6ai-archiver --gui`, or `e6ai-archiver-gui`). 17 modes, 51 option widgets in 4 tabs; options the mode does not use are greyed out.
- **Background runs:** Start runs `python -m e6ai_archiver --no-config ... <command>` with `CREATE_NO_WINDOW` on Windows. Progress arrives as JSON lines on stdout (`--progress-json`), logs on stderr.
- **Stop / Pause** go through a control file (`--control-file`) polled by the child every 0.3 s. Stop = the same `StopRequested` clean-up as Ctrl-C: `.part` deleted, commit, lock released, exit 130, resumable. Force stop (kill) only after 45 s and a confirmation. Closing the panel also stops the run cleanly.
- **Settings:** Save writes the same `e6ai-archiver.ini` the CLI reads (panel-only keys in `[gui]`, which the CLI ignores; `.bak` made once). The API key is never saved and never in argv (child env only).
- **CLI additions:** `--no-config`, `--login`, and the naming options `--director-exclude`, `--rating-style`, `--name-max-items`, `--multi-sep`, `--name-max`, `--max-path` (were ini-only). Fixed: `meta --max-items` and the naming `max_items` key shared one dest; the naming option is now `--name-max-items`.

- **Unchanged:** UA with username (now `e6ai-archiver/1.4 (by NAME on e6ai)`), 1 req/s default, 2/s hard cap (the panel refuses higher and warns above 1), immediate stop on 403 (the panel explains it and asks before a new network run), md5 checks, `.part` files, SQLite WAL.
- **No live requests** were made for 1.4 (all checks offline / local).

Options in the panel

tab	option (CLI flag)	widget	used by
Basics	e6ai username (`--username`)	text	all
Basics	Archive folder (`--out`)	folder + Browse	all
Basics	Parallel downloads (`--workers`)	number	all
Basics	API requests per second (`--rate`)	number	all
Basics	Check md5 while downloading (`--check-md5`)	checkbox	all
Basics	Verify when finished (full archive) (`--no-verify-after`)	checkbox	full
Basics	Bad files (wrong md5 / truncated / empty) (`--verify-action`)	list	full, verify
Basics	Completeness report when finished (`--no-report-after`)	checkbox	retry-failed, full
Basics	Re-download bad / missing files (verify) (`--redownload`)	checkbox	verify
Basics	Keep free on the drive (GB) (`--min-free-gb`)	number	all
Basics	When the drive is nearly full (`--on-low-disk`)	list	all
Basics	Keep the PC awake while running (`--no-keep-awake`)	checkbox	all
Download order & filters	Download order (`--order`)	list	download, full, preview, run, sync-exports, retry-failed, report
Download order & filters	Minimum score (`--min-score`)	number	download, full, preview, run, sync-exports, retry-failed, report
Download order & filters	Minimum favorites (`--min-favs`)	number	download, full, preview, run, sync-exports, retry-failed, report
Download order & filters	Ratings (`--rating`)	3 checkboxes	download, full, preview, run, sync-exports, retry-failed, report
Download order & filters	Directors (`--director`)	text	download, full, preview, run, sync-exports, retry-failed, report
Download order & filters	Must have tags (`--filter-tags`)	text	full, retry-failed, download, preview, run, report

tab	option (CLI flag)	widget	used by
Download order & filters	Skip files larger than (MiB) (`--max-size-mb`)	number	download, full, preview, run, sync-exports, retry-failed, report
Download order & filters	Skip file types (`--exclude-ext`)	text	download, full, preview, run, sync-exports, retry-failed, report
Download order & filters	Only the first N GiB (`--limit-gb`)	number	download, full, preview, run, sync-exports, retry-failed, report
Download order & filters	Only the first N files (`--top`)	number	download, full, preview, run, sync-exports, retry-failed, report
Download order & filters	Refresh API metadata before downloading (`--no-api-json`)	checkbox	download, full, preview, run, sync-exports, retry-failed, report
Download order & filters	Retry missing / failed files first (`--retry-failed`)	checkbox	download
Download order & filters	Dry run (no network) (`--dry-run`)	checkbox	download, run
Download order & filters	Stop after N posts (`--max-posts`)	number	full, run, recheck
Download order & filters	API tag search (update) (`--tags`)	text	run
Download order & filters	Only posts after id (`--since-id`)	number	run
Download order & filters	Also record deleted posts (`--include-deleted`)	checkbox	run
Download order & filters	Metadata only (no files) (`--metadata-only`)	checkbox	run
Naming & folders	File name (`--name-template`)	list + free text	all
Naming & folders	Folders (`--folder-template`)	list + free text	all
Naming & folders	Posts with several directors (`--multi-director`)	list	all
Naming & folders	Never use as director (`--director-exclude`)	text	all
Naming & folders	Rating written as (`--rating-style`)	list	all
Naming & folders	Max tags per field (`--name-max-items`)	number	all
Naming & folders	Separator between tags (`--multi-sep`)	text	all

tab	option (CLI flag)	widget	used by
Naming & folders	Max file name length (`--name-max`)	number	all
Naming & folders	Max full path length (`--max-path`)	number	all
Basics	Login name (`--login`)	text	all
Basics	API key (`E6AI_API_KEY` (env))	hidden text	all
Basics	When signed in, re-check posts that needed sign-in (`--no-login-retry`)	checkbox	download, full, run
Advanced	Custom User-Agent (`--user-agent`)	text	all
Advanced	Wait for outages up to (hours) (`--max-outage-hours`)	number	all
Advanced	Verbose log (debug) (`--verbose`)	checkbox	all
Advanced	Log file (`--log-file`)	file + Browse	all
Advanced	Trim the failure log above (MB) (`--failure-log-max-mb`)	number	all
Advanced	Export bundle (`--exports`)	list + free text	full, sync-exports
Advanced	Check the exports now (`full: --refresh-export / sync-exports: --force`)	checkbox	full, sync-exports
Advanced	Download exports (exports mode) (`--download`)	text	exports
Advanced	posts.csv.gz (`--csv`)	file + Browse	seed-from-export
Advanced	tags.csv.gz (`--tags-csv`)	file + Browse	seed-from-export
Advanced	Keep the API backfill (`--keep-backfill`)	checkbox	seed-from-export
Advanced	Compare with this posts export (`--export`)	file + Browse	report
Advanced	Trust the database (skip the disk scan) (`--no-disk-check`)	checkbox	report
Advanced	Fetch the tags export if it is missing / old (`--no-fetch-tags`)	checkbox	retry-failed, report
Advanced	Only these failures (`--reason`)	list + free text	retry-failed
Advanced	Only the posts in this CSV (`--from-csv`)	file + Browse	retry-failed
Advanced	Machine-readable (JSON) output (`--json`)	checkbox	report
Advanced	Rows to show (`--count`)	number	preview
Advanced	Files to list (`--status`)	list	preview
Advanced	Assumed bandwidth (Mbit/s) (`--assume-mbps`)	number	preview

tab	option (CLI flag)	widget	used by
Advanced	Machine-readable (JSON) output (`--json`)	checkbox	preview
Advanced	Only check the first N files (`--limit`)	number	reorganize
Advanced	Fetch (`positional kinds`)	checkboxes	meta
Advanced	Stop after N items (`--max-items`)	number	meta

Omitted on purpose: - `cmd`: chosen with the 'What to do' list (Mode) - `config`: the panel reads/writes e6ai-archiver.ini itself and passes every value explicitly (--no-config) - `no_config`: always passed by the panel, so the command is self-contained - `gui`: the old in-process progress window; the panel is the window now - `gui_autostart`: only for the old window's start panel - `progress_json`: added at launch: the panel's progress channel (not shown in the command preview) - `control_file`: added at launch: how Stop / Pause reach the run (not shown in the command preview) - `apply`: Mode 'Reorganize: move files' adds --apply - `dry_run@preview`: preview never uses the network; --dry-run is accepted there only for symmetry - `help`: argparse help - `version`: argparse --version - `hash_workers`: verify uses one hashing thread per CPU core; command line only (--hash-workers) - `only_new`: verify: 'Verify when finished' checks only unchecked files; command line only (--only-new) - `restart`: verify resumes by default; command line only (--restart starts over)

3f. What changed in 1.3

- **`--order`** `default|popular|favorites|newest|oldest|smallest-first|balanced` from CLI, ini, env, `run.ps1 -Order`, or the window's dropdown. `popular` = score desc, then favs desc; `balanced` = 0.5 x score percentile + 0.3 x favs percentile + 0.2 x small-file percentile. Ties: newest id first.
- **Filters** `--min-score --min-favs --rating --director --tags --max-size-mb --exclude-ext` and **tiers** `--limit-gb N` / `--top N` (the run stops cleanly at the tier end).
- **Exports are the source of truth.** `sync-exports` refreshes the bundle at most once per 20 h, verifies sha256, keeps the previous good copy, merges incrementally (new posts, changed scores/favs/tags, deleted posts; no re-downloads) and re-ranks. The API fills gaps: newer posts, plus fresh per-post JSON at 1 request per 100 files right before downloading them.
- **`preview --order popular --count 20`** and `download --dry-run`: top rows, cumulative GiB, time per tier.
- **Window:** order/filter panel before Start, tier bar, "rank X of N in tier".
- **Fixed:** a 403 during export sync was swallowed as an ordinary export failure; it now stops at once.
- **Live checks for 1.3** (8 requests, UA `e6ai-archiver/1.3 (testing; contact via owner)`, no 403): 1 API `id:` lookup, the export listing, and 6 small exports (pools, directors, tag_aliases, tag_implications, wiki_pages, post_replacements; 4.8 MB, all sha256 ok). posts and tags were the already-downloaded 2026-11-03 copies (not fetched again).

Tiers under `popular` (2026-11-03 exports, offline)

Hours = max(files at 1 request/s, bytes at 50 Mbit/s).

tier	files	GiB	hours
top 1,000	1,000	8.26	0.4

tier	files	GiB	hours
top 10,000	10,000	53.52	2.8
top 50,000	50,000	207.74	13.9
all	162,064	525.48	45.0

Top post: #109132 (score 1173, 2035 favs). First seed of the 193,537-row export: 17 s (peak 96 MB RAM); re-seed 12.8 s; queue rebuild 0.3 to 0.45 s; `preview` about 0.5 s.

Export bundle (2026-11-03 listing)

export	size	used for
posts	46.6 MB	seed + merge (score, favs, size, rating, tags, parent, deleted)
post_versions	114.2 MB	archived as-is
tags	6.4 MB	tag categories (director folders, filters)
wiki_pages	2.2 MB	archived as-is
tag_aliases	1.2 MB	archived as-is
tag_implications	1.0 MB	archived as-is
post_replacements	0.30 MB	archived as-is
bulk_update_requests	0.15 MB	archived as-is
pools	0.08 MB	`pool_posts` (716 pools, 7,506 memberships)
directors	0.01 MB	archived as-is
artists	99 B	empty (last updated 2026-07-01, no checksum)

3g. What changed in 1.2

- **Progress window** (`--gui`, `run_gui.bat`): overall and per-file bars, speed, ETA, counts, free disk, phase, requests, network state, last problem, log; Pause/Resume and Stop; console fallback.
- **Naming:** default `{source_name}_{id}.{ext}` (max 60 characters). e6ai does not store original filenames, so `source_name` comes from the source URLs. Presets: `detailed`, `full`, `short`, `minimal`, `md5`.
- **Folders:** default `{director}/{rating}`, `_no_director` when none, presets, `--multi-director first|multi|copy|hardlink`. The tags export is loaded so export-seeded posts know their directors.
- **Crash cases fixed** (each has a test): null fields, unknown categories, huge/emoji/RTL tag lists, duplicate md5, files/folders deleted mid-run, USB removed, read-only folder, DB corruption, power loss, clock change/sleep, network down for hours, DNS failures, 5xx storms, Retry-After, `.part` / zero-byte leftovers, repeated md5 mismatch, malformed ini, non-ASCII and >260-character paths, GUI closed abruptly, two instances, Ctrl-C while seeding.
- **Unchanged on purpose (until 1.6):** 1 req/s default, 2/s hard cap, descriptive UA, stop on 403.

Default layout, with examples

```
files/namsi/safe/123_p0_193829.png                 source URL ended in .../123_p0.png
files/alpha/questionable/1776022833235_161263.png  source was an x.com status link; 2 directors -> first
files/_no_director/explicit/2.webm                 no director, no source -> just the id
detailed preset: files/namsi/safe/193829_namsi_krystal_fox_safe.png
```

source_name coverage (2026-11-02 export, computed offline)

```
posts.csv.gz (live export, 193,537 rows), source_name classifier:
 media file name in a source URL: 3,019 (1.6%)
 other usable last URL segment (tweet/pixiv ids, channel names): 55,730 (28.8%)
 none usable -> name is just the id: 134,788 (69.6%)  [130,699 = 67.5% have no source at all]
examples: 161263 -> 1776022833235_161263.png ; 4918 -> artwork(1)_4918.png ; 7306 -> 110692564_7306.png ; 1257 ->
1257.png
```

4. Key code snippets for review

1. Politeness: User-Agent, rate limiter, 403 stop, backoff

`e6ai_archiver/client.py` lines 22-33

```
BASE_URL = "https://e6ai.net"
DEFAULT_RATE = 1.0         # API requests per second (user-settable, no hard cap since 1.6)
API_RATE_GUIDE = 2.0       # e621-family API docs ask for at most ~2 requests/s (informational only)
RETRY_STATUSES = {408, 429, 500, 502, 503, 504, 520, 521, 522, 523, 524, 525, 526, 529}
MIN_RATE = 0.25            # 429 slow-down floor (requests per second)
MAX_DELAY = 300.0          # longest single back-off wait
MAX_RETRY_AFTER = 900.0    # longest Retry-After we honor in one go
DEFAULT_MAX_OUTAGE_H = 12.0
# programming/config errors: retrying can't help
_FATAL_REQUEST_ERRORS = (requests.exceptions.InvalidURL, requests.exceptions.InvalidSchema,
                         requests.exceptions.MissingSchema, requests.exceptions.InvalidHeader)
BROWSER_UA_MARKERS = ("mozilla", "chrome", "safari", "applewebkit", "gecko/", "edg/", "opera")
```

What to check: API rate default 1.0 req/s, **no cap since 1.6** (owner's request); `API_RATE_GUIDE` = 2.0 is only used for a log note, because the e621-family API docs ask for about 2 req/s or less. 403 is **not** in `RETRY_STATUSES`. `DEFAULT_MAX_OUTAGE_H` (12 h) is how long an outage is waited out before a resumable stop.

`e6ai_archiver/client.py` lines 48-75

```
class RateLimiter:
    """Limiter for API requests (and for file URLs that are not on the static host)."""

    def __init__(self, rate: float):
        rate = float(rate)
        if not rate > 0:
            raise ValueError("rate must be > 0")
        if rate > API_RATE_GUIDE:  # 1.6: a note, not a clamp
            log.info("API rate %.2f/s is above the ~%g/s the e621-family API docs ask for", rate, API_RATE_GUIDE)
        self.rate = rate
        self.interval = 1.0 / rate
        self._next = 0.0
        self._lock = threading.Lock()

    def wait(self) -> None:
        with self._lock:
            now = time.monotonic()  # monotonic: wall-clock changes / DST can't break pacing
            if now < self._next:
                control.sleep(self._next - now)
                now = time.monotonic()
            self._next = now + self.interval

    def slow_down(self) -> None:
        """After HTTP 429: halve the rate for the rest of the run (floor MIN_RATE)."""
        new = max(MIN_RATE, self.rate / 2)
        if new < self.rate:
            log.warning("server asked us to slow down (429): rate %.2f/s -> %.2f/s for this run", self.rate, new)
            self.rate, self.interval = new, 1.0 / new
```

What to check: 1.6: the limiter covers **API requests only** (static files bypass it, see section 16). Behind a lock, on `time.monotonic()` (clock changes, DST and sleep/resume cannot speed it up). Rates above 2/s are no longer clamped, only noted in the log; 429 halves the rate for the rest of the run.

`e6ai_archiver/client.py` lines 97-106

```
def build_user_agent(username: str | None, override: str | None = None) -> str:
    if override:
        return override.strip()
    if not username or not username.strip():
        raise ValueError(
            "A username is required for the User-Agent. Pass --username, set E6AI_USERNAME, "
            "or put username= in the config file (or pass a full --user-agent)."
        )
    major_minor = ".".join(__version__.split(".")[:2])
    return f"e6ai-archiver/{major_minor} (by {username.strip()} on e6ai)"
```

What to check: No username, no start. Default UA `e6ai-archiver/1.9 (by NAME on e6ai)` (major.minor from `__version__`). (Browser-like UAs are refused in `Client.__init__`.)

`e6ai_archiver/client.py` lines 157-170

```
    def _backoff(self, attempt: int, waited: float, url: str, why: str, retry_after: str | None = None) -> float:
        """Sleep before retry `attempt`; returns the delay. Gives up only after max_outage_hours of
        waiting (a home connection down overnight is waited out, not treated as fatal)."""
        delay = min(MAX_DELAY, 2.0 ** min(attempt, 12)) + random.uniform(0, 1)
        ra = retry_after_seconds(retry_after)
        if ra is not None:
            delay = max(delay, ra)
        if (self.max_retries is not None and attempt > self.max_retries) or waited + delay > self.max_outage_s:
            raise RuntimeError(f"Giving up on {url}: still failing after {attempt - 1} retries over "
                               f"{waited / 3600:.1f} h ({why}). Rerun to resume.")
        log.warning("%s on %s; retry %d in %.0fs (waiting %.0f min so far)", why, url, attempt, delay, waited / 60)
        emit("net", state="waiting", why=why, retry_in=delay, waited=waited)
        control.sleep(delay)
        return delay
```

What to check: Delay 2, 4, 8 ... capped at 300 s + jitter; `Retry-After` honoured (capped at 900 s). It keeps waiting through a long outage (network down for hours) and gives up only after `max_outage_hours` of total waiting, with a resumable error. Waiting uses `control.sleep`, so Stop works during a wait.

`e6ai_archiver/client.py` lines 172-221

```
    def _request(self, url: str, *, params=None, stream=False, accept="application/json") -> requests.Response:
        """One logical GET: rate limited, retried on 429/5xx/network errors, stops on 403."""
        attempt, waited = 0, 0.0
        while True:
            control.checkpoint()  # Pause takes effect before the next request; Stop raises here
            self.limiter.wait()
            self.request_count += 1
            emit("request", count=self.request_count)
            try:
                r = self.s.get(url, params=params, stream=stream, timeout=self.timeout,
                               headers={"Accept": accept}, auth=self._auth_for(url))
            except _FATAL_REQUEST_ERRORS:
                raise
            except requests.RequestException as e:  # DNS failure, refused, reset, timeout, SSL, ...
                attempt += 1
                waited += self._backoff(attempt, waited, url, f"network error: {e.__class__.__name__}")
                continue
            if r.status_code == 403:
                body = "" if stream else r.text[:300]
                final_url = r.url
                r.close()
                raise ForbiddenError(
                    f"HTTP 403 Forbidden for {final_url}. Stopping all requests. "
                    f"Check your User-Agent/credentials and the site's rules. Body: {body!r}"
                )
            if r.status_code == 401:
                r.close()
                who = self.login or "(anonymous)"
                raise AuthError(
                    f"HTTP 401 for {url}: e6ai did not accept the sign-in for login '{who}'. "
                    "Check the login name and API key (the key is not your password). Stopping."
                )
            if r.status_code == 404:
                r.close()
                raise NotFoundError(url)
            if r.status_code < 400:
                if attempt:
                    log.info("connection back after %.0f min", waited / 60)
                    emit("net", state="ok")
                return r
            if r.status_code not in RETRY_STATUSES:
                text = "" if stream else r.text[:300]
                r.close()
                raise RuntimeError(f"HTTP {r.status_code} for {r.url}: {text!r}")
            retry_after = r.headers.get("Retry-After")
            r.close()
            if r.status_code == 429:
                self.limiter.slow_down()
            attempt += 1
            waited += self._backoff(attempt, waited, url, f"HTTP {r.status_code}", retry_after)
```

What to check: Every attempt (retries too) passes `control.checkpoint()` (pause/stop) and the limiter. DNS failures, refused/reset connections and timeouts are retried with backoff instead of crashing. **403 raises `ForbiddenError` immediately, no retry. 401 raises `AuthError` (1.5), no retry.** Other 4xx are hard errors; only 429/5xx and network errors are retried.

2. Crawl cursors and checkpoints

`e6ai_archiver/archiver.py` lines 211-238

```
    def incremental(self) -> None:
        """Fetch posts newer than the newest one already seen.

        page=a<id> returns the posts immediately after <id> (server: id > X ORDER BY id ASC LIMIT n),
        so repeatedly advancing to the max id seen never skips posts.
        """
        newest = self.s.get_state(self.p + "newest")
        if newest is None:
            return
        emit("phase", name="checking for new posts")
        while True:
            budget = self._budget()
            if budget == 0:
                return
            posts = self.c.posts(f"a{newest}", self._page_limit(budget), self.tags)
            posts = sorted((p for p in posts if p["id"] > newest), key=lambda p: p["id"])
            if not posts:
                log.info("incremental: up to date (newest id %s)", newest)
                return
            if budget is not None:
                posts = posts[:budget]
            self._save_page(posts)
            newest = posts[-1]["id"]
            self.s.set_state(self.p + "newest", newest)
            self.s.commit()  # metadata + checkpoint are durable before any file download
            log.info("incremental: saved %d posts, newest now %d", len(posts), newest)
            if not self.metadata_only:
                self.download_pending()
```

What to check: `page=a<id>`: only ids above `newest` are kept, so nothing is skipped or duplicated. Metadata and checkpoint are committed **before** that page's files are downloaded.

3. File download (1.6: in fast.py): size/md5, atomic write, crash edge cases

`e6ai_archiver/archiver.py` lines 441-448

```
    def _local_error(self, e: OSError):
        """Disk full / drive gone / read-only: stop the run (resumable) instead of failing files."""
        from .platform_utils import is_disk_full
        if is_disk_full(e):
            raise e
        why = storage_problem(e, self.s.out)
        if why:
            raise StorageError(why) from e
```

What to check: Disk full, an unplugged USB drive, a read-only folder: these stop the run (exit 4 / 6, resumable) instead of marking thousands of files `failed`.

`e6ai_archiver/fast.py` lines 414-447

```
    def _fetch(self, j: Job, idx: int) -> Result:
        a = self.a
        out = a.s.out
        try:
            if j.dest.exists():
                if j.dest.stat().st_size == 0:
                    safe_unlink(j.dest)
                else:
                    ok, got = self._matches(j.dest, j.md5, j.size)
                    if ok:
                        return self._place_extras(Result(j, "ok", stats={"already_present": 1}))
                    from .archiver import set_aside
                    kept = set_aside(j.dest, got or _md5_of(j.dest))
                    log.warning("%s has different content; kept it as %s and downloading the expected file",
                                j.dest.name, kept.name)
            legacy = abspath(out, legacy_relpath(j.md5, j.ext))
            dup = abspath(out, j.dup_rel) if j.dup_rel else None
            for src, how in ((legacy, "move"), (dup, "link")):
                if not src or src == j.dest or not src.exists() or not self._matches(src, j.md5, j.size)[0]:
                    continue
                j.dest.parent.mkdir(parents=True, exist_ok=True)
                if how == "move":
                    safe_replace(src, j.dest)
                    st = {"already_present": 1}
                else:
                    link_or_copy(src, j.dest, "hardlink")
                    st = {"reused": 1}
                return self._place_extras(Result(j, "ok", stats=st))
            self._wait_for_space(out, j.size or 0)
            j.dest.parent.mkdir(parents=True, exist_ok=True)
        except OSError as e:
            a._local_error(e)
            raise
        tmp = j.dest.with_name(j.dest.name + ".part")
```

What to check: Runs on a worker thread and never touches SQLite (the job carries everything it needs). An existing right-size file (md5 with `--check-md5`) is kept without a request; a zero-byte leftover is deleted; a file with different content is **set aside, not deleted**. A 1.1-layout file is moved; a duplicate upload is hard-linked. Free space is checked before each file.

`e6ai_archiver/fast.py` lines 451-477

```
            h = hashlib.md5() if self.check_md5 else None
            n = 0
            serial = self._api_lock if self.fetcher._is_api_host(j.url) else contextlib.nullcontext()
            try:
                with serial:
                    if self.halt.is_set():
                        raise _Halt()
                    try:
                        r = self.fetcher.get(j.url)
                    except (ForbiddenError, AuthError):
                        self.halt.set()  # before the lock is released: no request may follow
                        raise
                    try:
                        with open(tmp, "wb") as f:
                            for chunk in r.iter_content(CHUNK):
                                f.write(chunk)
                                if h is not None:
                                    h.update(chunk)
                                n += len(chunk)
                                with self._ilock:
                                    if idx in self._inflight:
                                        self._inflight[idx][0] = n
                                control.check_stop()
                                if self.halt.is_set():
                                    raise _Halt()
                    finally:
                        r.close()
```

What to check: Streams to `.part` in 1 MiB chunks. File URLs on the API host itself (legacy/tests) are serialised through `_api_lock` and the API limiter; a 403/401 there sets `halt` **before** the lock is released, so no other request follows. Stop/halt is checked between chunks.

`e6ai_archiver/fast.py` lines 485-552

```
            except StaticBlocked as e:
                safe_unlink(tmp)
                status = e.status
                if status == 403:
                    log.warning("file refused (HTTP 403): %s", j.url)
                    return Result(j, "failed", stats={"failed": 1}, forbidden=True, reason="http_403", tries=tries)
                throttled += 1
                if throttled > THROTTLE_RETRIES:
                    log.warning("giving up on %s for now: HTTP %s %d times", j.url, status, throttled)
                    return Result(j, "failed", stats={"failed": 1}, throttled=True, reason=f"http_{status}",
tries=tries)
                wait = e.retry_after if e.retry_after is not None else min(MAX_WORKER_BACKOFF, 5.0 * 2 ** throttled)
                log.warning("HTTP %s for %s; this worker waits %.0fs (retry %d/%d)", status, j.dest.name, wait,
                            throttled, THROTTLE_RETRIES)
                self._sleep(wait)
                continue
            except FileNotFoundError as e:
                safe_unlink(tmp)
                a._local_error(e)
                vanished += 1
                if vanished > 1:
                    raise
                j.dest.parent.mkdir(parents=True, exist_ok=True)
                continue
            except (requests.RequestException, OSError) as e:
                safe_unlink(tmp)
                if isinstance(e, OSError) and not isinstance(e, requests.RequestException):
                    a._local_error(e)
                    raise
                stream_errors += 1
                if stream_errors > STREAM_RETRIES:
                    log.error("giving up on %s after %d broken transfers: %s", j.url, stream_errors, e)
                    return Result(j, "failed", stats={"failed": 1}, reason="network_error", tries=tries)
                self._sleep(2 ** stream_errors)
                continue
            except BaseException:
                safe_unlink(tmp)
                raise
            # complete body received: size check always, md5 when enabled
            if h is not None:
                good = h.hexdigest() == j.md5
            elif j.size and n != j.size:
                good = _md5_of(tmp) == j.md5   # size in the metadata may be stale: md5 decides
            else:
                good = True
            if good:
                try:
                    safe_replace(tmp, j.dest)  # atomic: the final path is absent or complete
                except FileNotFoundError as e:
                    # folder deleted/renamed between the last write and the rename: re-create it and refetch once
                    safe_unlink(tmp)
                    a._local_error(e)
                    vanished += 1
                    if vanished > 1:
                        raise
                    j.dest.parent.mkdir(parents=True, exist_ok=True)
                    continue
                except OSError as e:
                    safe_unlink(tmp)
                    a._local_error(e)
                    raise
                return self._place_extras(Result(j, "ok", n, stats={"downloaded": 1, "bytes": n}))
            safe_unlink(tmp)
            mismatches += 1
            log.warning("%s for %s (attempt %d, got %d bytes, expected %s)",
                        "md5 mismatch" if h is not None else "wrong size", j.url, mismatches, n, j.size)
            if mismatches >= 2:
                return Result(j, "md5_mismatch", stats={"md5_mismatch": 1},
                              reason="md5_mismatch" if h is not None else "wrong_size", tries=tries)
```

What to check: Static 403: that file `failed` + worker back-off (the run-level counter is in `_apply`). 429/503: wait (`Retry-After`), 3 more tries, then `failed`. Folder deleted mid-run (also between the

last write and the rename): recreated, fetched once more. Broken transfer: 3 retries, then `failed`. Local disk error: run stops. The rename to the final name happens only after the size (and md5 when on) check; two bad copies: `md5_mismatch`.

4. Folder template and director grouping

`e6ai_archiver/naming.py` lines 33-50

```
NAME_PRESETS = {
    "default": "{source_name}_{id}.{ext}",
    "detailed": "{id}_{director}_{character}_{species}_{rating}.{ext}",
    "full": "{id}_{director}_{franchise}_{character}_{species}_{rating}_{date}.{ext}",
    "short": "{id}_{director}.{ext}",
    "minimal": "{id}.{ext}",
    "md5": "{md5}.{ext}",
}
SHORT_NAME_MAX = 60     # default cap for templates using {source_name}
LONG_NAME_MAX = 120     # default cap for metadata templates
MEDIA_EXTS = {"jpg", "jpeg", "jpe", "jfif", "png", "apng", "gif", "webp", "avif", "jxl", "heic", "heif",
              "bmp", "tif", "tiff", "psd", "webm", "mp4", "m4v", "mov", "mkv", "avi", "wmv", "flv", "swf"}
FOLDER_PRESETS = {
    "director-rating": "{director}/{rating}", "director": "{director}", "director-year": "{director}/{year}",
    "year-month": "{year}/{month}", "franchise": "{franchise}/{director}", "species": "{species}/{director}",
    "rating": "{rating}/{director}", "ext": "{ext}/{director}", "flat": "", "md5": "{md5_ab}/{md5_cd}",
}
MULTI_DIRECTOR = ("first", "multi", "copy", "hardlink")
```

What to check: Default name `{source_name}_{id}.{ext}`; default folders `{director}/{rating}`. Director-category tags that are really warnings are excluded from director folders.

`e6ai_archiver/naming.py` lines 260-289

```
    def __post_init__(self):
        self.name_template = NAME_PRESETS.get(self.name_template, self.name_template)
        self.folder_template = FOLDER_PRESETS.get(self.folder_template, self.folder_template)
        if self.multi_director not in MULTI_DIRECTOR:
            raise ValueError(f"multi_director must be one of {', '.join(MULTI_DIRECTOR)}")
        if self.rating_style not in ("word", "letter"):
            raise ValueError("rating_style must be 'word' or 'letter'")
        if not self.name_max:
            self.name_max = SHORT_NAME_MAX if "{source_name}" in self.name_template else LONG_NAME_MAX
        if not 1 <= self.max_items <= 50 or not 30 <= self.name_max <= 200 or self.max_path < 100:
            raise ValueError("max_items must be 1-50, name_max 30-200, max_path >= 100")
        if _ILLEGAL.search(self.sep) or _ILLEGAL.search(self.empty):
            raise ValueError("multi_sep / empty placeholder may not contain <>:\"/\\|?* or control characters")
        t = self.name_template.strip()
        if "/" in t or "\\" in t:
            raise ValueError("name_template may not contain / or \\ (use folder_template for folders)")
        if t.endswith(".{ext}"):
            t = t[:-len(".{ext}")]
        if "{ext}" in t:
            raise ValueError("name_template: put {ext} only at the end, as .{ext}")
        self._name = parse_template(t, "name_template")
        if not any(isinstance(p, tuple) and p[0] in ("id", "md5") for p in self._name):
            self._name += ["_", ("id", None)]  # uniqueness guarantee
        self._id_first = isinstance(self._name[0], tuple) and self._name[0][0] in ("id", "md5")
        levels = [x for x in re.split(r"[/\\]", self.folder_template.strip()) if x.strip()]
        if len(levels) > 6:
            raise ValueError("folder_template: at most 6 folder levels")
        self._folders = [parse_template(x, "folder_template") for x in levels]
        self._uses_cats = any(isinstance(p, tuple) and p[0] in MULTI_FIELDS
                              for p in self._name + [q for lv in self._folders for q in lv])
```

What to check: Templates are validated up front (bad template = exit 2, before any request). If a name template has neither `{id}` nor `{md5}`, `_{id}` is appended, so two posts can never collide. At most 6 folder levels.

`e6ai_archiver/naming.py` lines 370-380

```
    def plan(self, info: PostInfo, out_len: int = 0) -> list[str]:
        """Relative paths ('/'-separated, under files/) for a post: [primary, *extra copies].
        out_len = length of the absolute archive dir, used to keep full paths under max_path."""
        dirs = self.directors(info)
        if self.multi_director == "multi" or not dirs:
            groups = [None]
        elif self.multi_director == "first":
            groups = [dirs[:1]]
        else:  # copy / hardlink: one entry per director
            groups = [[d] for d in dirs]
        return [self._fit(info, g, out_len) for g in groups]
```

What to check: `first`: folder of the first director; `multi`: one folder `a,b`; `copy`/`hardlink`: one path per director. No director: `_no_director`.

`e6ai_archiver/naming.py` lines 382-389

```
    def _fit(self, info: PostInfo, director, out_len: int) -> str:
        """Shorten (name first, then folder names) until the absolute path fits max_path."""
        for name_room, folder_room in ((None, None), (60, None), (40, 32), (24, 16), (24, 8)):
            parts = [FILES_DIR, *self.folders(info, director, folder_room), self.file_name(info, name_room)]
            rel = "/".join(parts)
            if out_len + 1 + _u16(rel) <= self.max_path:
                return rel
        return rel  # still long (very deep archive dir): long-path APIs handle it; logged by caller
```

What to check: Shortens the name first, then folder names, until the full path (archive folder included) fits `max_path` (240).

5. Sanitizer (Windows-safe names)

`e6ai_archiver/naming.py` lines 68-78

```
def _u16(s: str) -> int:
    """Length as NTFS counts it (UTF-16 code units; emoji count 2)."""
    return len(s.encode("utf-16-le")) // 2


def truncate(s: str, limit: int) -> str:
    while s and _u16(s) > limit:
        s = s[:-1]
    while s and (unicodedata.combining(s[-1]) or s[-1] in "\u200d"):  # no dangling accent / ZWJ
        s = s[:-1]
    return s
```

What to check: Lengths are counted in UTF-16 units, as NTFS does (an emoji counts 2). Truncation never leaves a dangling combining accent or zero-width joiner.

`e6ai_archiver/naming.py` lines 81-92

```
def sanitize_component(s: str, limit: int = 120, placeholder: str = "unknown") -> str:
    """One Windows-safe path component: NFC, no illegal/control/bidi chars, no trailing dot or
    space, not a reserved device name (CON, NUL, COM1, LPT¹, ...), at most `limit` UTF-16 units."""
    s = unicodedata.normalize("NFC", str(s))
    s = _BIDI.sub("", s)
    s = _ILLEGAL.sub("_", s)
    s = truncate(s.strip(), limit).rstrip(" .")
    if not s or set(s) <= {"."}:
        s = placeholder
    if s.split(".")[0].rstrip(" ").lower() in _RESERVED:
        s = truncate("_" + s, limit)
    return s
```

What to check: NFC normalisation; bidi/direction-override characters removed (RTL tags can't reorder the displayed name); `<>:"/\|?*` and control characters become `_`; no trailing dot/space; reserved device names (CON, NUL, COM1, LPT¹ ...) get a `_` prefix; empty becomes a placeholder.

6. source_name (shorter names from source URLs)

`e6ai_archiver/naming.py` lines 95-131

```
def _segment(url: str) -> str:
    try:
        path = urlsplit(url.strip()).path     # drops ?query and #fragment
    except ValueError:
        return ""
    return unquote(path.rstrip("/").rsplit("/", 1)[-1]).strip()


def _clean_stem(stem: str) -> str:
    stem = _ILLEGAL.sub("_", re.sub(r"\s+", "_", stem))
    return stem.strip("._- ") if re.search(r"\w", stem) else ""


def source_name(sources) -> str | None:
    """Best-effort original filename stem from a post's sources (list, or newline-separated string).
    1st choice: the first source whose last path segment is a media file name ("abc.png" -> "abc").
    2nd choice: the first source with any usable last path segment (e.g. ".../status/1234" -> "1234").
    Returns None when nothing usable is found (the caller then uses the post id)."""
    if isinstance(sources, str):
        sources = sources.splitlines()
    urls = [u for u in (sources or []) if isinstance(u, str) and u.strip()]
    for u in urls:
        seg = _segment(u)
        if "." in seg:
            stem, ext = seg.rsplit(".", 1)
            if ext.lower() in MEDIA_EXTS and _clean_stem(stem):
                return _clean_stem(stem)
    for u in urls:
        seg = _segment(u)
        if "//" not in u and "/" not in seg and not u.lower().startswith(("http", "www.")):
            continue  # plain text, not a URL
        if urlsplit(u.strip()).path.strip("/") == "":
            continue  # bare domain, no path
        stem = seg.rsplit(".", 1)[0] if re.fullmatch(r".+\.[A-Za-z0-9]{1,5}", seg) else seg
        if _clean_stem(stem):
            return _clean_stem(stem)
    return None
```

What to check: e6ai keeps no original filename, so this is a best-effort stem: a media filename in a source URL first, else the last usable URL path part, else `None` (the name is then just the id). Query strings and fragments are ignored; illegal characters are replaced.

`e6ai_archiver/naming.py` lines 347-363

```
    def file_name(self, info: PostInfo, room: int | None = None) -> str:
        ext = info.ext or "bin"
        room = min(room or self.name_max, self.name_max) - len(ext) - 1
        stem = sanitize_component(self._render(self._name, info, False, None).strip("_"), 10_000, str(info.id))
        if _u16(stem) > room and any(p == ("source_name", None) for p in self._name):
            # shorten the source name first so id and the other fields survive
            rest = _u16(sanitize_component(self._render(self._name, info, False, None, 0), 10_000, ""))
            stem = self._render(self._name, info, False, None, max(0, room - rest)).strip("_")
            stem = sanitize_component(stem, 10_000, str(info.id))
        if _u16(stem) > room:
            if self._id_first:
                stem = truncate(stem, room).rstrip(" ._")
            else:  # keep the id so the name stays unique after cutting
                tail = f"_{info.id}"
                stem = truncate(stem, max(1, room - len(tail))).rstrip(" ._") + tail
            stem = sanitize_component(stem, room, str(info.id))
        return f"{stem}.{ext}"
```

What to check: Name cap (60 chars for the default): `source_name` is shortened first so the id always survives; if still too long, the id is kept at the end.

7. Reorganize safety

`e6ai_archiver/layout.py` lines 38-46

```
def _free_target(out: Path, rel: str) -> str:
    """rel, or rel with __2, __3 ... before the extension if something else is already there."""
    folder, _, name = rel.rpartition("/")
    stem, dot, ext = name.rpartition(".")
    for n in range(2, 1000):
        cand = f"{folder}/{stem}__{n}{dot}{ext}"
        if not abspath(out, cand).exists():
            return cand
    raise RuntimeError(f"too many name collisions for {rel}")
```

What to check: A different file already at the target is never overwritten: the moved file gets `name__2.ext`.

`e6ai_archiver/layout.py` lines 49-53

```
def _is_collision_variant(cur: str, target: str) -> bool:
    """True if cur is target with __N before the extension (a name _free_target handed out earlier)."""
    folder, _, name = target.rpartition("/")
    stem, dot, ext = name.rpartition(".")
    return re.fullmatch(re.escape(f"{folder}/{stem}__") + r"[0-9]{1,3}" + re.escape(f"{dot}{ext}"), cur) is not None
```

What to check: A file already sitting at such a `__N` name counts as in place, so rerunning reorganize is idempotent.

`e6ai_archiver/layout.py` lines 113-144

```
def _one(store, pid, md5, cur, target, apply, res) -> tuple[str, str]:
    out = store.out
    if cur == target or _is_collision_variant(cur, target):
        res["unchanged"] += 1  # already at target (or at target__N because target is taken by another file)
        return "unchanged", cur
    src, dst = abspath(out, cur), abspath(out, target)
    if not src.exists():
        if dst.exists() and md5_of(dst) == md5:  # moved earlier, DB update was interrupted
            res["db_only"] += 1
            if apply:
                store.move_file_record(pid, md5, target)
            return "db_only", target
        res["missing"] += 1
        if apply:  # the next download run fetches it again (to the new layout)
            store.db.execute("UPDATE posts SET file_status='pending', file_path=NULL WHERE id=?", (pid,))
        return "missing", ""
    if dst.exists():
        if md5_of(dst) == md5:  # same content already there: point the DB at it, keep the old file
            res["db_only"] += 1
            if apply:
                store.move_file_record(pid, md5, target)
            return "db_only", target
        target = _free_target(out, target)
        dst = abspath(out, target)
        res["collision_renamed"] += 1
    res["move"] += 1
    if apply:
        dst.parent.mkdir(parents=True, exist_ok=True)
        safe_replace(src, dst)
        store.move_file_record(pid, md5, target)
        _prune_empty_dirs(out, src.parent)
    return "move", target
```

What to check: Dry run unless `--apply`. Only moves, never deletes. If the source is gone but the target has the right md5 (an interrupted earlier run), only the DB is updated; a truly missing file goes back to `pending`. Each move is committed at once and logged in an audit CSV, so stopping mid-way is safe.

8. GUI threading

`e6ai_archiver/gui.py` lines 1-15

```
"""Optional progress window (stdlib tkinter). Start with --gui, gui = yes in the ini, or run_gui.bat.

Threading model (the important part for reviewers):
  * The archiver runs in ONE worker thread, exactly the same code as the console mode.
  * The worker only calls events.emit(...) and logging; the GUI's sink and log handler just do
    queue.put_nowait(). The worker never touches a Tk widget (Tk is not thread-safe).
  * The Tk main thread drains the queue every 200 ms (root.after), folds events into GuiModel
    (plain Python, unit-tested without a display) and redraws from model.snapshot().
  * Pause / Stop only set threading.Events in events.control; the worker honours them before the
    next request (pause) or between 1 MiB chunks / during waits (stop). Stop is a normal,
    checkpointed exit: the partial .part file is removed, the DB is committed, the lock released.
  * Closing the window = Stop. Closing it again while still stopping hides the window; the
    process keeps finishing in the console. If Tk itself fails, the worker carries on and the run
    continues in the console: a display problem can never break or corrupt a download.
"""
```

What to check: The design contract: one worker thread runs the unchanged archiver; Tk is touched only on the main thread.

`e6ai_archiver/events.py` lines 23-47

```
class StopRequested(KeyboardInterrupt):
    """Raised in the worker after the user pressed Stop / closed the GUI. Subclasses
    KeyboardInterrupt so every existing clean-up path (delete .part, commit, release lock) runs."""


def add_sink(fn) -> None:
    with _lock:
        _sinks.append(fn)


def remove_sink(fn) -> None:
    with _lock:
        if fn in _sinks:
            _sinks.remove(fn)


def emit(kind: str, **data) -> None:
    if not _sinks:
        return
    for fn in list(_sinks):
        try:
            fn(kind, data)
        except Exception as e:  # never let a display problem reach the download code
            log.warning("progress display error (%s); display detached", e)
            remove_sink(fn)
```

What to check: `StopRequested` subclasses `KeyboardInterrupt`, so Stop runs exactly the Ctrl-C clean-up (remove `.part`, commit, release lock). A failing display sink is detached; it can never break the download.

`e6ai_archiver/events.py` lines 92-118

```
    # --- called from the worker ------------------------------------------
    def check_stop(self) -> None:
        if self._stop.is_set():
            raise StopRequested("stop requested")

    def checkpoint(self) -> None:
        """Before each request: honour stop, and block while paused."""
        self.check_stop()
        if not self._run.is_set():
            emit("phase", name="paused")
            log.info("paused")
            while not self._run.wait(0.5):
                pass
            self.check_stop()
            log.info("resumed")
            emit("phase", name="resumed")

    def sleep(self, seconds: float) -> None:
        """time.sleep() in 0.5 s slices that stops early (StopRequested) when a stop is requested.
        Counts slices instead of reading the clock, so clock jumps and sleep/resume can't extend it."""
        left = max(0.0, float(seconds))
        while left > 0:
            self.check_stop()
            step = min(left, 0.5)
            time.sleep(step)
            left -= step
        self.check_stop()
```

What to check: Pause/Stop are plain `threading.Event`s. Pause takes effect before the next request. `sleep` counts 0.5 s slices, so a clock jump can't extend a wait and Stop interrupts it.

`e6ai_archiver/gui.py` lines 701-740

```
def run_with_gui(fn, args=None, close_when_done: float | None = None, on_ready=None):
    """Run fn() (returns an exit code) in a worker thread with the progress window on this thread.
    Returns fn's exit code, or None if no window could be created (caller then runs fn itself).
    With a start panel (see wants_start_panel) the worker starts only when Start is pressed;
    closing the window before that returns 130 (nothing was done)."""
    root = _make_root()
    if root is None:
        return None
    q: queue.Queue = queue.Queue(maxsize=50000)

    def sink(kind, data):
        try:
            q.put_nowait((kind, data))
        except queue.Full:  # GUI not draining (window frozen/closed): drop display updates, never block
            pass

    worker = threading.Thread(target=lambda: work(), name="archiver-worker")
    panel = wants_start_panel(args)
    try:
        win = ArchiverWindow(root, q, GuiModel(), close_when_done=close_when_done,
                             start_args=args if panel else None, on_start=lambda: worker.start())
        if on_ready is not None:  # tests / screenshots: called on the Tk thread with the window
            root.after(0, on_ready, win)
    except Exception as e:
        log.warning("GUI unavailable (%s); using the console", e)
        try:
            root.destroy()
        except Exception:
            pass
        return None
    handler = QueueLogHandler(q)
    add_sink(sink)
    log.addHandler(handler)
    result = {"code": 1}

    def work():
        try:
            result["code"] = fn()
        except BaseException as e:  # execute() handles everything expected; this is the last net
            log.exception("worker crashed: %s", e)
```

What to check: The sink never blocks (queue full: display updates are dropped, the download goes on). If no window can be made, `None` is returned and the caller runs in the console. Worker crashes are caught and logged; the window always gets a `done` event.

`e6ai_archiver/gui.py` lines 747-779

```
    old_int = None
    if threading.current_thread() is threading.main_thread():
        try:  # Ctrl-C in the console window = Stop button (clean stop), not a crash in Tk
            old_int = signal.signal(signal.SIGINT, lambda *_: control.stop())
        except (ValueError, OSError):
            old_int = None
    if not panel:
        worker.start()
    try:
        win.run()
    except KeyboardInterrupt:
        control.stop()
    except Exception as e:
        log.error("progress window failed (%s); the run continues in the console", e)
    finally:
        remove_sink(sink)
        log.removeHandler(handler)
    if worker.ident is None:  # window closed before Start
        log.info("closed before Start; nothing was downloaded")
        result["code"] = 130
    if worker.is_alive():
        print("window closed; finishing in the console (Ctrl-C again = stop)...", file=sys.stderr)
    while worker.is_alive():
        try:
            worker.join(0.5)
        except KeyboardInterrupt:
            control.stop()
    if old_int is not None:
        try:
            signal.signal(signal.SIGINT, old_int)
        except (ValueError, OSError):
            pass
    return result["code"]
```

What to check: Ctrl-C in the console = Stop button. After the window closes, the worker is joined, so the process never exits mid-write.

`e6ai_archiver/gui.py` lines 628-643

```
    def poll(self):
        try:
            for _ in range(MAX_EVENTS_PER_POLL):
                try:
                    kind, data = self.q.get_nowait()
                except queue.Empty:
                    break
                self.m.apply(kind, data)
            self.refresh()
        except Exception as e:  # never stop polling because of one bad event
            log.debug("gui poll error: %s", e)
        finally:
            try:
                self.root.after(POLL_MS, self.poll)
            except Exception:
                pass
```

What to check: Runs on the Tk thread every 200 ms, at most 5000 events per tick, and always reschedules itself even if one event is bad.

9. Crash handling: storage, database, exit codes

`e6ai_archiver/platform_utils.py` lines 211-247

```
def storage_problem(e: BaseException, out: Path) -> str | None:
    """Explain an OSError that means the archive location is unusable, else None."""
    import errno
    if not isinstance(e, OSError) or is_disk_full(e):
        return None
    out = Path(out)
    try:
        gone = not out.exists()
    except OSError:
        gone = True
    if gone:
        return f"the archive folder {out} is no longer reachable (drive unplugged or share disconnected?)"
    code = errno.errorcode.get(e.errno or -1, "")
    if code in ("EROFS",) or getattr(e, "winerror", None) == 19:
        return f"the drive holding {out} is read-only / write-protected"
    if isinstance(e, PermissionError):
        return f"no permission to write in {out} (read-only folder? check its properties/permissions)"
    if code in _GONE_ERRNO or getattr(e, "winerror", None) in _GONE_WINERROR:
        return f"I/O error on the drive holding {out}: {e}"
    return None


def check_writable(out: Path) -> None:
    """Startup preflight: create, write, fsync and delete a probe file in the archive folder."""
    out = Path(out)
    try:
        out.mkdir(parents=True, exist_ok=True)
        probe = out / f".write-test-{os.getpid()}"
        with open(probe, "wb") as f:
            f.write(b"ok")
            f.flush()
            os.fsync(f.fileno())
        os.unlink(probe)
    except OSError as e:
        if is_disk_full(e):
            raise DiskFullError(f"The drive holding {out} is full.") from e
        raise StorageError(storage_problem(e, out) or f"cannot write to {out}: {e}") from e
```

What to check: Recognises an unplugged drive, write-protect and permission errors (POSIX errno and Windows error codes). `check_writable` probes the folder before anything starts.

`e6ai_archiver/store.py` lines 196-228

```
def fix_jsonl_tail(path: Path) -> None:
    """After a power loss the last JSONL line may be cut off; end it so new lines stay parseable."""
    try:
        with open(path, "rb+") as f:
            f.seek(0, os.SEEK_END)
            if f.tell() == 0:
                return
            f.seek(-1, os.SEEK_END)
            if f.read(1) != b"\n":
                f.write(b"\n")
                log.warning("%s did not end with a newline (interrupted write?); repaired", path.name)
    except FileNotFoundError:
        pass


def iter_jsonl(path: Path):
    """Yield dicts from a JSONL file, skipping damaged lines."""
    bad = 0
    try:
        f = open(path, encoding="utf-8", errors="replace")
    except FileNotFoundError:
        return
    with f:
        for line in f:
            try:
                obj = json.loads(line)
            except ValueError:
                bad += line.strip() != ""
                continue
            if isinstance(obj, dict):
                yield obj
    if bad:
        log.warning("%s: skipped %d damaged lines", path.name, bad)
```

What to check: After a power loss the last JSONL line may be half-written; it is terminated so later lines stay readable, and damaged lines are skipped on reading.

`e6ai_archiver/store.py` lines 268-366

```
    def _open(self) -> None:
        # timeout: wait (not fail) if the DB is briefly locked, e.g. by a DB browser or backup tool
        existed = self.db_path.is_file() and self.db_path.stat().st_size > 0
        self.db = sqlite3.connect(str(self.db_path), timeout=60)
        try:
            self.db.execute("PRAGMA journal_mode=WAL")      # readers never block the writer
            self.db.execute("PRAGMA synchronous=NORMAL")    # durable at checkpoints, fast commits in WAL
            self.db.execute("PRAGMA temp_store=MEMORY")
            # 2.0: a database from an older version is upgraded in place, in one transaction, after a
            # backup copy; one from a NEWER version is refused untouched (it may hold things we'd lose).
            found_v, written_by = _db_version(self.db) if existed else (None, None)
            if found_v is not None and found_v > SCHEMA_VERSION:
                raise DatabaseTooNewError(self.db_path, found_v, written_by)
            plan = _pending_changes(self.db, found_v) if existed else []
            if existed and self.get_state_raw("open") == "true":  # last run didn't close cleanly
                log.info("previous run did not shut down cleanly; checking database integrity")
                res = self.db.execute("PRAGMA quick_check").fetchone()[0]
                if res != "ok":
                    raise sqlite3.DatabaseError(f"database disk image is malformed (quick_check: {res})")
            self.backup_path = None
            if existed and (plan or written_by != __version__):
                self.backup_path = self._backup_before_upgrade(written_by, required=bool(plan))
            if plan:
                log.info("upgrading the database from %s to %s: %s", written_by or "an older version",
                         __version__, "; ".join(plan))
            self.db.execute("BEGIN IMMEDIATE")
            try:
                for stmt in _schema_statements():
                    self.db.execute(stmt)
                self._migrate()
                if existed and written_by != __version__:
                    hist = self.get_state("upgrade_history", []) or []
                    hist.append({"from": written_by or "<=1.9.1", "to": __version__, "at": time.time(),
                                 "changes": plan, "backup": str(self.backup_path) if self.backup_path else None})
                    self.set_state("upgrade_history", hist[-20:])
                self.set_state("app_version", __version__)
                self.set_state("open", True)
                self.db.commit()
            except BaseException:
                self.db.rollback()  # all or nothing: the old database stays exactly as it was
                raise
        except BaseException:
            self.db.close()
            raise

    def get_state_raw(self, key: str):
        try:
            row = self.db.execute("SELECT value FROM state WHERE key=?", (key,)).fetchone()
        except sqlite3.OperationalError:  # no state table yet
            return None
        return row[0] if row else None

    def _backup_before_upgrade(self, written_by, required: bool):
        """Copy the database (consistent, WAL included) to <out>/db-backups/ before it is changed.
        required=True (schema changes pending): no backup -> no upgrade (RuntimeError, nothing touched)."""
        ts = time.strftime("%Y%m%d-%H%M%S")
        dest_dir = self.out / "db-backups"
        dest = dest_dir / f"{DB_NAME}.before-{__version__}-from-{written_by or 'pre-2.0'}-{ts}"
        size = sum((self.out / (DB_NAME + x)).stat().st_size for x in ("", "-wal")
                   if (self.out / (DB_NAME + x)).exists())
        log.info("backing up the database (%.0f MiB) before opening it with version %s ...", size / 2**20,
                 __version__)
        try:
            dest_dir.mkdir(exist_ok=True)
            from .platform_utils import free_bytes
            if free_bytes(dest_dir) < size + 64 * 1024 * 1024:
                raise OSError(f"not enough free space for a {size / 2**20:.0f} MiB copy")
            tmp = dest.with_name(dest.name + ".tmp")
            dst = sqlite3.connect(str(tmp))
            try:
                self.db.backup(dst)
            finally:
                dst.close()
            safe_replace(tmp, dest)
        except (OSError, sqlite3.Error) as e:
```

```
            if required:
                raise RuntimeError(f"could not back up the database before upgrading it ({e}); nothing was "
                                   f"changed. Free some space on the archive drive and start again.") from None
            log.warning("could not back up the database (%s); continuing (no schema change is needed)", e)
            return None
        log.info("database backed up before the upgrade to %s: %s", __version__, dest)
        return dest

    # --- corruption recovery ----------------------------------------
    def recover(self, reason: str) -> None:
        """Back up the damaged DB (never deleted) and rebuild it from the JSONL journals."""
        try:
            self.db.close()
        except Exception:
            pass
        ts = time.strftime("%Y%m%d-%H%M%S")
        log.error("database is damaged (%s). Backing it up as %s.corrupt-%s and rebuilding from the "
                  "JSONL journals; no data files are touched.", reason, DB_NAME, ts)
        for suffix in ("", "-wal", "-shm"):
            p = self.out / (DB_NAME + suffix)
            if p.exists():
                safe_replace(p, self.out / f"{DB_NAME}{suffix}.corrupt-{ts}")
        self._open()
        self.recovered = self.rebuild_from_journals()
```

What to check: WAL, 60 s busy timeout. If the previous run didn't close cleanly, `quick_check` runs; a corrupt DB is backed up (never deleted) and rebuilt from the JSONL journals and the export (no re-download; files are re-hashed before being trusted).

`e6ai_archiver/store.py` lines 475-486

```
    def commit(self) -> None:
        """Durable commit: JSONL journals are flushed+fsynced first, then SQLite commits."""
        if self._jsonl_dirty:
            for f in (self.jsonl, self.files_jsonl):
                f.flush()
                os.fsync(f.fileno())
            self._jsonl_dirty = False
        for f in self.meta_jsonl.values():
            f.flush()
        self.db.commit()
        self._last_commit = time.monotonic()
        self._uncommitted = 0
```

What to check: JSONL journals are fsynced **before** the SQLite commit, so the journal is never behind the DB.

`e6ai_archiver/cli.py` lines 863-905

```
        except ForbiddenError as e:
            log.error("%s", e)
            log.error("STOPPED on 403. Do not retry with a different/browser User-Agent; "
                      "fix the cause (UA format, username, API key, or ask site staff).")
            return EXIT_403
        except StopRequested:
            log.warning("Stopped. Progress is checkpointed; run again to resume.")
            return EXIT_STOPPED
        except KeyboardInterrupt:
            log.warning("Interrupted. Progress is checkpointed; rerun the same command to resume.")
            return EXIT_STOPPED
        except DiskFullError as e:
            log.error("%s", e)
            return EXIT_DISK_FULL
        except StorageError as e:
            log.error("Storage problem: %s. Nothing was lost; reconnect the drive / fix permissions and "
                      "rerun to resume.", e)
            return EXIT_STORAGE
        except sqlite3.Error as e:
            return _db_error(e, a)
        except OSError as e:
            if is_disk_full(e):
                log.error("Disk full: %s. Free space and rerun to resume.", e)
                return EXIT_DISK_FULL
            why = storage_problem(e, Path(a.out))
            if why:
                log.error("Storage problem: %s. Rerun to resume once fixed.", why)
                return EXIT_STORAGE
            log.error("OS error: %s", e)
            return EXIT_ERROR
        except ValueError as e:
            log.error("%s", e)
            return EXIT_USAGE
        except DatabaseTooNewError as e:
            log.error("%s", e)
            return EXIT_DB_NEWER
        except RuntimeError as e:
            msg = str(e)
            if msg.startswith("Another e6ai-archiver"):
                log.error("%s", msg)
                return EXIT_LOCKED
            log.error("%s (progress is checkpointed; rerun to resume)", msg)
            return EXIT_ERROR
```

What to check: Exit codes: 0 ok, 1 other (resumable), 2 config, **3 HTTP 403 (stop)**, 4 disk full, 5 another instance / DB locked, 6 drive gone or read-only, 7 database damaged, 130 stopped. Every expected failure is a message, not a traceback.

`e6ai_archiver/cli.py` lines 908-921

```
def _db_error(e: sqlite3.Error, a) -> int:
    m = str(e).lower()
    if "locked" in m or "busy" in m:
        log.error("Database is locked by another program (DB browser? backup tool?): %s", e)
        return EXIT_LOCKED
    if any(k in m for k in ("disk i/o", "readonly", "read-only", "unable to open", "disk is full", "full")):
        if "full" in m:
            log.error("Disk full while writing the database: %s", e)
            return EXIT_DISK_FULL
        log.error("Database storage problem: %s (drive unplugged or read-only?). Rerun to resume.", e)
        return EXIT_STORAGE
    log.error("Database error: %s. If this repeats, run `repair-db` (backs up the DB and rebuilds it "
              "from posts.jsonl / files.jsonl; no downloads needed).", e)
    return EXIT_DB
```

What to check: A locked DB (e.g. opened in a DB browser) is exit 5, a vanished/read-only drive exit 6, real damage exit 7 with a `repair-db` hint.

`e6ai_archiver/config.py` lines 111-138

```
def load_config_file(explicit: str | None, env=os.environ) -> tuple[dict, Path | None]:
    path = explicit or env.get("E6AI_CONFIG")
    if path:
        p = Path(path)
        if not p.is_file():
            raise ValueError(f"config file not found: {p}")
        candidates = [p]
    else:
        candidates = [p for p in default_config_paths() if p.is_file()]
    if not candidates:
        return {}, None
    cp = configparser.ConfigParser(interpolation=None, inline_comment_prefixes=(";", "#"))
    try:
        with open(candidates[0], encoding="utf-8-sig") as f:  # -sig: tolerate Notepad's BOM
            cp.read_file(f)
    except UnicodeDecodeError as e:
        raise ValueError(f"{candidates[0]}: not valid UTF-8 (save it as UTF-8 in Notepad): {e}") from None
    except configparser.Error as e:
        raise ValueError(f"{candidates[0]}: cannot parse config file: {e}".replace("\n", " ")) from None
    if not cp.has_section(SECTION):
        raise ValueError(f"{candidates[0]}: missing [{SECTION}] section")
    values = {k.replace("-", "_"): v.strip() for k, v in cp.items(SECTION)}
    unknown = set(values) - set(SETTINGS) - {"no_keep_awake"}
    if unknown:
        raise ValueError(f"{candidates[0]}: unknown setting(s): {', '.join(sorted(unknown))}")
    if values.get("api_key"):
        log.warning("api_key is stored in %s; prefer the E6AI_API_KEY environment variable", candidates[0])
    return values, candidates[0]
```

What to check: A malformed or non-UTF-8 ini file gives a clear error (exit 2). Unknown keys and bad values are errors. An API key in the file triggers a warning.

10. Download order: priority SQL

The queue is one SQLite table `queue(rank PRIMARY KEY, post_id UNIQUE, cum_bytes)`. Ranking, filtering and the running byte total are computed inside SQLite with window functions, so 162k rows take about 0.4 s and almost no Python memory.

`e6ai_archiver/priority.py` lines 26-45

```
ORDER_SQL = {
    "default": "id DESC",  # 1.2 behaviour: newest first
    "newest": "id DESC",
    "oldest": "id ASC",
    "popular": "COALESCE(score, 0) DESC, COALESCE(fav_count, 0) DESC, id DESC",
    "favorites": "COALESCE(fav_count, 0) DESC, COALESCE(score, 0) DESC, id DESC",
    "smallest-first": "size IS NULL, size ASC, id DESC",
    "balanced": "bal DESC, id DESC",
}

# balanced = 0.5 * score percentile + 0.3 * favorites percentile + 0.2 * small-file percentile
# (PERCENT_RANK over the candidate set: 0 = lowest, 1 = highest). Percentiles instead of raw
# values keep one viral post from flattening everything else, and need no math functions.
BALANCED_SQL = ("0.5 * PERCENT_RANK() OVER (ORDER BY COALESCE(score, 0))"
                " + 0.3 * PERCENT_RANK() OVER (ORDER BY COALESCE(fav_count, 0))"
                " + 0.2 * (1.0 - PERCENT_RANK() OVER (ORDER BY COALESCE(size, 9e18)))")

# Files that can still be downloaded, or already were. Deleted / no-URL posts are not ranked.
CANDIDATE_STATUSES = ("pending", "ok", "missing", "failed", "md5_mismatch")
RATINGS = {"s", "q", "e"}
```

What to check: Every order ends in `id DESC`, so ties are deterministic (same rank on every rebuild). `COALESCE(..., 0)` = a null score/favs ranks like 0, never crashes or floats to the top. `balanced` uses percentiles (no extension functions needed).

`e6ai_archiver/priority.py` lines 112-135

```
    def where(self, candidates: bool = True) -> tuple[str, list]:
        """SQL WHERE clause (on posts) for the filters, with its parameters. candidates=False: the
        filters only (1.7 report: was a post left out by a filter?)."""
        sql = [f"file_url IS NOT NULL AND file_status IN ({','.join('?' * len(CANDIDATE_STATUSES))})"
               if candidates else "1"]
        args: list = list(CANDIDATE_STATUSES) if candidates else []
        if self.min_score is not None:
            sql.append("COALESCE(score, 0) >= ?"); args.append(self.min_score)
        if self.min_favs is not None:
            sql.append("COALESCE(fav_count, 0) >= ?"); args.append(self.min_favs)
        if self.rating:
            sql.append(f"rating IN ({','.join('?' * len(self.rating))})"); args += self.rating
        if self.director:  # any of them; tags is the space-separated list of all tags
            sql.append("(" + " OR ".join(["instr(' ' || COALESCE(tags, '') || ' ', ?) > 0"] * len(self.director)) +
")")
            args += [f" {d} " for d in self.director]
        for t in self.tags:
            neg = t.startswith("-")
            sql.append(f"instr(' ' || COALESCE(tags, '') || ' ', ?) {'=' if neg else '>'} 0")
            args.append(f" {t.lstrip('-')} ")
        if self.max_size_mb is not None:
            sql.append("(size IS NULL OR size <= ?)"); args.append(int(self.max_size_mb * 2**20))
        if self.exclude_ext:
            sql.append(f"COALESCE(ext, '') NOT IN ({','.join('?' * len(self.exclude_ext))})"); args +=
self.exclude_ext
        return " AND ".join(sql), args
```

What to check: 1.7: `candidates=False` gives only the filters (the completeness report uses them). All filters are bound parameters (no string-built values). Tag match is on whole words (`tag` inside `tags`). Only downloadable statuses are ranked; `ok` files stay ranked so tier progress and ranks don't shift after a resume.

`e6ai_archiver/priority.py` lines 138-146

```
def _ranked_select(spec: Spec, extra_where: str = "") -> tuple[str, list]:
    """SELECT (rn, id, size) for the candidates in priority order."""
    where, args = spec.where()
    if extra_where:
        where += " AND " + extra_where
    cols = "id, score, fav_count, size" + (f", {BALANCED_SQL} AS bal" if spec.order == "balanced" else "")
    sql = (f"SELECT ROW_NUMBER() OVER (ORDER BY {ORDER_SQL[spec.order]}) AS rn, id, COALESCE(size, 0) AS sz "
           f"FROM (SELECT {cols} FROM posts WHERE {where})")
    return sql, args
```

What to check: One `ROW_NUMBER() OVER (ORDER BY ...)` over the filtered subquery.

`e6ai_archiver/priority.py` lines 161-198

```
    def ensure(self) -> bool:
        """Build the queue if the order/filters/export changed; else append newly found posts.
        Returns True if it was rebuilt."""
        sig = self.signature()
        if self.s.get_state("queue_sig") != sig:
            self.rebuild(sig)
            return True
        self.append_new()
        return False

    def rebuild(self, sig: str | None = None) -> int:
        db = self.s.db
        prev = self.s.get_state("queue_spec")
        db.execute("DELETE FROM queue")
        sel, args = _ranked_select(self.spec)
        db.execute(f"INSERT INTO queue(rank, post_id, cum_bytes) SELECT rn, id, "
                   f"SUM(sz) OVER (ORDER BY rn ROWS UNBOUNDED PRECEDING) FROM ({sel})", args)
        n = db.execute("SELECT COUNT(*) FROM queue").fetchone()[0]
        self.s.set_state("queue_sig", sig or self.signature())
        self.s.set_state("queue_spec", self.spec.describe())
        self.s.commit()
        if prev and prev != self.spec.describe():
            log.info("download order changed (%s -> %s); queue rebuilt, finished files are kept", prev,
                     self.spec.describe())
        log.info("priority queue: %d files ranked (%s)", n, self.spec.describe())
        return n

    def append_new(self) -> int:
        """Posts found after the queue was built (API crawl): ranked among themselves, appended."""
        db = self.s.db
        last = db.execute("SELECT COALESCE(MAX(rank), 0), COALESCE(MAX(cum_bytes), 0) FROM queue").fetchone()
        sel, args = _ranked_select(self.spec, "file_status = 'pending' AND id NOT IN (SELECT post_id FROM queue)")
        db.execute(f"INSERT INTO queue(rank, post_id, cum_bytes) SELECT ? + rn, id, ? + "
                   f"SUM(sz) OVER (ORDER BY rn ROWS UNBOUNDED PRECEDING) FROM ({sel})", [last[0], last[1], *args])
        n = db.execute("SELECT changes()").fetchone()[0]
        if n:
            log.debug("priority queue: appended %d new files", n)
        return n
```

What to check: The queue is rebuilt only when the signature (order + filters + export generation) changes, e.g. the user switched order mid-run or a new export was merged. Tier limits (`--top`, `--limit-gb`) are not in the signature: they only cut the queue. Posts found later by the API crawl are appended (ranked among themselves).

`e6ai_archiver/priority.py` lines 201-211

```
    def tier_end(self) -> int:
        """Highest rank inside the selected tier (--top / --limit-gb), or the whole queue."""
        db = self.s.db
        end = db.execute("SELECT COALESCE(MAX(rank), 0) FROM queue").fetchone()[0]
        if self.spec.top is not None:
            end = min(end, self.spec.top)
        if self.spec.limit_gb is not None:
            r = db.execute("SELECT COALESCE(MAX(rank), 0) FROM queue WHERE cum_bytes <= ?",
                           (int(self.spec.limit_gb * 2**30),)).fetchone()[0]
            end = min(end, r)
        return end
```

What to check: `--limit-gb` uses the precomputed cumulative bytes (an index lookup), `--top` the rank.

`e6ai_archiver/priority.py` lines 221-234

```
    def iter_pending(self, end: int | None = None, batch: int = 500):
        """Yield (rank, id, md5, ext, size, file_url, file_path) for pending files in rank order,
        keyset-paginated on rank (constant memory; files finished meanwhile are skipped)."""
        end = self.tier_end() if end is None else end
        last = 0
        while True:
            rows = self.s.db.execute(
                "SELECT q.rank, p.id, p.md5, p.ext, p.size, p.file_url, p.file_path FROM queue q "
                "JOIN posts p ON p.id = q.post_id WHERE q.rank > ? AND q.rank <= ? AND p.file_status = 'pending' "
                "ORDER BY q.rank LIMIT ?", (last, end, batch)).fetchall()
            if not rows:
                return
            yield from rows
            last = rows[-1][0]
```

What to check: Keyset pagination on rank: constant memory, and files finished meanwhile are skipped. Restarting resumes at the first pending rank.

11. Export merge SQL (incremental re-seed)

Each export is streamed into a temporary `export_stage` table in chunks, then merged with set-based SQL in one transaction (a stop mid-merge leaves the DB unchanged). Files are never re-downloaded unless the md5 changed.

`e6ai_archiver/exports.py` lines 317-345

```
_FRESH = "(p.fetched_at IS NULL OR p.fetched_at <= :t OR (s.change_seq IS NOT NULL AND s.change_seq >
COALESCE(p.change_seq, -1)))"
_NEW_STATUS = """CASE
      WHEN s.is_deleted = 1 THEN CASE WHEN p.file_status = 'ok' THEN 'ok' ELSE 'deleted' END
      WHEN s.md5 IS NOT NULL AND p.md5 IS NOT NULL AND s.md5 <> p.md5 THEN 'pending'
      WHEN p.file_status IN ('deleted', 'no_url') AND s.file_url IS NOT NULL THEN 'pending'
      ELSE p.file_status END"""
MERGE_SQL = f"""UPDATE posts AS p SET
    score = s.score, fav_count = s.fav_count, rating = COALESCE(s.rating, p.rating),
    tags = COALESCE(s.tags, p.tags), tag_cats = COALESCE(s.tag_cats, p.tag_cats),
    is_deleted = s.is_deleted, change_seq = COALESCE(s.change_seq, p.change_seq),
    updated_at = COALESCE(s.updated_at, p.updated_at), parent_id = s.parent_id,
    size = COALESCE(s.size, p.size), width = COALESCE(s.width, p.width), height = COALESCE(s.height, p.height),
    file_status = {_NEW_STATUS},
    md5 = COALESCE(s.md5, p.md5), ext = COALESCE(s.ext, p.ext),
    file_url = CASE WHEN s.file_url IS NOT NULL AND (p.file_url IS NULL OR s.md5 <> p.md5) THEN s.file_url
                    ELSE p.file_url END,
    json = CASE WHEN instr(p.json, '"_source":"db_export"') > 0 THEN s.json ELSE p.json END,
    fetched_at = :t
  FROM export_stage AS s
  WHERE p.id = s.id AND s.id > :lo AND s.id <= :hi AND {_FRESH} AND (
    p.score IS NOT s.score OR p.fav_count IS NOT s.fav_count OR p.tags IS NOT COALESCE(s.tags, p.tags)
    OR p.rating IS NOT COALESCE(s.rating, p.rating) OR p.is_deleted IS NOT s.is_deleted
    OR p.md5 IS NOT COALESCE(s.md5, p.md5) OR p.parent_id IS NOT s.parent_id
    OR p.change_seq IS NOT COALESCE(s.change_seq, p.change_seq) OR p.file_status IS NOT ({_NEW_STATUS}))"""
INSERT_NEW_SQL = f"""INSERT INTO posts({_STAGE_COLS}, fetched_at)
  SELECT {_STAGE_COLS}, :t FROM export_stage s WHERE s.id > :lo AND s.id <= :hi AND NOT EXISTS (SELECT 1 FROM posts p
WHERE p.id = s.id)"""
# Rows that came from an earlier export but are absent now (destroyed on the site).
VANISHED_WHERE = """instr(json, '"_source":"db_export"') > 0 AND id <= :max_id
  AND NOT EXISTS (SELECT 1 FROM export_stage s WHERE s.id = posts.id)"""
```

What to check: Only rows that actually changed are written. An API-fetched JSON is never overwritten by older export data (`_FRESH`: export time / `change_seq`). Deleted on the site: `deleted`, but an already-saved file stays `ok` and is never removed. md5 changed (replacement): back to `pending`.

`e6ai_archiver/exports.py` lines 437-444

```
        if prev.get("rows") and staged >= 0.9 * prev.get("staged", prev["rows"]):
            db.execute(f"UPDATE posts SET is_deleted = 1, file_status = CASE WHEN file_status = 'ok' THEN 'ok' "
                       f"ELSE 'deleted' END WHERE file_status <> 'deleted' AND {VANISHED_WHERE}",
                       {"max_id": stats["max_id"]})
            stats["vanished"] = db.execute("SELECT changes()").fetchone()[0]
        elif prev.get("rows"):
            log.warning("posts export has %d rows, far fewer than last time (%s); not treating missing posts "
                        "as deleted", staged, prev.get("staged", prev["rows"]))
```

What to check: Posts that vanished between exports are marked deleted only if the new export has at least 90 % of the previous row count, so a truncated export can't mass-delete.

`e6ai_archiver/exports.py` lines 241-277

```
class HeaderMap:
    """Maps a CSV header to canonical column names (case/space-insensitive, with aliases).

    Unknown columns are kept in the raw JSON and warned about once; a missing used column
    becomes NULL (with a warning); a missing required column is a clear error."""

    def __init__(self, kind: str, header: list[str] | None, known: tuple, used: tuple = ()):
        if not header:
            raise RuntimeError(f"{kind} export has no header row")
        self.kind, self.rename, unknown = kind, {}, []
        for h in header:
            k = (h or "").strip().lower().replace(" ", "_")
            k = ALIASES.get(k, k) if k not in known else k
            self.rename[h] = k
            if k not in known:
                unknown.append(h)
        have = set(self.rename.values())
        miss_req = [c for c in REQUIRED.get(kind, ()) if c not in have]
        if miss_req:
            raise RuntimeError(f"{kind} export is missing required column(s) {', '.join(miss_req)} "
                               f"(header: {', '.join(header[:12])}...)")
        self.missing = [c for c in used if c not in have]
        self.unknown = unknown
        if unknown:
            log.warning("%s export has new/unknown column(s) %s; kept in the raw JSON, otherwise ignored",
                        kind, ", ".join(unknown))
        if self.missing:
            log.warning("%s export lacks column(s) %s; those values will be empty", kind, ", ".join(self.missing))
        self.identity = all(h == k for h, k in self.rename.items())

    def row(self, row: dict) -> dict:
        if self.identity:
            return row
        return {self.rename.get(k, k): v for k, v in row.items()}


def _bool(v) -> bool:
```

What to check: Columns are mapped by header name, not position. Known renames are aliased, unknown columns are logged once and ignored, a missing required column raises (the previous good copy and the DB stay as they were).

12. sync-exports: refresh, verify, fall back

`e6ai_archiver/exports.py` lines 593-642

```
def sync_exports(client, store, names: str | None = "all", force: bool = False, apply: bool = True) -> dict:
    """Refresh the export bundle (at most once per SYNC_MIN_HOURS unless force) and merge it.

    Per export: same sha256 as the listing -> nothing to do. Otherwise download to .part,
    verify sha256, decompress fully + check the header, move the old copy to previous/, and
    swap in the new one. Any failure (other than 403) keeps the last good copy and carries on.
    Then tags -> pools -> posts are merged into the DB if they changed since the last merge."""
    out = store.out
    names = with_tags(names)
    d = export_dir(out)
    d.mkdir(parents=True, exist_ok=True)
    man = read_manifest(out)
    res: dict = {"checked": False, "downloaded": [], "unchanged": [], "failed": {}, "applied": [], "sizes": {}}
    last = store.get_state("exports_checked_at") or 0
    recent = not force and time.time() - last < SYNC_MIN_HOURS * 3600
    # 1.8: a missing tags export is fetched even inside the once-a-day window
    tags_only = (recent and "tags" in wanted_exports(names, list(APPLY_ORDER))
                 and not (export_dir(out) / "tags.csv.gz").is_file() and not store.has_tag_categories())
    if recent and not tags_only:
        log.info("exports were checked %.1f h ago (they are rebuilt once a day); not checking again "
                 "(use --refresh-export to force)", (time.time() - last) / 3600)
    else:
        if tags_only:
            log.info("the tags export is missing (director folders need it): fetching it now")
        emit("phase", name="checking database exports")
        listing = None
        try:
            listing = list_exports(client)
        except _FATAL:  # 403 / disk full / drive gone: stop, never fall back silently
            raise
        except (requests.RequestException, RuntimeError, ValueError) as e:
            log.warning("could not read the export listing (%s); using the local copies", e)
        if listing is not None:
            res["checked"] = True
            if not tags_only:
                store.set_state("exports_checked_at", time.time())
            by = {e.get("name"): e for e in listing if isinstance(e, dict) and e.get("name")}
            for name in (["tags"] if tags_only else wanted_exports(names, list(by))):
                e = by.get(name)
                if e is None:
                    if name == "tags":
                        res["failed"]["tags"] = "the site's export listing has no tags export"
                    log.warning("no export named %r on the site", name)
                    continue
                res["sizes"][name] = int(e.get("file_size") or 0)
                _sync_one(client, out, e, man, res)
                write_manifest(out, man)
        else:
            log.warning("no export listing available; using whatever is already downloaded")
    store.commit()
```

What to check: Checks at most once per 20 h unless `--force`. A listing error falls back to local copies; **403, disk full and drive gone are re-raised** (`_FATAL`) because `ForbiddenError` is a `RuntimeError` subclass and would otherwise be swallowed (fixed in 1.3, with a test that no further request follows a 403). 1.8: `with_tags`, and the tags-only fetch inside the window.

`e6ai_archiver/exports.py` lines 742-780

```
def _sync_one(client, out, e: dict, man: dict, res: dict) -> None:
    d = export_dir(out)
    name = e["name"]
    fname = safe_filename(e.get("file_name") or f"{name}.csv.gz", f"{name}.csv.gz")
    cur = d / fname
    want = (e.get("checksum") or "").lower() or None
    upd = _parse_ts(e.get("updated_at"))
    if upd and time.time() - upd > STALE_HOURS * 3600:
        log.warning("the %s export on the site is stale (last rebuilt %s)", name, e.get("updated_at"))
    have = man.get(fname, {})
    if cur.is_file() and want and have.get("sha256") == want and cur.stat().st_size == have.get("size"):
        res["unchanged"].append(name)
        return
    if cur.is_file() and want and sha256_of(cur) == want:  # e.g. downloaded by 1.2 (no manifest)
        man[fname] = {"sha256": want, "size": cur.stat().st_size, "updated_at": e.get("updated_at"),
                      "downloaded_at": have.get("downloaded_at") or cur.stat().st_mtime}
        res["unchanged"].append(name)
        return
    inc = d / "incoming"
    try:
        new = download_export(client, e, inc)
        validate_export(new, name)
    except _FATAL:
        safe_unlink(inc / fname)
        raise
    except (RuntimeError, OSError, requests.RequestException) as ex:
        log.warning("%s export not updated (%s); keeping the last good copy", name, ex)
        res["failed"][name] = str(ex)
        safe_unlink(inc / fname)
        return
    if cur.is_file():
        (d / "previous").mkdir(exist_ok=True)
        safe_replace(cur, d / "previous" / fname)
        if fname in man:
            man["previous/" + fname] = man[fname]
    safe_replace(new, cur)
    man[fname] = {"sha256": sha256_of(cur) if not want else want, "size": cur.stat().st_size,
                  "updated_at": e.get("updated_at"), "downloaded_at": time.time()}
    res["downloaded"].append(name)
```

What to check: Same sha256 as the listing: no download. Otherwise `incoming/` -> sha256 check -> full decompress + header check -> old copy moved to `previous/` -> swap. Any failure keeps the last good copy; 403 deletes the partial file and stops everything.

`e6ai_archiver/exports.py` lines 574-586

```
def good_export_path(out, name: str) -> Path | None:
    """The newest copy of an export that passes validation: current, else db_export/previous/."""
    d = export_dir(out)
    for p in (d / f"{name}.csv.gz", d / "previous" / f"{name}.csv.gz"):
        if p.is_file():
            try:
                validate_export(p, name)
                if p.parent.name == "previous":
                    log.warning("using the previous good copy of the %s export (the current one is damaged)", name)
                return p
            except RuntimeError as e:
                log.warning("%s", e)
    return None
```

What to check: Fallback order: current copy if valid per the manifest, else `previous/`.

13. Order selection and priority download

`e6ai_archiver/priority.py` lines 73-94

```
    def from_args(cls, a) -> "Spec":
        g = lambda k: getattr(a, k, None)  # noqa: E731
        order = (g("order") or "default").strip().lower()
        if order not in ORDERS:
            raise ValueError(f"unknown order {order!r}; choose one of: {', '.join(ORDERS)}")
        rating = _list(g("rating"))
        bad = [r for r in rating if r not in RATINGS]
        if bad:
            raise ValueError(f"unknown rating(s) {', '.join(bad)}; use s, q, e (e.g. --rating s,q)")
        director = [d.replace(" ", "_") for d in _list(g("director"))]
        for name, v in (("min_score", g("min_score")), ("min_favs", g("min_favs")), ("top", g("top"))):
            if v is not None and not isinstance(v, int):
                raise ValueError(f"{name} must be a whole number")
        for name, v in (("max_size_mb", g("max_size_mb")), ("limit_gb", g("limit_gb"))):
            if v is not None and v <= 0:
                raise ValueError(f"{name} must be greater than 0")
        if g("top") is not None and g("top") <= 0:
            raise ValueError("top must be greater than 0")
        return cls(order=order, min_score=g("min_score"), min_favs=g("min_favs"), rating=sorted(set(rating)),
                   director=director, tags=_list(g("filter_tags")), max_size_mb=g("max_size_mb"),
                   exclude_ext=[e.lstrip(".") for e in _list(g("exclude_ext"))],
                   limit_gb=g("limit_gb"), top=g("top"))
```

What to check: One validation path for CLI, ini, env, `run.ps1` and the GUI panel: unknown order or rating, negative sizes or non-integers give a clear error (exit 2 before any request; in the GUI an inline message).

`e6ai_archiver/cli.py` lines 775-778

```
        a.spec = Spec.from_args(a) if a.cmd in PRIORITY_COMMANDS | {"sync-exports", "retry-failed"} else None
        if a.cmd == "retry-failed":
            from .failures import parse_reasons
            parse_reasons(a.reason)  # unknown words fail here, before anything runs
```

What to check: The Spec is built once, after config precedence (CLI > env > ini > default) has been applied.

`e6ai_archiver/archiver.py` lines 358-412

```
    def _download_by_priority(self) -> None:
        """Download pending files in queue-rank order, within the selected tier (in parallel)."""
        pq = self.pq
        rebuilt = pq.ensure()
        end = pq.tier_end()
        n, b = pq.pending_in_tier(end)
        tier = pq.tier_progress(end)
        emit("priority", order=self.spec.describe(), tier_end=end,
             ranked=self.s.db.execute("SELECT COUNT(*) FROM queue").fetchone()[0], rebuilt=rebuilt)
        emit("tier", **tier)
        if not n:
            left = self.s.pending_summary()[0]
            if left and (self.spec.top or self.spec.limit_gb or self.spec.filtered()):
                log.info("tier complete (%s); %d other pending files are outside the selected order/filters/limit",
                         self.spec.describe(), left)
            return
        if n > 1:
            log.info("downloading %d files (%s) in %s order, tier = ranks 1-%d, %d parallel downloads", n,
                     fmt_bytes(b), self.spec.order, end, self.workers)
        emit("phase", name=f"downloading files ({self.spec.order} order)")
        self._emit_archive()
        emit("queue", files=n, bytes=b)
        prog = Progress(n, b, None)

        def done(job, status, nbytes):
            prog.update(nbytes)
            if status == "ok":
                tier["done_files"] += 1
                tier["done_bytes"] += job.size or nbytes
            emit("rank", rank=job.meta.get("rank"), tier_end=end)
            emit("file_done", post_id=job.post_id, status=status, nbytes=nbytes, size=job.size)
            emit("tier", **tier)
            emit("stats", **self.stats)
            disk_tick(self)
        try:
            with self._engine(done) as pd:
                for chunk in _chunks(pq.iter_pending(end, batch=100), 100):
                    if self.api_json:
                        pd.pump(0)
                        self._refresh_api_json([r[1] for r in chunk])
                    stop = False
                    for rank, post_id, *_ in chunk:
                        row = self.s.db.execute("SELECT md5, ext, size, file_url, file_path FROM posts WHERE id=? "
                                                "AND file_status='pending'", (post_id,)).fetchone()
                        if row is None:  # e.g. the API says it was deleted meanwhile
                            continue
                        if not pd.submit(post_id, *row, rank=rank):
                            stop = True
                            break
                    if stop:
                        break
        finally:
            self.s.commit()
        if n > 1:
            log.info("downloaded batch: %d files, %s", prog.files, fmt_bytes(prog.bytes))
```

What to check: Downloads in rank order within the tier, 100 at a time; each chunk first gets its fresh API JSON (`_refresh_api_json`, 1 request per 100 files). Rows that stopped being pending (deleted per the API) are skipped. Progress events drive the GUI tier bar and "rank X of N". 1.6: each pending file is handed to the parallel downloader (`submit`).

`e6ai_archiver/archiver.py` lines 414-427

```
    def _refresh_api_json(self, ids: list[int]) -> None:
        """One API request per 100 files: fresh per-post JSON for export-seeded posts (the export
        has no per-post sources/relationships objects and may be up to a day old)."""
        need = [i for (i,) in self.s.db.execute(
            f"SELECT id FROM posts WHERE id IN ({','.join('?' * len(ids))}) "
            "AND instr(json, '\"_source\":\"db_export\"') > 0", ids)]
        if not need:
            return
        posts = self.c.posts(None, len(need), "id:" + ",".join(map(str, need)))
        for post in posts:
            if post["id"] in need:
                self.s.merge_api_json(post)
                self.stats["api_json"] += 1
        self.s.commit()
```

What to check: Only export-seeded posts are looked up; the API JSON replaces the export-derived JSON (`merge_api_json`). Uses the same rate limiter and 403 stop as everything else.

`e6ai_archiver/gui.py` lines 558-586

```
    def on_start_clicked(self):
        from .priority import Spec
        import argparse
        vals = self.panel_values()
        ns = argparse.Namespace(**vars(self.start_args))
        conv = {"min_score": int, "min_favs": int, "top": int, "max_size_mb": float, "limit_gb": float}
        try:
            for k, v in vals.items():
                setattr(ns, k, (conv[k](v) if k in conv else v) if v else None)
            ns.order = ns.order or "default"
            spec = Spec.from_args(ns)
        except ValueError as e:
            self.panel_msg.set(f"Invalid value: {e}")
            return
        for k in vals:
            setattr(self.start_args, k, getattr(ns, k))
        self.start_args.spec = spec
        for _, w in self.inputs.values():
            w.configure(state="disabled")
        self.start_btn.configure(state="disabled")
        self.panel_msg.set(f"running with {spec.describe()}")
        self.m.order = spec.describe()
        self.started = True
        self.m.phase = "starting"
        self.pause_btn.configure(state="normal")
        self.stop_btn.configure(text="Stop")
        log.info("starting with %s", spec.describe())
        if self.on_start is not None:
            self.on_start()
```

What to check: The panel is shown before anything starts (no requests until Start). Invalid input keeps the panel open; closing the window before Start exits with 130 and makes no request.

14. Control panel (1.4): the panel builds a command line and runs it in the background

`e6ai_archiver/control_panel.py` lines 485-534

```
def build_command(state: dict, *, python: str | None = None) -> list[str]:
    """Full argv for the child process, e.g. [python, -m, e6ai_archiver, --no-config, ..., full, ...].

    Global options come first (argparse needs them before the subcommand). Values equal to the CLI
    defaults are left out; --out and --log-file are always absolute. Not included: the API key (it goes
    into the child's environment) and the panel-only --progress-json / --control-file (CliRunner adds them).
    """
    st = normalized(state)
    mode = MODE[st.get("mode") or DEFAULT_MODE]
    args = ["--no-config", "--out", _abs_out(st["out"])]
    if st["username"]:
        args += ["--username", st["username"]]
    args += ["--log-file", os.path.abspath(st["log_file"]) if st["log_file"] else default_log_file(st["out"])]
    tail, positional = [], []
    for opt in OPTIONS:
        if opt.key in ("out", "username", "log_file", "api_key") or not applies(opt, mode):
            continue
        v = st[opt.key]
        target = args if opt.cmds is ALL else tail
        flag = opt.flag_for(mode.cmd)
        if opt.kind == "flag":
            if v is True or str(v).lower() in ("1", "true", "yes", "on"):
                target.append(flag)
        elif opt.kind == "neg":
            if not (v is True or str(v).lower() in ("1", "true", "yes", "on")):
                target.append(flag)
        elif opt.kind == "kinds":
            positional += [k for k in META_KINDS if k in str(v).replace(" ", ",").split(",")]
        elif opt.key == "exports_download":
            if v:
                tail += [flag, *str(v).replace(",", " ").split()]
        elif opt.kind == "ratings":
            r = _ratings(v)
            if r not in ("", "e,q,s"):
                target += [flag, r]
        elif v in ("", None) or _is_default(opt, v):
            continue
        elif opt.kind in ("int", "float"):
            try:
                target += [flag, _fmt_num(_num(opt, v))]
            except ValueError:
                target += [flag, str(v)]  # invalid: validate() reports it; keep the preview honest
        else:
            target += [flag, str(v)]
    exports_dl = [i for i, a in enumerate(tail) if a == "--download"]
    if exports_dl:  # nargs="*" option: keep it last so nothing after it is taken as a name
        i = exports_dl[0]
        n = 1 + len(str(st["exports_download"]).replace(",", " ").split())
        tail = tail[:i] + tail[i + n:] + tail[i:i + n]
    return [python or child_python(), "-m", "e6ai_archiver", *args, mode.cmd, *positional, *mode.fixed, *tail]
```

What to check: Every value is passed explicitly with `--no-config`, so the run does exactly what the window shows. Options the mode does not use are skipped (`applies`), defaults are left out, the API key is never in argv.

`e6ai_archiver/control_panel.py` lines 556-566

```
def child_env(state: dict, base: dict | None = None) -> dict:
    """Environment for the child: no E6AI_* settings (the command line is the whole truth) except the API
    key, UTF-8 everywhere, unbuffered output, and this project on PYTHONPATH."""
    env = {k: v for k, v in (os.environ if base is None else base).items() if not k.upper().startswith("E6AI_")}
    key = str(state.get("api_key") or "").strip() or (os.environ if base is None else base).get("E6AI_API_KEY", "")
    if key:
        env["E6AI_API_KEY"] = key
    env.update(PYTHONUTF8="1", PYTHONIOENCODING="utf-8", PYTHONUNBUFFERED="1")
    pp = env.get("PYTHONPATH")
    env["PYTHONPATH"] = str(PROJECT_ROOT) + (os.pathsep + pp if pp else "")
    return env
```

What to check: The API key reaches the child only through its environment (`E6AI_API_KEY`); every other `E6AI_*` variable is removed because the command line is complete.

`e6ai_archiver/control_panel.py` lines 327-341

```
OMITTED = {
    "cmd": "chosen with the 'What to do' list (Mode)",
    "config": "the panel reads/writes e6ai-archiver.ini itself and passes every value explicitly (--no-config)",
    "no_config": "always passed by the panel, so the command is self-contained",
    "gui": "the old in-process progress window; the panel is the window now",
    "gui_autostart": "only for the old window's start panel",
    "progress_json": "added at launch: the panel's progress channel (not shown in the command preview)",
    "control_file": "added at launch: how Stop / Pause reach the run (not shown in the command preview)",
    "apply": "Mode 'Reorganize: move files' adds --apply",
    "dry_run@preview": "preview never uses the network; --dry-run is accepted there only for symmetry",
    "help": "argparse help", "version": "argparse --version",
    "hash_workers": "verify uses one hashing thread per CPU core; command line only (--hash-workers)",
    "only_new": "verify: 'Verify when finished' checks only unchecked files; command line only (--only-new)",
    "restart": "verify resumes by default; command line only (--restart starts over)",
}
```

What to check: The CLI options deliberately without a widget, with the reason. A test parses `cli.build_parser()` and fails if any option is neither a widget nor listed here.

`e6ai_archiver/control_panel.py` lines 1118-1136

```
    def start(self, argv: list[str], env: dict | None = None, cwd: str | None = None) -> None:
        if self.running:
            raise RuntimeError("a run is already in progress")
        fd, self.control_path = tempfile.mkstemp(prefix="e6ai-control-", suffix=".txt")
        os.close(fd)
        write_control_word(self.control_path, "run")
        full = [*argv[:3], "--progress-json", "--control-file", self.control_path, *argv[3:]]
        kw = dict(stdin=subprocess.DEVNULL, stdout=subprocess.PIPE, stderr=subprocess.PIPE, cwd=cwd or
str(PROJECT_ROOT),
                  env=env, bufsize=1, text=True, encoding="utf-8", errors="replace")
        kw.update(popen_platform_kwargs())
        self.saw_traceback = False
        self.stop_requested_at = None
        self.proc = subprocess.Popen(full, **kw)
        self.started_at = time.monotonic()
        self._threads = [threading.Thread(target=self._read, args=(self.proc.stdout, True), daemon=True),
                         threading.Thread(target=self._read, args=(self.proc.stderr, False), daemon=True)]
        for t in self._threads:
            t.start()
        threading.Thread(target=self._wait, daemon=True).start()
```

What to check: One child at a time. The control file starts as `run`; `--progress-json` / `--control-file` are added here (not in the preview). stdin is closed; stdout/stderr are read by daemon threads that only put to a queue.

`e6ai_archiver/control_panel.py` lines 1166-1199

```
    def send(self, word: str) -> None:
        if word not in CONTROL_WORDS:
            raise ValueError(word)
        if self.running and self.control_path:
            write_control_word(self.control_path, word)
            if word == "stop" and self.stop_requested_at is None:
                self.stop_requested_at = time.monotonic()

    def stop(self) -> None:
        self.send("stop")

    def pause(self) -> None:
        self.send("pause")

    def resume(self) -> None:
        self.send("run")

    def recheck_disk(self, current: str = "run") -> None:
        """1.9: ask a run that waits for disk space to look again now (then restore the current word)."""
        self.send("recheck")
        def back():
            time.sleep(1.0)
            try:
                self.send(current)
            except (OSError, ValueError):
                pass
        threading.Thread(target=back, daemon=True).start()

    def kill(self) -> None:
        if self.running:
            self.proc.kill()


# ---------------------------------------------------------------------------------------------
```

What to check: Stop / Pause / Resume are single words written atomically to the control file. `kill()` is only used by Force stop.

`e6ai_archiver/control_panel.py` lines 1339-1346

```
def popen_platform_kwargs() -> dict:
    """Windows: no console window flashes up (CREATE_NO_WINDOW) and the child gets its own process group,
    so a Ctrl+C in a console that started the panel does not hit the child mid-write; elsewhere a new
    session for the same reason."""
    if os.name == "nt":
        return {"creationflags": getattr(subprocess, "CREATE_NO_WINDOW", 0x08000000)
                | getattr(subprocess, "CREATE_NEW_PROCESS_GROUP", 0x200)}
    return {"start_new_session": True}
```

What to check: Windows: `CREATE_NO_WINDOW` (no console flashes up) plus a new process group; that is also why Stop uses a file and not a console Ctrl event (a windowless child has no console to receive one).

`e6ai_archiver/events.py` lines 133-155

```
def attach_progress_stream(stream) -> None:
    """Mirror every event to `stream` as `@e6ai-progress ["kind", {...}]` lines (ASCII JSON, flushed).
    If the panel goes away (broken pipe), the run is stopped cleanly instead of running headless."""
    import json
    lock = threading.Lock()

    def sink(kind, data):
        line = PROGRESS_PREFIX + json.dumps([kind, data], default=str, ensure_ascii=True) + "\n"
        try:
            with lock:
                stream.write(line)
                stream.flush()
        except (OSError, ValueError):  # BrokenPipeError / closed pipe: nobody is watching any more
            control.stop()
            import os
            import sys
            if stream is sys.stdout:     # later prints (and the flush at exit) must not fail as well
                try:
                    sys.stdout = open(os.devnull, "w", encoding="utf-8")
                except OSError:
                    pass
            raise
    add_sink(sink)
```

What to check: Progress goes to stdout as JSON lines; logs go to stderr. If the panel disappears (broken pipe), the run stops cleanly instead of continuing invisibly.

`e6ai_archiver/events.py` lines 184-205

```
def watch_control_file(path, interval: float = 0.3) -> threading.Thread:
    """Poll `path` in a daemon thread: "stop" -> control.stop(), "pause" -> pause(), "run" -> resume().
    Missing / empty / unknown content is ignored (the panel creates the file with "run")."""
    def loop():
        last = None
        while not control.stopping:
            word = read_control_word(path)
            if word and word != last:
                last = word
                if word == "stop":
                    log.warning("Stop requested by the control panel; finishing cleanly...")
                    control.stop()
                elif word == "pause":
                    control.pause()
                elif word == "run" and control.paused:
                    control.resume()
                elif word == "recheck":
                    control.recheck()
            time.sleep(interval)
    t = threading.Thread(target=loop, name="control-file", daemon=True)
    t.start()
    return t
```

What to check: The child polls the file every 0.3 s; `stop` calls the same `control.stop()` as Ctrl-C / the 1.3 Stop button, so the clean-up path (delete `.part`, commit, release lock, exit 130) is unchanged.

`e6ai_archiver/cli.py` lines 732-745

```
def wants_control_panel(argv: list[str]) -> bool:
    """`e6ai-archiver --gui` with no command (only global options) opens the control panel."""
    if "--gui" not in argv:
        return False
    i = 0
    while i < len(argv):
        key = argv[i].split("=", 1)[0]
        if key in GLOBAL_VALUE_OPTS:
            i += 1 if "=" in argv[i] else 2
        elif key in GLOBAL_FLAG_OPTS:
            i += 1
        else:
            return False
    return True
```

What to check: `e6ai-archiver --gui` with only global options opens the panel; `--gui <command>` keeps the 1.3 window.

`e6ai_archiver/panel_ui.py` lines 1505-1519

```
    def stop(self):
        if not self.runner.running:
            return
        r = self.runner
        if r.stop_requested_at is None:
            r.stop()
            self._set_status("Stopping...", "warn")
            self.set_message("Stopping cleanly (finishing the current step; at most a few seconds)...", "warn")
            self.log_line("Stop requested.", logging.WARNING)
        elif time.monotonic() - r.stop_requested_at >= cp.FORCE_STOP_AFTER_S:
            if messagebox.askyesno("Force stop", "The run has not stopped yet. End it immediately?\n\nIt is still "
                                   "resumable (unfinished downloads are kept as .part files and redone).",
                                   icon="warning", default="no", parent=self.root):
                r.kill()
                self.log_line("Force stop.", logging.WARNING)
```

What to check: First press: clean stop. After 45 s without exit, a second press offers Force stop (with a confirmation).

`e6ai_archiver/control_panel.py` lines 585-605

```
def path_warnings(path: str) -> list[str]:
    """Problems with an archive folder that are allowed but worth a warning."""
    w = []
    if not path:
        return w
    p = os.path.abspath(os.path.expanduser(path))
    low = p.replace("\\", "/").lower()
    parts = [x for x in low.split("/") if x]
    if any(m in part for part in parts for m in CLOUD_MARKERS):
        w.append("This looks like a cloud-synced folder (OneDrive / Dropbox / Google Drive ...). Syncing "
                 "hundreds of GB and a live SQLite database is slow and can corrupt it: use a plain local "
                 "folder such as D:\\e6ai-archive.")
    if path.startswith(("\\\\", "//")) or _drive_type(p) == 4:
        w.append("This is a network location. It works, but it is slower, and a dropped connection stops the "
                 "run (resumable); the database is safest on a local drive.")
    if len(p) > 120:
        w.append(f"The folder path is long ({len(p)} characters); file names get shortened to stay under the "
                 "Windows path limit. A short folder like D:\\e6ai is better.")
    if os.path.isfile(p):
        w.append("This is a file, not a folder.")
    return w
```

What to check: Cloud-synced, network and very long archive folders are allowed but warned about under the field.

15. Sign-in (1.5): login name + API key, never a password

e6ai is e621ng: API requests are signed in with HTTP Basic auth (login name : API key). There is no password login and no HTML login form anywhere in the code.

`e6ai_archiver/client.py` lines 148-151

```
    def _auth_for(self, url: str):
        if self._auth is None:
            return None
        return self._auth if url.startswith(self.base_url + "/") or url == self.base_url else None
```

What to check: Credentials go only to `https://e6ai.net` (the API). The session has no default auth, so file downloads from the static host never carry them.

`e6ai_archiver/client.py` lines 197-203

```
            if r.status_code == 401:
                r.close()
                who = self.login or "(anonymous)"
                raise AuthError(
                    f"HTTP 401 for {url}: e6ai did not accept the sign-in for login '{who}'. "
                    "Check the login name and API key (the key is not your password). Stopping."
                )
```

What to check: 401 is never retried: `AuthError` (exit 8). The message names the login, not the key.

`e6ai_archiver/auth.py` lines 27-56

```
SELF_FIELDS = ("blacklisted_tags", "api_burst_limit", "remaining_api_limit", "per_page", "email",
               "enable_safe_mode", "upload_limit")
API_KEY_PAGE = "https://e6ai.net/users/home"
CHUNK = 100
RETRY_AFTER_S = 24 * 3600  # same account + same hidden set: check again at most once a day


def test_login(c: Client, login: str) -> dict:
    """ONE signed-in request (GET /users/<login>.json). Raises AuthError (401), ForbiddenError (403)."""
    if not c.authenticated:
        raise ValueError("no API key: set E6AI_API_KEY (the control panel does this for you)")
    data = c.get_json(f"/users/{quote(login, safe='')}.json")
    if isinstance(data, dict) and isinstance(data.get("user"), dict):
        data = data["user"]
    if not isinstance(data, dict):
        raise RuntimeError("unexpected reply from /users: not a JSON object")
    self_view = any(k in data for k in SELF_FIELDS)
    res = {"ok": self_view, "login": login, "user_id": data.get("id"), "name": data.get("name"),
           "level": data.get("level_string"), "safe_mode": data.get("enable_safe_mode"),
           "requests": c.request_count}
    if self_view:
        res["message"] = f"Signed in as {data.get('name') or login}" + (
            f" ({data['level_string']})" if data.get("level_string") else "") + "."
        if data.get("enable_safe_mode"):
            res["warning"] = ("Safe mode is on in your e6ai account settings: questionable and explicit "
                              "posts stay hidden even when signed in.")
    else:
        res["message"] = ("The site answered, but not with your private account fields: the key may "
                          "belong to another account, or the login name is misspelled.")
    return res
```

What to check: `test_login` makes exactly one GET. "Signed in" only if the reply contains fields e621ng returns only to the account owner; a public profile (key of another account, misspelled login) is not success. Safe mode on the account is reported because it keeps q/e posts hidden even when signed in.

`e6ai_archiver/auth.py` lines 70-92

```
def retry_login_required(c: Client, s, max_posts: int | None = None, include_deleted: bool = False) -> dict:
    """Re-fetch metadata for posts recorded without a file URL, 100 per request, and queue the ones
    whose file is now visible (file_status -> 'pending'). Uses the client's limiter and Stop/Pause."""
    ids = _ids(s, include_deleted, max_posts)
    res = {"checked": len(ids), "now_downloadable": 0, "still_hidden": 0, "not_returned": 0, "requests": 0}
    if not ids:
        return res
    emit("phase", name=f"re-checking {len(ids)} posts that needed sign-in")
    before = c.request_count
    for i in range(0, len(ids), CHUNK):
        chunk = ids[i:i + CHUNK]
        got = c.posts(None, 320, "status:any id:" + ",".join(map(str, chunk)))
        seen = set()
        for p in got:
            seen.add(p.get("id"))
            if s.upsert_post(p) == "pending":
                res["now_downloadable"] += 1
            else:
                res["still_hidden"] += 1
        res["not_returned"] += len(set(chunk) - seen)
        s.commit()
    res["requests"] = c.request_count - before
    return res
```

What to check: Posts recorded as `no_url` are looked up again, 100 ids per request (`status:any`), through the same client (limiter, Pause/Stop, 403 stop). `upsert_post` returns `pending` when a file URL is now present; those join the normal download queue.

`e6ai_archiver/auth.py` lines 95-128

```
def auto_retry(c: Client, s, enabled: bool = True) -> dict | None:
    """Called at the start of signed-in run / full / download. Skipped when not signed in, when
    disabled, when nothing is waiting, or when this account already checked the same posts in the
    last 24 hours (so repeated runs don't re-ask for posts it still can't see)."""
    if not c.authenticated:
        n = login_required_count(s)
        if n:
            log.info("%d posts were saved without a file URL (hidden from anonymous visitors?); "
                     "sign in with a login name + API key to fetch them", n)
        return None
    if not enabled:
        log.info("signed in; automatic retry of login-required posts is off (--no-login-retry)")
        return None
    ids = _ids(s, False, None)
    if not ids:
        log.info("signed in; no posts are waiting for sign-in")
        return None
    sig = hashlib.sha256((c.login or "").encode() + b"|" + ",".join(map(str, ids)).encode()).hexdigest()[:16]
    last = s.get_state("login_retry_last") or {}
    if isinstance(last, dict) and last.get("sig") == sig and time.time() - float(last.get("at", 0)) < RETRY_AFTER_S:
        log.info("signed in; %d posts still hidden were already re-checked with this account in the last "
                 "24 h; skipping (run 'recheck' to force)", len(ids))
        return None
    log.info("signed in; re-checking %d posts saved without a file URL (%d request(s))", len(ids),
             -(-len(ids) // CHUNK))
    res = retry_login_required(c, s)
    left = _ids(s, False, None)
    sig = hashlib.sha256((c.login or "").encode() + b"|" + ",".join(map(str, left)).encode()).hexdigest()[:16]
    s.set_state("login_retry_last", {"sig": sig, "at": time.time(), "login": c.login})
    s.commit()
    log.info("login-required posts: %d now downloadable, %d still hidden, %d not returned by the site",
             res["now_downloadable"], res["still_hidden"], res["not_returned"])
    emit("login_retry", **res)
    return res
```

What to check: Anonymous: only a hint. Signed in: re-check non-deleted `no_url` posts once, then skip for 24 h if the same account + same remaining set was already checked (no repeated requests for posts the account can't see). `--no-login-retry` turns it off.

`e6ai_archiver/cli.py` lines 64-75

```
def make_client(a, **kw) -> Client:
    ua = build_user_agent(a.username, a.user_agent)
    key = (a.api_key or "").strip() or None
    login = effective_login(a) if key else None
    if a.login and not key:
        log.warning("login name '%s' is set but there is no API key (E6AI_API_KEY): running signed out", a.login)
    kw.setdefault("max_outage_hours", a.max_outage_hours)
    c = Client(ua, rate=a.rate, login=login, api_key=key, **kw)
    # never log the key itself; the log formatter also masks it should it appear anywhere
    log.info("User-Agent: %s | rate: %.2f req/s | auth: %s", ua, c.limiter.rate,
             f"signed in as {login} (API key from E6AI_API_KEY, not shown)" if c.authenticated else "no (anonymous)")
    return c
```

What to check: The log says whether the run is signed in, never the key. A login name without a key runs signed out with a warning.

`e6ai_archiver/cli.py` lines 366-401

```
def cmd_test_login(a) -> int:
    """One signed-in request to check the login name + API key. No archive folder needed."""
    if not (a.api_key or "").strip():
        print("error: no API key. Set the E6AI_API_KEY environment variable (never put it on the command "
              "line); the control panel does this for you.", file=sys.stderr)
        return EXIT_USAGE
    if not a.username:
        print("error: --username is required (it goes into the User-Agent)", file=sys.stderr)
        return EXIT_USAGE
    try:
        c = make_client(a, max_retries=2, max_outage_hours=0.05)
    except ValueError as e:
        log.error("%s", e)
        return EXIT_USAGE
    try:
        res = test_login(c, c.login)
    except AuthError:
        print(json.dumps({"test_login": {"ok": False, "login": c.login, "reason": "rejected (HTTP 401)",
                                         "message": "e6ai did not accept this login name + API key."}}))
        log.error("sign-in rejected (HTTP 401): check the login name and API key (not your password)")
        return EXIT_AUTH
    except ForbiddenError as e:
        log.error("%s", e)
        log.error("STOPPED on 403. Do not retry with a different/browser User-Agent.")
        return EXIT_403
    except StopRequested:
        return EXIT_STOPPED
    except (RuntimeError, OSError) as e:
        log.error("sign-in test failed: %s", e)
        return EXIT_ERROR
    finally:
        c.close()
    print(json.dumps({"test_login": res}, indent=2))
    if res.get("warning"):
        log.warning("%s", res["warning"])
    return EXIT_OK if res["ok"] else EXIT_AUTH
```

What to check: `test-login`: no key = exit 2 with no request; 401 = 8; 403 = 3 (stop); short retry budget so a test cannot hang for hours.

`e6ai_archiver/redact.py` lines 15-36

```
def register(secret: str | None) -> None:
    """Remember a secret so redact() masks it. Very short values are ignored (they would mask
    ordinary words); e6ai API keys are 24 characters."""
    if secret and len(secret.strip()) >= 6:
        _SECRETS.add(secret.strip())


def forget_all() -> None:
    _SECRETS.clear()


def redact(text, extra: str | None = None):
    """Return text with every registered secret (and `extra`, if given) replaced by '***'."""
    if not isinstance(text, str):
        return text
    secrets = set(_SECRETS)
    if extra and len(extra.strip()) >= 6:
        secrets.add(extra.strip())
    for s in sorted(secrets, key=len, reverse=True):
        if s in text:
            text = text.replace(s, MASK)
    return text
```

What to check: Every secret of 6+ characters is replaced by `***` in log records (file and console), uncaught tracebacks and panel text.

`e6ai_archiver/control_panel.py` lines 1207-1265

```
def test_login_command(state: dict, python: str | None = None) -> list[str]:
    """argv for `test-login`: identity, User-Agent and rate only. The key goes in the environment."""
    st = normalized(state)
    args = ["--no-config", "--username", st["username"]]
    if st["user_agent"]:
        args += ["--user-agent", st["user_agent"]]
    try:
        r = float(st["rate"])
        if r != float(_D["rate"]):
            args += ["--rate", _fmt_num(r)]
    except (TypeError, ValueError):
        pass
    if st["login"]:
        args += ["--login", st["login"]]
    return [python or child_python(), "-m", "e6ai_archiver", *args, "test-login"]


def parse_test_login(code: int | None, stdout: str, stderr: str, key: str = "") -> tuple[str, str]:
    """(level, message) for the inline result: level is ok | warn | error | forbidden."""
    from .redact import redact
    res = None
    try:
        start = stdout.index("{")
        res = json.loads(stdout[start:]).get("test_login")
    except (ValueError, AttributeError):
        pass
    last = next((ln for ln in reversed(stderr.strip().splitlines()) if ln.strip()), "")
    if code == 0 and isinstance(res, dict):
        msg = res.get("message") or "Signed in."
        if res.get("warning"):
            return "warn", f"{msg} {res['warning']}"
        return "ok", msg
    if code == 8:
        return "error", ("Not accepted (HTTP 401): wrong login name or API key. The key is the one from your "
                         "account page, not your password.")
    if code == 3:
        return "forbidden", ("STOPPED: the site answered 403 Forbidden. Do not retry right away; check your "
                             "username / User-Agent, or ask site staff.")
    if code == 2:
        return "error", redact(last.removeprefix("error: ") or "a setting was rejected", key)
    return "error", redact(f"Could not test the sign-in (exit {code}): {last or 'no output'}", key)


def run_test_login(state: dict, timeout: float = 120, python: str | None = None) -> tuple[str, str]:
    """Run `test-login` in a child (UA + rate limiter + 403 stop of the CLI); blocking, call from a thread."""
    key = str(state.get("api_key") or "").strip()
    if not key:
        return "error", "Enter the API key first."
    if not normalized(state)["username"]:
        return "error", "Fill in your e6ai username first (it goes into the User-Agent)."
    argv = test_login_command(state, python)
    try:
        p = subprocess.run(argv, env=child_env(state), capture_output=True, text=True, encoding="utf-8",
                           errors="replace", timeout=timeout, stdin=subprocess.DEVNULL, **popen_platform_kwargs())
    except subprocess.TimeoutExpired:
        return "error", "No answer from the site in time; try again later."
    except OSError as e:
        return "error", f"Could not start the test: {e}"
    return parse_test_login(p.returncode, p.stdout, p.stderr, key)
```

What to check: The panel's Test sign-in runs the CLI command in a child (same UA, API limiter at the chosen rate, 403 stop). argv has no key; `run_test_login` passes it in the child environment (`child_env`).

`e6ai_archiver/control_panel.py` lines 1268-1315

```
def _keyring():
    """The keyring module if a usable backend exists (Windows Credential Manager, macOS Keychain,
    Secret Service ...), else None. keyring is optional: any problem means 'not available'."""
    try:
        import keyring  # type: ignore
        from keyring.backends import fail  # type: ignore
        kr = keyring.get_keyring()
        if isinstance(kr, fail.Keyring) or getattr(kr, "priority", 1) <= 0:
            return None
        return keyring
    except Exception:  # noqa: BLE001 - ImportError, broken backend, D-Bus missing ...
        return None


def keyring_available() -> bool:
    return _keyring() is not None


def saved_key(login: str) -> str:
    kr = _keyring()
    if kr is None or not login:
        return ""
    try:
        return kr.get_password(KEYRING_SERVICE, login) or ""
    except Exception:  # noqa: BLE001
        return ""


def save_key(login: str, key: str) -> bool:
    kr = _keyring()
    if kr is None or not login or not key:
        return False
    try:
        kr.set_password(KEYRING_SERVICE, login, key)
        return True
    except Exception:  # noqa: BLE001
        return False


def forget_key(login: str) -> bool:
    kr = _keyring()
    if kr is None or not login:
        return False
    try:
        kr.delete_password(KEYRING_SERVICE, login)
        return True
    except Exception:  # noqa: BLE001 - PasswordDeleteError when nothing was saved
        return False
```

What to check: keyring is optional: missing module, the fail backend or priority 0 all mean "not available" and the panel hides Remember. Stored under service `e6ai-archiver`, user = login name.

`e6ai_archiver/panel_ui.py` lines 579-623

```
    def test_signin(self):
        if self._signin_busy or self.runner.running:
            return
        st = self.get_state()
        if self._signin_forbidden or self.last_exit == 3:
            if not messagebox.askyesno("Last answer was 403", "The site answered 403 Forbidden last time. Testing "
                                       "again right away is not recommended.\n\nTest anyway?", icon="warning",
                                       default="no", parent=self.root):
                return
        self._remember_key()
        self._signin_busy = True
        self._signin_result("info", "Testing (one request to e6ai.net)...")
        self._update_buttons()
        box = {}

        def work():
            try:
                box["res"] = cp.run_test_login(st)
            except Exception as e:  # noqa: BLE001
                box["res"] = ("error", f"Could not test the sign-in: {e}")

        threading.Thread(target=work, name="test-login", daemon=True).start()

        def check():
            if self.destroyed:
                return
            if "res" not in box:
                self.root.after(150, check)
                return
            level, text = box["res"]
            text = redact(text, st.get("api_key"))
            self._signin_result(level, text)
            self.log_line(f"Test sign-in: {text}", logging.INFO if level == "ok" else logging.WARNING)
            if level == "forbidden":
                self._signin_forbidden = True
                self.last_exit = 3
                messagebox.showwarning("Stopped on 403", text, parent=self.root)
            # short cool-down so the button can't be used to hammer the site
            self.root.after(3000, self._signin_done)

        self.root.after(150, check)

    def _signin_done(self):
        self._signin_busy = False
        self._update_buttons()
```

What to check: Background thread + `after` poll (the window never freezes); asks before testing again after a 403; 3 s cool-down; result text is redacted.

16. Fast mode (1.6): parallel downloads, single DB writer, 403 stop

Workers only do network and file I/O; the thread that owns the database prepares every job and applies every result. That keeps SQLite single-writer exactly as in 1.5.

`e6ai_archiver/fast.py` lines 1-18

```
"""1.6 parallel file downloads ("fast mode").

Design:
  * N worker threads (default 16, 1-64) download files from the static host. Each worker has its own
    keep-alive requests.Session (StaticFetcher); there is no per-request rate limit for static files.
    API requests (gap filling, per-post JSON refresh) still go through Client and its own limiter.
  * Workers never touch SQLite. The calling (main) thread is the single DB writer: it prepares jobs
    (path planning reads the DB), and applies every result (set_file_status / record_file / commit).
  * Each file is streamed to <dest>.part and moved into place with os.replace (atomic): a file at its
    final path is always complete. Interrupted transfers delete their .part; leftovers are swept.
  * Integrity while downloading: the byte count is checked against the expected size; the md5 is
    checked inline only when check_md5 is on (default off; `verify` checks everything afterwards).
  * 403 / 429 / 503 from the static host: that file is marked failed (429/503 are retried a few times
    first) and that worker backs off. After MAX_CONSECUTIVE_403 consecutive 403s the whole run stops
    cleanly (ForbiddenError -> exit code 3).
  * Stop / Pause (control file or GUI) reach every worker: pause blocks before the next request, stop
    aborts transfers within one chunk and deletes their .part files.
"""
```

What to check: The design contract.

`e6ai_archiver/client.py` lines 278-332

```
class StaticFetcher:
    """GETs for file downloads, one keep-alive requests.Session per worker thread.

    No rate limiter and no auth (credentials only ever go to the API host). The descriptive
    User-Agent is kept. A URL on the API host itself is routed through Client.stream() so the API
    limiter still applies to it."""

    def __init__(self, user_agent: str, api_client: "Client | None" = None,
                 timeout: tuple[float, float] = (15.0, 120.0)):
        if any(b in user_agent.lower() for b in BROWSER_UA_MARKERS):
            raise ValueError("Refusing to use a browser-like User-Agent (the site bans these for API tools).")
        self.user_agent, self.api, self.timeout = user_agent, api_client, timeout
        self._local = threading.local()
        self._sessions: list = []
        self._lock = threading.Lock()
        self.request_count = 0

    def session(self) -> requests.Session:
        s = getattr(self._local, "s", None)
        if s is None:
            s = requests.Session()
            adapter = HTTPAdapter(pool_connections=4, pool_maxsize=4, max_retries=0)
            s.mount("https://", adapter)
            s.mount("http://", adapter)
            s.headers.update({"User-Agent": self.user_agent, "Accept": "*/*"})
            self._local.s = s
            with self._lock:
                self._sessions.append(s)
        return s

    def _is_api_host(self, url: str) -> bool:
        return bool(self.api) and url.startswith(self.api.base_url + "/")

    def get(self, url: str) -> requests.Response:
        """Streaming GET. 404 -> NotFoundError; 403/429/503 -> StaticBlocked; other HTTP errors
        -> RuntimeError (5xx: requests.HTTPError so the caller retries); network errors propagate.
        Pause is handled by the download workers; Stop is honoured here."""
        control.check_stop()
        if self._is_api_host(url):
            return self.api.stream(url)
        with self._lock:
            self.request_count += 1
        r = self.session().get(url, stream=True, timeout=self.timeout)
        if r.status_code < 400:
            return r
        status = r.status_code
        ra = retry_after_seconds(r.headers.get("Retry-After"))
        r.close()
        if status == 404:
            raise NotFoundError(url)
        if status in STATIC_BLOCK_STATUSES:
            raise StaticBlocked(status, url, ra)
        if status in RETRY_STATUSES:
            raise requests.HTTPError(f"HTTP {status} from {url}")
        raise RuntimeError(f"HTTP {status} from {url}")
```

What to check: One keep-alive `requests.Session` per worker thread (`threading.local`), the same descriptive UA (browser UAs refused), **no auth** and **no limiter** for the static host; a URL on the API host goes through `Client.stream` (API limiter).

`e6ai_archiver/fast.py` lines 165-189

```
    def submit(self, post_id, md5, ext, size, url, relpath, **meta) -> bool:
        """Queue one pending file. Returns False once the run is being stopped (caller stops feeding)."""
        if self._stopping():
            return False
        a = self.a
        while md5 in self._md5s:  # the same file is already being fetched for another post: wait for it,
            self.pump(0.1)        # then this post reuses it (hardlink) instead of a second download
            if self._stopping():
                return False
        info = a.s.post_info(post_id)
        paths = a.paths_for(info, relpath)
        dest = abspath(a.s.out, paths[0])
        job = Job(post_id, md5, ext, _size(size), url, paths[0], paths[1:], dest,
                  a.s.find_local_copy(md5, post_id), meta)
        while True:
            self.pump(0)
            if self._stopping():
                return False
            try:
                self.jobs.put(job, timeout=0.1)
                self.outstanding += 1
                self._md5s[md5] = self._md5s.get(md5, 0) + 1
                return True
            except queue.Full:
                self.pump(0.1)
```

What to check: Bounded queue (2 x workers). A second post with an md5 already in flight waits for it and is then hard-linked, so the same bytes are never downloaded twice in parallel. Returns False once Stop/halt is set.

`e6ai_archiver/fast.py` lines 222-265

```
    def _apply(self, r: Result) -> None:
        self.outstanding -= 1
        a, j = self.a, r.job
        if self._md5s.get(j.md5, 0) <= 1:
            self._md5s.pop(j.md5, None)
        else:
            self._md5s[j.md5] -= 1
        for k, v in r.stats.items():
            a.stats[k] = a.stats.get(k, 0) + v
        if r.status in ("stopped", "halted"):
            if r.status == "stopped" and self.abort is None:
                self.abort = StopRequested("stop requested")
            return  # stays pending
        if r.status == "fatal":
            if self.abort is None:
                self.abort = r.exc
            self.halt.set()
            return
        self._done_bytes += r.nbytes
        if r.status == "ok":
            had = a.s.file_error(j.post_id)  # 1.8: a file that failed before and is now saved
            a.s.record_file(j.post_id, j.md5, j.primary, r.extras_ok)
            if self.check_md5 and r.nbytes:
                a.s.mark_verified(j.post_id, j.md5)
            if had:
                self._log_failure(j, r, had[1], event="recovered")
        else:
            a.s.set_file_status(j.post_id, r.status)
            n = a.s.set_file_error(j.post_id, r.reason or r.status, j.url)
            self._log_failure(j, r, n)
        if r.forbidden:
            self.consecutive_403 += 1
            if self.consecutive_403 >= self.max_403 and self.abort is None:
                self.abort = ForbiddenError(
                    f"{self.consecutive_403} file downloads in a row were refused with HTTP 403 by the file "
                    f"server. Stopped cleanly; files already saved are kept and the refused ones are marked "
                    f"'failed' (use 'Retry failed' later). Check that your User-Agent/username is right and "
                    f"wait before running again.")
                log.error("%s", self.abort)
                self.halt.set()
        elif r.status in ("ok", "missing"):
            self.consecutive_403 = 0
        a.s.maybe_commit()
        self.on_result(j, r.status, r.nbytes)
```

What to check: Main thread only: stats, `record_file` / `set_file_status`, batched commit, progress. **20 consecutive file 403s** (any 'ok' or 404 resets the count) set `ForbiddenError` (exit 3) and halt all workers; jobs not started stay `pending`.

`e6ai_archiver/fast.py` lines 308-340

```
    def _worker(self, idx: int) -> None:
        k403 = 0
        while True:
            job = self.jobs.get()
            if job is None:
                return
            try:
                if self.halt.is_set():
                    raise _Halt()
                self._checkpoint()  # Pause blocks here; Stop raises
                with self._ilock:
                    self._inflight[idx] = [0, job.size, job.dest.name]
                r = self._fetch(job, idx)
            except StopRequested:
                r = Result(job, "stopped")
            except _Halt:
                r = Result(job, "halted")
            except BaseException as e:  # disk full, drive gone, 403 from the API host...: stop (resumable)
                self.halt.set()         # the other workers stop before their next request
                r = Result(job, "fatal", exc=e)
            finally:
                with self._ilock:
                    self._inflight.pop(idx, None)
            self.results.put(r)
            if r.forbidden or r.throttled:
                k403 = k403 + 1 if r.forbidden else k403
                wait = min(MAX_WORKER_BACKOFF, 2.0 ** max(1, k403))
                try:
                    self._sleep(wait)
                except (StopRequested, _Halt):
                    pass
            elif r.status == "ok":
                k403 = 0
```

What to check: Pause blocks before the next file, Stop raises; any unexpected error halts the others before their next request. After a 403 the worker sleeps 2, 4, 8 ... up to 60 s; after 429/503 likewise.

`e6ai_archiver/fast.py` lines 129-144

```
    def __exit__(self, et, ev, tb):
        try:
            if et is None:
                self.finish()
        finally:
            if et is not None or self.abort is not None:
                self.halt.set()
            for _ in self._threads:
                self._put_sentinel()
            for t in self._threads:
                t.join()
            while not self.results.empty():  # results of the last jobs (after an abort)
                self._apply(self.results.get_nowait())
        if et is None and self.abort is not None:
            raise self.abort
        return False
```

What to check: Every exit path joins all workers and applies their last results, so the DB and the disk agree after Stop, a 403 stop or a crash in one worker.

`e6ai_archiver/cli.py` lines 764-769

```
        if not 1 <= int(a.workers) <= 64:
            raise ValueError(f"--workers must be 1-64 (got {a.workers})")
        a.on_low_disk = str(a.on_low_disk or "pause").strip().lower()
        if a.on_low_disk not in ("pause", "stop"):
            raise ValueError(f"on_low_disk must be pause or stop (got {a.on_low_disk!r})")
        if a.failure_log_max_mb is None or a.failure_log_max_mb < 0:
```

What to check: Validation: workers 1-64, verify action, rate > 0 (no upper limit). Exit 2 before any request.

17. Checksum verification (1.6)

`e6ai_archiver/archiver.py` lines 486-499

```
        def check(row):
            post_id, md5, ext, rel, size = row
            control.checkpoint()
            p = abspath(out, rel or legacy_relpath(md5, ext))
            try:
                st = p.stat().st_size
            except FileNotFoundError:
                return row, p, "absent", 0
            if st == 0:
                return row, p, "zero_byte", 0
            got = md5_of(p)
            if got == md5:
                return row, p, "ok", st
            return row, p, ("truncated" if size and st < int(size) else "md5_mismatch"), st
```

What to check: Runs on the hashing pool (one thread per core by default; hashlib releases the GIL). Verdicts: absent, zero_byte, ok, truncated (smaller than the recorded size) or md5_mismatch.

`e6ai_archiver/archiver.py` lines 509-547

```
                for row, p, verdict, nbytes in pool.map(check, rows):  # results in order; DB writes here only
                    post_id, md5 = row[0], row[1]
                    res["bytes_hashed"] += nbytes
                    if verdict == "ok":
                        res["ok"] += 1
                        self.s.mark_verified(post_id, md5)
                        continue
                    if verdict == "absent":
                        res["absent"] += 1
                    else:
                        res["bad"] += 1
                        res[verdict] += 1
                        try:
                            if action == "quarantine":
                                dst = out / "quarantine" / p.relative_to(out)
                                dst.parent.mkdir(parents=True, exist_ok=True)
                                if dst.exists():
                                    dst = dst.with_name(f"{dst.name}.{int(time.time())}")
                                safe_replace(p, dst)
                                res["quarantined"] += 1
                            else:
                                safe_unlink(p)
                                res["deleted"] += 1
                        except OSError as e:
                            self._local_error(e)
                            raise
                        log.warning("verify: %s %s (post %d) -> %s", verdict.replace("_", " "), p.name, post_id,
                                    "deleted" if action == "delete" else "quarantined")
                    self.s.set_file_status(post_id, "pending")  # not on disk any more: download again
                    self.s.set_file_error(post_id, "verify_" + verdict, str(p))  # 1.7: shown in the report
                    res["requeued"] += 1
                checked += len(rows)
                cursor = rows[-1][0]
                self.s.set_state(key, cursor)
                self.s.commit()
                el = max(1e-6, time.monotonic() - t0)
                log.info("verify: %d / %d checked (%s/s)", checked, total, fmt_bytes(res["bytes_hashed"] / el))
                emit("verify", checked=checked, total=total, bps=round(res["bytes_hashed"] / el),
                     **{k: res[k] for k in ("ok", "bad", "absent")})
```

What to check: Results are applied on the calling thread (single writer). Bad files are deleted or moved to `quarantine/<same path>` (never overwriting an earlier quarantined copy), then set back to `pending` with absent files. The cursor is checkpointed per batch, so an interrupted verify resumes; it is cleared at the end.

`e6ai_archiver/cli.py` lines 470-484

```
def cmd_verify(a, s, c=None):
    """Check every saved file's md5 (parallel, resumable); bad files are deleted or quarantined and
    re-queued. With --redownload (needs the network) the re-queued files are downloaded right away."""
    arch = Archiver(c, s, naming=a.naming, workers=a.workers, check_md5=bool(a.check_md5),
                    min_free_bytes=int(a.min_free_gb * 2**30))
    res = arch.verify_all(action=a.verify_action, hash_workers=a.hash_workers, only_unverified=a.only_new,
                          restart=a.restart)
    out = {"verify": res}
    if c is not None:
        n, b = s.pending_summary()
        log.info("re-downloading %d missing / damaged files (%s)", n, fmt_bytes(b))
        arch.download_pending()
        out["redownload"] = arch.stats
        out["requests"] = c.request_count
    print(json.dumps(out, indent=2))
```

What to check: `verify` is offline unless `--redownload` (the panel turns it on); with it, the re-queued files are fetched by the normal parallel downloader. `full` runs the same check on not-yet-verified files when it finishes (`--verify-after`, default on).

18. Completeness report (1.7)

Read-only: the report loads the export into a TEMP table, joins it with the archive tables, and writes only its own report files and one `state` row.

`e6ai_archiver/report.py` lines 154-182

```
def build_report(store, spec=None, naming=None, export_path=None, check_disk: bool = True,
                 write: bool = True, stamp: str | None = None) -> dict:
    from .exports import good_export_path
    from .store import abspath, legacy_relpath
    out = Path(store.out)
    db = store.db
    path = Path(export_path) if export_path else good_export_path(out, "posts")
    if path is None or not path.is_file():
        raise NoExportError("no posts export to compare with: run `sync-exports` (or `exports`) first, or pass "
                            "--export PATH to a posts-YYYY-MM-DD.csv.gz from https://e6ai.net/db_exports")
    t0 = time.monotonic()
    emit("phase", name="completeness report")
    log.info("report: comparing the archive with %s", path)
    exp = _load_export(db, path)
    exp["path"], exp["date"] = str(path), _export_date(path, out)
    exp["deleted"] = db.execute("SELECT COUNT(*) FROM rep_export WHERE deleted=1").fetchone()[0]

    # filters / tier (what the user chose not to download)
    match_sql, match_args = (spec.where(candidates=False) if spec is not None else ("1", []))
    filtered = spec is not None and spec.filtered()
    in_tier = None
    if spec is not None and (spec.top is not None or spec.limit_gb is not None):
        from .priority import PriorityQueue
        pq = PriorityQueue(store, spec)
        pq.ensure()
        end = pq.tier_end()
        in_tier = {r[0] for r in db.execute("SELECT post_id FROM queue WHERE rank <= ?", (end,))}
    matching = ({r[0] for r in db.execute(f"SELECT id FROM posts WHERE {match_sql}", match_args)}
                if filtered else None)
```

What to check: No export = `NoExportError` (`report` exits 1; `full` skips with a note). Filters use the same SQL as the download queue (`Spec.where(candidates=False)`), and the tier the same `PriorityQueue` ranks, so "skipped" matches what a download would leave out.

`e6ai_archiver/report.py` lines 209-248

```
    for (pid, md5, ext, size, tags, status, rel, pmd5, purl, tag_cats, vmd5, err, _detail) in db.execute(q):
        reason = ""
        if status is None:
            bucket, reason = "missing", "not_in_archive"
        elif status == "ok":
            rel = rel or legacy_relpath(pmd5 or md5, ext)
            known.add(_norm(rel))
            on_disk = True
            if check_disk:
                try:
                    on_disk = abspath(out, rel).stat().st_size > 0
                except OSError:
                    on_disk = False
            if pmd5 and pmd5 != md5:
                bucket, reason = "missing", "replaced_on_site"
            elif not on_disk:
                bucket, reason = "missing", "file_missing_on_disk"
            else:
                bucket = "verified" if vmd5 and vmd5 == md5 else "unverified"
        elif status == "pending":
            if matching is not None and pid not in matching:
                bucket, reason = "skipped", "filter"
            elif in_tier is not None and pid not in in_tier:
                bucket, reason = "skipped", "tier_limit"
            else:
                bucket, reason = "missing", "not_downloaded_yet"
        else:
            bucket = "failed"
            reason = err if status in ("failed", "md5_mismatch") and err else FAILED_STATUS_REASON.get(status,
status)
        counts[bucket] += 1
        bytes_[bucket] += size or 0
        if reason:
            reasons[bucket][reason] += 1
        ds = directors_of(tag_cats, tags)
        primary = ds[0] if ds else NO_DIRECTOR
        by_dir[primary]["expected"] += 1
        by_dir[primary][bucket] += 1
        if bucket in ("missing", "failed", "skipped"):
            rows_out.append((pid, md5, ext, size or "", " ".join(ds) or NO_DIRECTOR, bucket, reason, err or "",
                             purl or file_url(md5, ext)))
```

What to check: One pass over the export rows in id order. Saved files are stat()ed (size > 0) unless `--no-disk-check`; an md5 change on the site counts as missing; a pending post outside the filters / tier is skipped; any other status is failed, with the recorded reason when there is one.

`e6ai_archiver/report.py` lines 130-151

```
def _scan_files(out: Path, known: set) -> tuple[list, int, int]:
    """Files under files/ and data/ that no post points at; .part leftovers; quarantined files."""
    extra, parts = [], 0
    for top in (FILES_DIR, LEGACY_DIR):
        base = out / top
        if not base.is_dir():
            continue
        for root, _dirs, names in os.walk(base):
            for name in names:
                p = Path(root) / name
                rel = _norm(os.path.relpath(p, out))
                if name.endswith(".part"):
                    parts += 1
                    continue
                if rel not in known:
                    try:
                        size = p.stat().st_size
                    except OSError:
                        size = 0
                    extra.append((rel, size))
    quarantined = sum(len(n) for _r, _d, n in os.walk(out / "quarantine")) if (out / "quarantine").is_dir() else 0
    return sorted(extra), parts, quarantined
```

What to check: Extra files are only listed (never deleted); `.part` leftovers and quarantined files are counted separately.

`e6ai_archiver/store.py` lines 692-698

```
    def set_file_error(self, post_id: int, reason: str, detail: str | None = None) -> int:
        """1.7: remember why the last attempt at this file failed (http_403, http_404, md5_mismatch ...).
        1.8: also counts the failures (until the file is saved); returns that count."""
        self.db.execute("INSERT INTO file_errors(post_id, reason, detail, at, attempts) VALUES(?,?,?,?,1) "
                        "ON CONFLICT(post_id) DO UPDATE SET reason=excluded.reason, detail=excluded.detail, "
                        "at=excluded.at, attempts=file_errors.attempts+1", (post_id, reason, detail, time.time()))
        return self.db.execute("SELECT attempts FROM file_errors WHERE post_id=?", (post_id,)).fetchone()[0]
```

What to check: New `file_errors` table (one row per post, last reason wins); `record_file` deletes the row when the file is finally saved.

`e6ai_archiver/cli.py` lines 166-182

```
def _report_after(a, c, s, out: dict):
    """1.7/1.8: completeness report at the end of full / retry-failed. Its failure is the only thing
    that changes the exit code; the downloads themselves already finished."""
    if not a.report_after:
        return None
    t = _tags_for_report(a, s, c)
    out["tags_export"] = t
    try:
        out["report"] = _report_brief(run_report(a, s))
    except NoExportError as e:  # e.g. the export was unavailable and the API crawl was used
        log.warning("completeness report skipped: %s", e)
        out["report"] = {"skipped": str(e)}
    except (ReportError, OSError, sqlite3.Error, RuntimeError) as e:
        log.error("completeness report failed: %s", e)
        out["report"] = {"error": str(e)}
        return EXIT_ERROR
    return None
```

What to check: `full` (after the download and the checksum pass) and `retry-failed`: 1.8 checks the tags export first. Only a failing report changes the exit code (1).

`e6ai_archiver/exports.py` lines 121-137

```
    # 1.7: the CDN in front of static1 can keep serving yesterday's file for the plain URL (seen live
    # on 2026-11-04: Cloudflare cache-control max-age=2678400, last-modified a day behind the listing).
    # On a checksum mismatch, fetch once more with a cache-busting query (?<checksum prefix>, a new
    # cache key; the file itself is unchanged) before giving up and keeping the last good copy.
    urls = [url] + ([f"{url}{'&' if '?' in url else '?'}{want[:12]}"] if want else [])
    for n, u in enumerate(urls):
        digest = _fetch_to(client, u, tmp, dest.name, int(entry.get("file_size") or 0), retries)
        if not want or digest == want:
            break
        safe_unlink(tmp)
        if n + 1 < len(urls):
            log.warning("export %s: sha256 %s does not match the listing (%s...); probably a stale CDN copy - "
                        "retrying once with a cache-busting URL", dest.name, digest[:12], want[:12])
            continue
        raise RuntimeError(f"sha256 mismatch for {dest.name}: got {digest}, expected {want} "
                           "(the export may have been regenerated mid-download, or the CDN serves a stale "
                           "copy; rerun later)")
```

What to check: Fix for a stale copy served by the CDN (seen live on 2026-11-04): one retry with a cache-busting query, then a clear error; the `.part` file is removed either way.

19. Tags export, failure log and retry-failed (1.8)

Tags export: fetched with every posts export, and checked before every report. Failure log: written by the single DB-writer thread only. retry-failed: selects only failed posts and restores their status if stopped.

`e6ai_archiver/exports.py` lines 519-530

```
def with_tags(names: str | None) -> str | None:
    """1.8: the tags export always comes with the posts export (tag categories = director folders and
    per-director report counts). 'all' and 'core' already include it."""
    spec = (names or "all").strip().lower()
    if spec in ("all", "core"):
        return names
    parts = [x.strip() for x in spec.replace(" ", ",").split(",") if x.strip()]
    if "posts" in parts and "tags" not in parts:
        log.info("adding the tags export to %s (director folders and per-director report counts need it)",
                 ",".join(parts))
        parts.append("tags")
    return ",".join(parts)
```

What to check: `posts` always brings `tags` (`all` / `core` already include it).

`e6ai_archiver/exports.py` lines 609-611

```
    tags_only = (recent and "tags" in wanted_exports(names, list(APPLY_ORDER))
                 and not (export_dir(out) / "tags.csv.gz").is_file() and not store.has_tag_categories())
    if recent and not tags_only:
```

What to check: Inside the once-a-day window a missing tags export is still fetched (tags only; `exports_checked_at` is not moved).

`e6ai_archiver/exports.py` lines 676-739

```
def ensure_tags(store, client_factory=None, fetch: bool = True) -> dict:
    """1.8: before a report, make sure tag categories are loaded and not older than the posts export.

    1. If there is no good local tags export, or it is clearly older than the local posts export
       (manifest dates more than 6 h apart), fetch it (one listing request + one file, anonymous,
       sha256-verified with the cache-busting retry) - only when `fetch` and a client factory are given.
    2. Load the local tags export into tag_categories if it differs from what was loaded last.
    3. Fill tag_cats of export-seeded posts that have none yet.
    Never raises for network problems: the result carries a clear note instead."""
    out = store.out
    res = {"fetched": False, "loaded": False, "categories": 0, "note": ""}
    man = read_manifest(out)
    local = good_export_path(out, "tags")
    t_tags = _parse_ts(man.get("tags.csv.gz", {}).get("updated_at"))
    t_posts = _parse_ts(man.get("posts.csv.gz", {}).get("updated_at"))
    stale = local is None or bool(t_posts and t_tags and t_tags < t_posts - 6 * 3600)
    if stale and fetch and client_factory is not None:
        why = "missing" if local is None else "older than the posts export"
        log.info("tags export %s: fetching it (anonymous, one listing + one file)", why)
        emit("phase", name="fetching the tags export")
        c = None
        try:
            c = client_factory()
            listing = list_exports(c)
            e = next((x for x in (listing or []) if isinstance(x, dict) and x.get("name") == "tags"), None)
            if e is None:
                res["note"] = "the site's export listing has no tags export"
            else:
                export_dir(out).mkdir(parents=True, exist_ok=True)
                r = {"downloaded": [], "unchanged": [], "failed": {}}
                _sync_one(c, out, e, man, r)
                write_manifest(out, man)
                if r["failed"]:
                    res["note"] = f"tags export download failed: {r['failed']['tags']}"
                else:
                    res["fetched"] = "tags" in r["downloaded"]
        except _FATAL:
            raise
        except (requests.RequestException, RuntimeError, ValueError, OSError) as ex:
            res["note"] = f"could not fetch the tags export ({ex})"
        finally:
            if c is not None and hasattr(c, "close"):
                c.close()
        if res["note"]:
            log.warning("%s; using what is available", res["note"])
    elif stale and local is None:
        res["note"] = "no tags export in the archive and fetching is off"
    p = good_export_path(out, "tags")
    if p is not None:
        applied = store.get_state("applied_exports") or {}
        sha = sha256_of(p)
        if applied.get("tags") != sha or not store.has_tag_categories():
            load_tags_export(store, p)
            applied["tags"] = sha
            store.set_state("applied_exports", applied)
            store.commit()
            res["loaded"] = True
        else:
            categorize_seeded(store)
    res["categories"] = store.db.execute("SELECT COUNT(*) FROM tag_categories").fetchone()[0]
    if not res["categories"]:
        res["note"] = TAGS_MISSING_NOTE + (f" [{res['note']}]" if res["note"] else "")
        log.warning("%s", res["note"])
    return res
```

What to check: Before a report: fetch only if missing or >6 h older than posts, via a client factory (the CLI passes an **anonymous** client: `api_key=None, login=None`). Network problems become a note; 403 / disk errors (`_FATAL`) still stop. Loads the export only when its sha256 changed.

`e6ai_archiver/cli.py` lines 107-110

```
def _anon_client(a) -> Client:
    """1.8: anonymous client for the tags export only (never the login / API key; few retries)."""
    anon = argparse.Namespace(**{**vars(a), "api_key": None, "login": None})
    return make_client(anon, max_retries=2, max_outage_hours=0.05)
```

What to check: The tags fetch never carries the API key or login, even when the user signed in; short retry budget (2 tries, 3 min outage).

`e6ai_archiver/failures.py` lines 53-62

```
def safe_url(url: str | None) -> str:
    """Scheme://host/path only (no user-info, query or fragment), then redacted."""
    if not url:
        return ""
    try:
        p = urlsplit(url)
        url = urlunsplit((p.scheme, p.hostname or "", p.path, "", "")) if p.scheme else url.split("?")[0]
    except ValueError:
        url = url.split("?")[0]
    return redact(url)
```

What to check: No user-info, query string or fragment, then the secret redactor: an API key can't reach the CSV/log.

`e6ai_archiver/failures.py` lines 128-149

```
    def record(self, *, post_id: int, md5: str = "", ext: str = "", director: str = "", status: str = "",
               reason: str = "", tries: int = 1, failures_total: int = 1, url: str = "",
               event: str = "failed") -> None:
        try:
            self._open()
            u = safe_url(url)
            http = http_status_of(reason)
            self._w.writerow((_now(), self.run, event, post_id, md5 or "", ext or "", director or "", status or "",
                              reason or "", http, tries, failures_total, u))
            if event == "failed":
                self.failed[reason or status] += 1
                self._log.write(f"{_now()}  FAILED     post {post_id}  {reason or status}"
                                f"{' (HTTP ' + http + ')' if http else ''}  tries {tries} this run, "
                                f"failure #{failures_total}  director {director or '?'}  {u}\n")
            else:
                self.recovered += 1
                self._log.write(f"{_now()}  RECOVERED  post {post_id}  saved after {failures_total} earlier "
                                f"failure(s)  director {director or '?'}  {u}\n")
            self._csv.flush()
            self._log.flush()
        except OSError as e:  # the log must never stop a download run
            log.warning("could not write %s: %s", FAIL_CSV, e)
```

What to check: Append-only CSV + readable log, flushed per row; an OSError is logged and never stops the run.

`e6ai_archiver/fast.py` lines 267-284

```
    def _log_failure(self, j: Job, r: Result, total: int, event: str = "failed") -> None:
        """1.8: failed-downloads.csv / .log (+ one line in the run log / panel)."""
        from .failures import directors_of, safe_url
        a = self.a
        try:
            director = directors_of(a.s, j.post_id, getattr(a.naming, "director_exclude", ()))
        except Exception:  # noqa: BLE001 - the log must never stop the run
            director = ""
        fl = getattr(a.s, "failures", None)
        if fl is not None:
            fl.record(post_id=j.post_id, md5=j.md5, ext=j.ext, director=director, status=r.status,
                      reason=r.reason or r.status, tries=r.tries, failures_total=total, url=j.url, event=event)
        if event == "failed":
            log.warning("download failed: post %d (%s, %s) - %s, %d tr%s this run, failure #%d -> %s", j.post_id,
                        director or "?", j.md5, r.reason or r.status, r.tries, "y" if r.tries == 1 else "ies",
                        total, safe_url(j.url))
        emit("file_failed" if event == "failed" else "file_recovered", post_id=j.post_id,
             reason=r.reason or r.status, tries=r.tries, failures_total=total)
```

What to check: Called from `_apply` (main thread): one CSV/log row, one panel-log warning, one GUI event per failure or recovery.

`e6ai_archiver/failures.py` lines 326-362

```
def select_retry(store, reasons=None, from_csv=None) -> dict:
    """Which posts `retry-failed` re-attempts: {selected: [ids], by_reason: Counter, skipped: Counter}."""
    from .store import RETRYABLE_STATUSES, abspath
    toks = parse_reasons(reasons)
    sel, by, skipped = [], Counter(), Counter()
    if from_csv is None:
        for pid, reason, _st in store.failed_files():
            if reason_matches(reason, toks):
                sel.append(pid)
                by[reason or "unknown"] += 1
            else:
                skipped["other_reason"] += 1
        return {"selected": sel, "by_reason": by, "skipped": skipped}
    for pid in read_ids_csv(from_csv):
        row = store.db.execute("SELECT p.file_status, p.file_url, p.file_path, fe.reason FROM posts p LEFT JOIN "
                               "file_errors fe ON fe.post_id=p.id WHERE p.id=?", (pid,)).fetchone()
        if row is None:
            skipped["not_in_database"] += 1
            continue
        st, url, rel, reason = row
        if not url:
            skipped["no_file_url (deleted or hidden)"] += 1
            continue
        if st == "ok":
            if rel and abspath(store.out, rel).exists():
                skipped["already_saved"] += 1
                continue
            reason = reason or "gone_from_disk"
        elif st not in RETRYABLE_STATUSES and st != "pending":
            skipped[f"status_{st}"] += 1
            continue
        if not reason_matches(reason if reason != "gone_from_disk" else None, toks):
            skipped["other_reason"] += 1
            continue
        sel.append(pid)
        by[reason or ("not_downloaded_yet" if st == "pending" else "unknown")] += 1
    return {"selected": sel, "by_reason": by, "skipped": skipped}
```

What to check: Status-based selection (failed / missing / md5_mismatch), optional reason groups, optional CSV list; every skipped post is counted by why.

`e6ai_archiver/cli.py` lines 210-244

```
def cmd_retry_failed(a, c, s):
    """1.8: re-attempt only files currently marked failed / missing / md5_mismatch (optionally only some
    failure reasons, or the posts listed in a CSV such as missing.csv). Nothing else is downloaded."""
    sel = select_retry(s, a.reason, a.from_csv)
    ids = sel["selected"]
    summary = {"selected": len(ids), "by_reason": dict(sel["by_reason"].most_common()),
               "skipped": dict(sel["skipped"]), "reason_filter": a.reason or "all",
               "from_csv": a.from_csv or None}
    if a.dry_run or not ids:
        if not ids:
            log.info("retry-failed: no failed files match (reason filter: %s)", a.reason or "all")
        summary["dry_run"] = bool(a.dry_run)
        print(json.dumps({"retry_failed": summary}, indent=2))
        return None
    prior = dict(_status_map(s, ids))
    log.info("retry-failed: %d file(s) selected (%s)", len(ids),
             ", ".join(f"{k} {v}" for k, v in summary["by_reason"].items()))
    s.set_pending(ids)
    try:
        counts = _archiver(a, c, s, spec=None).download_ids(ids)
    finally:
        # stopped part-way (Stop / 403 streak / disk): files not attempted keep their old failed status
        n = s.restore_failed(ids, prior)
        if n:
            log.info("retry-failed: %d file(s) not attempted this time keep their failed status", n)
    summary["recovered"] = counts.get("ok", 0)
    summary["still_failed"] = sum(counts.get(k, 0) for k in RETRYABLE_STATUSES)
    summary["status_after"] = counts
    summary["failure_log"] = {"csv": str(s.out / FAIL_CSV), "log": str(s.out / FAIL_LOG)}
    out = {"retry_failed": summary}
    out["requests"] = c.request_count
    rc = _report_after(a, c, s, out)
    out["requests"] = c.request_count  # (key order kept: the report stays last)
    print(json.dumps(out, indent=2, default=str))
    return rc
```

What to check: Only the selected ids are set pending and downloaded (`download_ids`); `finally` puts not-reached ones back to their failed status. Then the failure log and the report are updated.

20. Dark mode and quality of life (1.9)

Theme: pure functions (normalize / resolve / palette / contrast) plus one `apply` that styles every ttk widget; the panel is built hidden and shown after styling. Low disk: workers wait instead of failing. Failure-log trimming archives before it rewrites.

`e6ai_archiver/theme.py` lines 71-82

```
def system_prefers_dark(reader=None, platform: str | None = None) -> bool:
    """Windows: HKCU\\...\\Themes\\Personalize AppsUseLightTheme == 0 means dark. Elsewhere (or when the
    value cannot be read): light. `reader` returns the DWORD (tests pass a fake)."""
    platform = platform or sys.platform
    if reader is None:
        if not platform.startswith("win"):
            return False
        reader = _winreg_reader
    try:
        return int(reader()) == 0
    except Exception:  # noqa: BLE001 - missing key / old Windows / no winreg: light
        return False
```

What to check: Follow Windows reads `AppsUseLightTheme` (0 = dark). Any error, or a non-Windows OS, means light. The reader is injectable, so the tests run the Windows branch on Linux.

`e6ai_archiver/theme.py` lines 102-108

```
def contrast(a: str, b: str) -> float:
    """WCAG contrast ratio of two #rrggbb colours (1 .. 21)."""
    def lum(c):
        r, g, b_ = (x / 12.92 if x <= 0.03928 else ((x + 0.055) / 1.055) ** 2.4 for x in _hex(c))
        return 0.2126 * r + 0.7152 * g + 0.0722 * b_
    la, lb = sorted((lum(a), lum(b)), reverse=True)
    return (la + 0.05) / (lb + 0.05)
```

What to check: WCAG contrast ratio; the tests require at least 4.5:1 for normal text and every log colour on both palettes.

`e6ai_archiver/panel_ui.py` lines 1682-1714

```
def run_panel(settings_path: Path, prefill: dict | None = None, on_ready=None) -> int:
    try:
        root = make_root()
    except tk.TclError as e:
        print(f"The control panel needs a display ({e}). Use the command line instead: e6ai-archiver --help",
              file=sys.stderr)
        return 1
    ui = cp.load_ui_state()
    root.withdraw()                      # 1.9: no white flash: show the window once it is themed
    try:
        root.configure(background=th.palette(cp.load_settings(Path(settings_path))[0].get("theme"))["bg"])
    except Exception:  # noqa: BLE001
        pass
    place_window(root, ui)
    panel = ControlPanel(root, settings_path, prefill)
    root.deiconify()
    try:
        panel.nb.select(int(ui.get("tab", 0)))
        def place_sash():
            root.update_idletasks()
            width = panel.paned.winfo_width()
            if width < 200:                      # not laid out yet
                root.after(50, place_sash)
                return
            want = int(ui.get("sash") or 0) or int(width * 0.55)   # options side gets a little more room
            panel.paned.sashpos(0, min(max(want, int(width * 0.3)), int(width * 0.75)))
        root.after(50, place_sash)
    except (tk.TclError, ValueError):
        pass
    if on_ready:
        on_ready(panel)
    try:
        root.mainloop()
```

What to check: No white flash: the root is withdrawn, themed, filled, then shown. The saved `[gui] theme` is applied before the first paint.

`e6ai_archiver/fast.py` lines 351-382

```
    def _wait_for_space(self, out: Path, need: int) -> None:
        """1.9: before each file, make sure it fits above the --min-free-gb reserve. on_low_disk=stop: stop
        cleanly (DiskFullError, exit 4, as in 1.8). pause (default): every worker waits here, the run
        reports 'paused: low disk space' (panel notification) and continues by itself as soon as space is
        freed; the panel's Resume button asks for an immediate re-check (control word 'recheck')."""
        a = self.a
        reserve = int(getattr(a, "min_free_bytes", 0) or 0)
        if getattr(a, "on_low_disk", "pause") == "stop":
            ensure_free(out, need, reserve)
            return
        while True:
            free = platform_utils.free_bytes(out)
            if free >= need + reserve:
                with self._disk_lock:
                    if self._disk_waiting:
                        self._disk_waiting = False
                        log.info("disk space OK again (%.1f GiB free): downloads continue", free / 2**30)
                        emit("disk_ok", free=free)
                        emit("phase", name="resumed")
                return
            with self._disk_lock:
                first = not self._disk_waiting
                self._disk_waiting = True
                self.disk_pauses += first
            if first:
                log.warning("PAUSED - low disk space at %s: %.1f GiB free, the next file needs %.1f MiB plus "
                            "the %.1f GiB reserve (--min-free-gb). Free some space (or lower the reserve and "
                            "restart); downloads continue by themselves.", out, free / 2**30, need / 2**20,
                            reserve / 2**30)
                emit("disk_low", free=free, need=need, reserve=reserve)
                emit("phase", name="paused: low disk space")
            self._disk_sleep(DISK_RECHECK_S)
```

What to check: `on_low_disk = pause` (default): every worker waits here, re-checking free space every 2 s; `control.recheck()` (the panel's Resume) wakes it at once; Stop raises as usual. `stop` keeps the 1.8 exit 4.

`e6ai_archiver/failures.py` lines 208-264

```
def trim_failure_logs(out, max_bytes: int, keep_ids) -> dict:
    """Trim failed-downloads.csv / .log when larger than max_bytes. Rows / lines of posts in keep_ids
    (still failed) always stay; the rest are appended to the gzip archives first (a crash in between
    leaves duplicates, never a loss). Returns {"csv": rows moved, "log": lines moved} for trimmed files."""
    import gzip
    import io
    out, keep = Path(out), {int(i) for i in keep_ids}
    done = {}
    cp = out / FAIL_CSV
    if max_bytes > 0 and cp.is_file() and cp.stat().st_size > max_bytes:
        with open(cp, encoding="utf-8", newline="") as f:
            rows = list(csv.reader(f))
        header, body = (rows[0], rows[1:]) if rows and rows[0] and rows[0][0] == "time" else (list(COLUMNS), rows)
        pi = header.index("post_id") if "post_id" in header else 3
        def kept(r):
            try:
                return int(r[pi]) in keep
            except (ValueError, IndexError):
                return False
        stay = [r for r in body if kept(r)]
        move = [r for r in body if not kept(r)]
        if move:
            arch = out / FAIL_CSV_ARCHIVE
            new = not arch.exists()
            buf = io.StringIO()
            w = csv.writer(buf)
            if new:
                w.writerow(header)
            w.writerows(move)
            with gzip.open(arch, "at", encoding="utf-8", newline="") as g:
                g.write(buf.getvalue())
            buf = io.StringIO()
            w = csv.writer(buf)
            w.writerow(header)
            w.writerows(stay)
            _replace_text(cp, buf.getvalue())
            done["csv"] = len(move)
            log.info("%s was over %.0f MB: %d row(s) of files that are not failed any more moved to %s; "
                     "%d row(s) of %d still-failed file(s) kept", FAIL_CSV, max_bytes / 2**20, len(move),
                     FAIL_CSV_ARCHIVE, len(stay), len({r[pi] for r in stay}))
    lp = out / FAIL_LOG
    if max_bytes > 0 and lp.is_file() and lp.stat().st_size > max_bytes:
        with open(lp, encoding="utf-8", newline="") as f:
            lines = f.readlines()
        def keep_line(s):
            m = _POST_RE.search(s)
            return bool(m) and " FAILED " in s and int(m.group(1)) in keep
        stay = [s for s in lines if keep_line(s)]
        move = [s for s in lines if not keep_line(s)]
        if move:
            with gzip.open(out / FAIL_LOG_ARCHIVE, "at", encoding="utf-8", newline="") as g:
                g.write("".join(move))
            note = (f"=== trimmed {_now()}: {len(move)} older line(s) moved to {FAIL_LOG_ARCHIVE}; the lines "
                    f"of files that are still failed are kept below\n")
            _replace_text(lp, note + "".join(stay))
            done["log"] = len(move)
    return done
```

What to check: Rows of posts still failed / missing / md5_mismatch are kept; every other row is appended to the gzip archive **before** the file is rewritten (atomic replace), so a crash in between leaves duplicates, never a loss.

`e6ai_archiver/control_panel.py` lines 1446-1478

```
def resume_offer(status: dict | None, dismissed: str | None = None, alive=pid_alive) -> dict | None:
    """Should the panel offer 'Resume' at launch? Only for a download run that did not finish (no end
    recorded, or exit 1 / 4 / 6 / 7 / 130) while files are still to download, not one still running and
    not one the user dismissed. 403 / sign-in / settings / in-use exits need attention first, never a
    one-click resume."""
    lr = (status or {}).get("last_run")
    if not isinstance(lr, dict) or lr.get("cmd") not in RESUME_MODE:
        return None
    if dismissed and dismissed == lr.get("started"):
        return None
    ended, code = lr.get("ended"), lr.get("code")
    if ended is None:
        if alive(lr.get("pid")):
            return None            # still running (another window / a console)
        why = "was interrupted (the window was closed, the PC slept or crashed, or power was lost)"
    elif code in RESUMABLE_CODES:
        why = {1: "ended with an error", 4: "stopped because the disk was full", 6: "lost the archive drive",
               7: "hit a database error (run 'Repair the database' first if it happens again)",
               130: "was stopped"}.get(code, "did not finish")
    else:
        return None
    files = status.get("failed_files", 0) if lr["cmd"] == "retry-failed" else \
        status.get("pending_files", 0) + status.get("failed_files", 0)
    nbytes = status.get("failed_bytes", 0) if lr["cmd"] == "retry-failed" else \
        status.get("pending_bytes", 0) + status.get("failed_bytes", 0)
    if not files:
        return None
    started = str(lr.get("started") or "")
    when = started[:16].replace("T", " ")
    return {"cmd": lr["cmd"], "mode": RESUME_MODE[lr["cmd"]], "started": started, "files": files,
            "bytes": nbytes, "code": code,
            "text": f"The last run ('{lr['cmd']}', started {when}) {why}. {files:,} file(s) "
                    f"({_gib(nbytes)}) are still to download. Resume it with the same settings?"}
```

What to check: The banner only appears for an interrupted run (no clean end, or an error / disk / storage / DB / Stop exit) whose process is gone and that still has files left; 'Don't ask' is remembered per run.

`e6ai_archiver/control_panel.py` lines 1495-1510

```
def disk_check(need: int | None, free: int | None, reserve_gb) -> tuple[bool, str]:
    """Before Start: will the files still to download fit above the reserve (+ a small margin)?"""
    if need is None or free is None:
        return True, ""
    try:
        reserve = int(float(reserve_gb or 0) * 2**30)
    except ValueError:
        reserve = 0
    want = int(need * (1 + DISK_MARGIN)) + reserve
    if free >= want:
        return True, f"{_gib(need)} still to download; {_gib(free)} free - fits."
    return False, (f"About {_gib(need)} of files are still to download (export sizes), plus the "
                   f"{_gib(reserve)} you keep free and a {DISK_MARGIN:.0%} margin = {_gib(want)}, but the drive "
                   f"has only {_gib(free)} free (short by {_gib(want - free)}).\n\nThe run will pause when the "
                   f"drive gets full (or stop, if you chose that) and can continue after you free space or move "
                   f"the archive folder to a bigger drive.")
```

What to check: Pre-Start estimate: remaining bytes vs free space minus the reserve; the panel asks before starting when it will not fit.

`e6ai_archiver/notify.py` lines 60-96

```
def notify(title: str, message: str, *, urgent: bool = False, platform: str | None = None,
           which=shutil.which, popen=subprocess.Popen) -> str | None:
    platform = platform or sys.platform
    title, message = _clean(title, 120), _clean(message, 380)
    try:
        if platform.startswith("win"):
            exe = which("powershell") or which("powershell.exe")
            if not exe:
                return None
            kw = {}
            if os.name == "nt":
                kw["creationflags"] = 0x08000000  # CREATE_NO_WINDOW
            p = popen([exe, "-NoProfile", "-NonInteractive", "-ExecutionPolicy", "Bypass", "-Command", "-"],
                      stdin=subprocess.PIPE, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL,
                      text=True, encoding="utf-8", **kw)
            try:
                p.stdin.write(windows_script(title, message))
                p.stdin.close()
            except (OSError, ValueError, AttributeError):
                pass
            return "windows-toast"
        if platform == "darwin":
            exe = which("osascript")
            if not exe:
                return None
            esc = lambda s: s.replace("\\", "\\\\").replace('"', '\\"')  # noqa: E731
            popen([exe, "-e", f'display notification "{esc(message)}" with title "{esc(title)}"'],
                  stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
            return "osascript"
        exe = which("notify-send")
        if not exe or not (os.environ.get("DISPLAY") or os.environ.get("WAYLAND_DISPLAY")):
            return None
        popen([exe, "-a", "e6ai archiver", "-u", "critical" if urgent else "normal", "--", title, message],
              stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
        return "notify-send"
    except (OSError, ValueError):
        return None
```

What to check: Toast, then tray balloon (Windows, via PowerShell with quoted arguments), osascript (macOS), notify-send (Linux); any failure returns False and the panel shows its own small window. Never raises.

21. Finding Python on Windows (1.9.1)

The owner's first run stopped at `py -3 -m venv .venv` (`py` not recognized). `run.ps1` now dot-sources `find_python.ps1`: candidates are found with Get-Command / Test-Path, each one is run once, and only a printed version >= 3.11 counts.

`find_python.ps1` lines 24-42

```
function Get-PythonVersion {
    # Runs the candidate and returns its [version], or $null if it does not run / is not Python.
    param([string]$Exe, [string[]]$PreArgs = @())
    if (-not $Exe) { return $null }
    $ErrorActionPreference = "Continue"   # local: a candidate writing to stderr must not stop run.ps1
    $out = $null
    $code = 1
    try {
        $out = & $Exe @PreArgs -c "import sys; print('E6AI-PY %d.%d.%d' % tuple(sys.version_info[:3]))" 2>$null
        $code = $LASTEXITCODE
    } catch {
        return $null
    }
    if ($code -ne 0) { return $null }
    foreach ($line in @($out)) {
        if ("$line" -match '^E6AI-PY (\d+\.\d+\.\d+)\s*$') { return [version]$Matches[1] }
    }
    return $null
}
```

What to check: Local `$ErrorActionPreference = Continue` so stderr from a candidate cannot stop run.ps1; a missing file is caught; the version must come back on stdout with the `E6AI-PY` marker and exit 0. The Store shortcut prints only to stderr and exits 9009, so it returns `$null`.

`find_python.ps1` lines 85-117

```
function Get-PythonCandidateList {
    $list = @()
    # 2. py -3 (the launcher): on PATH, or in its usual places even when PATH was not updated.
    $launchers = @()
    foreach ($cmd in @(Get-Command "py" -CommandType Application -All -ErrorAction SilentlyContinue)) {
        $launchers += $cmd.Path
    }
    if ($env:SystemRoot) { $launchers += (Join-Path $env:SystemRoot "py.exe") }
    if ($env:LOCALAPPDATA) {
        $launchers += (Join-Path (Join-Path (Join-Path (Join-Path $env:LOCALAPPDATA "Programs") "Python") "Launcher")
"py.exe")
    }
    foreach ($p in $launchers) {
        if ($p -and (Test-Path -LiteralPath $p)) { $list += New-PythonCandidate -Exe $p -PreArgs @("-3") -Label "py
-3 ($p)" }
    }
    # 3. / 4. python and python3 on PATH (all of them: the Store shortcut may come first).
    foreach ($name in @("python", "python3")) {
        foreach ($cmd in @(Get-Command $name -CommandType Application -All -ErrorAction SilentlyContinue)) {
            $list += New-PythonCandidate -Exe $cmd.Path -PreArgs @() -Label "$name ($($cmd.Path))"
        }
    }
    # 5. the usual install folders.
    foreach ($exe in @(Get-InstallDirPython)) {
        $list += New-PythonCandidate -Exe $exe -PreArgs @() -Label $exe
    }
    # Drop duplicates (same file + same arguments), keeping the first.
    $seen = @{}
    $unique = @()
    foreach ($c in $list) {
        $key = ($c.Exe + "|" + ($c.PreArgs -join " ")).ToLowerInvariant()
        if (-not $seen.ContainsKey($key)) { $seen[$key] = $true; $unique += $c }
    }
    return $unique
}
```

What to check: Order: py -3 (only if the launcher file exists), every python / python3 on PATH (`-All`, so a Store shortcut earlier on PATH does not hide a real one), then the install folders newest first; duplicates dropped.

`find_python.ps1` lines 119-141

```
function Find-Python {
    # Returns the first candidate that runs and is >= $E6aiMinPython (with .Version set), else $null.
    # Every candidate tried is recorded in $script:E6aiPythonTried for the error message.
    param([object[]]$Candidates = $null)
    if ($null -eq $Candidates) { $Candidates = @(Get-PythonCandidateList) }
    $script:E6aiPythonTried = @()
    foreach ($c in $Candidates) {
        $v = Get-PythonVersion -Exe $c.Exe -PreArgs $c.PreArgs
        if ($null -eq $v) {
            $why = "did not run"
            if ($c.Exe -match '[\\/]Microsoft[\\/]WindowsApps[\\/]') { $why = "Microsoft Store shortcut, not a real
Python" }
            $script:E6aiPythonTried += "  $($c.Label): $why"
            continue
        }
        if ($v -lt $E6aiMinPython) {
            $script:E6aiPythonTried += "  $($c.Label): Python $v is too old (need $E6aiMinPython or newer)"
            continue
        }
        $c.Version = $v
        return $c
    }
    return $null
}
```

What to check: First candidate that runs and is new enough; every rejected one is recorded with why (did not run / Store shortcut / too old) for the message.

`run.ps1` lines 205-258

```
function Initialize-Venv {
    # Returns the path of .venv\Scripts\python.exe, creating .venv first if needed.
    # Exits 9 when no suitable Python is found, 10 when installing into .venv fails.
    $venvDir = Join-Path $E6aiRoot ".venv"
    $py = Join-Path $venvDir "Scripts\python.exe"
    # 1. the existing .venv
    if (Test-Path -LiteralPath $py) {
        if (Test-VenvPython $py) {
            if (-not (Test-ArchiverCurrent $py)) {
                # missing, half-installed, or an older version's .venv (e.g. 1.9.1): reuse it, just point
                # it at this copy. pip replaces the old editable install; nothing in the archive is touched.
                Write-Host "Updating .venv to e6ai-archiver $(Get-AppVersion) ..."
                Install-Archiver $py
            }
            return $py
        }
    }
    if (Test-Path -LiteralPath $venvDir) {
        # Broken or half-made .venv: keep it aside (not deleted) and make a new one.
        $aside = $venvDir + ".broken-" + (Get-Date -Format "yyyyMMdd-HHmmss")
        Write-Host "The existing .venv does not work (its Python was removed, is too old, or setup was interrupted)."
-ForegroundColor Yellow
        Write-Host "Moving it to $(Split-Path $aside -Leaf) and creating a new one. You can delete that folder
later."
        try {
            Move-Item -LiteralPath $venvDir -Destination $aside
        } catch {
            Write-Host "Could not move .venv ($($_.Exception.Message))." -ForegroundColor Red
            Write-Host "Close the control panel / any running archiver, delete the .venv folder by hand, and run
again."
            exit 1
        }
    }
    # 2.-5. py -3, python, python3, the usual install folders
    $found = Find-Python
    if ($null -eq $found) { Write-NoPythonHelp; exit $E6aiExitNoPython }
    Write-Host "Using Python $($found.Version): $($found.Label)"
    if (-not (Test-PythonModule -Exe $found.Exe -PreArgs $found.PreArgs -Modules "venv, ensurepip")) {
        Write-NoVenvHelp $found
        exit $E6aiExitNoPython
    }
    Write-Host "Creating virtual environment (.venv) ..."
    $ErrorActionPreference = "Continue"
    & $found.Exe @($found.PreArgs) -m venv $venvDir | Out-Host
    $code = $LASTEXITCODE
    $ErrorActionPreference = "Stop"
    if ($code -ne 0 -or -not (Test-Path -LiteralPath $py)) {
        Write-Host ""
        Write-Host "Python $($found.Version) could not create .venv (exit code $code)." -ForegroundColor Red
        Write-Host "Check that this folder is writable (not inside OneDrive / Program Files), delete any half-made
.venv,"
        Write-Host "and run the same command again. If it keeps failing, reinstall Python from
https://www.python.org/downloads/windows/"
        Write-Host "(Exit code $E6aiExitNoPython = no suitable Python.)"
        exit $E6aiExitNoPython
    }
    Install-Archiver $py
    return $py
}
```

What to check: Existing .venv reused if it runs (and finishes a half install); a broken one is moved aside, not deleted; no Python -> help + exit 9; no venv module -> exit 9; venv / pip exit codes checked (pip failure -> exit 10); pip output goes to Out-Host so it is not part of the return value.

22. 2.0: dashboard fix, upgrades from 1.6 - 1.9.1, distribution

Root cause of the stuck 'Whole archive' box: the CLI sent the whole-archive snapshot only once, before the posts export was merged, and a new snapshot reset the run counters. 2.0 sends fresh

totals before every download queue and adds only files saved after the latest snapshot. Upgrades: refuse a newer DB before touching anything, back up, then migrate in one transaction.

`e6ai_archiver/archiver.py` lines 294-300

```
    def _emit_archive(self) -> None:
        """2.0: fresh whole-archive totals right before each download queue, i.e. after the posts export
        has been loaded/merged (the dashboard used to show only the snapshot taken at the very start)."""
        try:
            emit("archive", **self.s.archive_counts())
        except Exception as e:  # noqa: BLE001 - the dashboard must never stop a download
            log.debug("archive totals unavailable: %s", e)
```

What to check: Called right before each download queue, i.e. after the export merge; any error is only logged at DEBUG so the dashboard can never stop a download.

`e6ai_archiver/gui.py` lines 156-163

```
    def _on_archive(self, d):
        self.archive = {k: int(d.get(k) or 0) for k in (
            "expected_files", "expected_bytes", "done_files", "done_bytes", "pending_files", "pending_bytes",
            "failed_files", "failed_bytes", "no_url_files")}
        # 2.0: the snapshot already contains every file saved so far; only files saved after it are added on
        # top (the 1.9 code reset the run counters instead, and the CLI sent the snapshot only once, before
        # the posts export was loaded - so the totals stayed at the handful of rows of the old database).
        self._arch_base = (self.ok_files, self.ok_bytes)
```

What to check: The snapshot already counts every file saved so far; only files saved after it are added on top (no double counting, no reset).

`e6ai_archiver/store.py` lines 71-86

```
def refuse_if_newer(db_path: Path) -> None:
    """Raise DatabaseTooNewError if db_path exists and was written by a newer schema (read-only check)."""
    if not (db_path.is_file() and db_path.stat().st_size > 0):
        return
    try:
        con = sqlite3.connect(f"{db_path.as_uri()}?mode=ro", uri=True, timeout=10)
    except sqlite3.Error:
        return  # unreadable here: the normal open reports it (or recovers a damaged file)
    try:
        v, app = _db_version(con)
    except sqlite3.Error:
        return
    finally:
        con.close()
    if v is not None and v > SCHEMA_VERSION:
        raise DatabaseTooNewError(db_path, v, app)
```

What to check: Read-only peek at the DB's app_version / schema before anything is created, locked or migrated; newer -> DatabaseTooNewError -> exit 11.

`e6ai_archiver/store.py` lines 320-349

```
    def _backup_before_upgrade(self, written_by, required: bool):
        """Copy the database (consistent, WAL included) to <out>/db-backups/ before it is changed.
        required=True (schema changes pending): no backup -> no upgrade (RuntimeError, nothing touched)."""
        ts = time.strftime("%Y%m%d-%H%M%S")
        dest_dir = self.out / "db-backups"
        dest = dest_dir / f"{DB_NAME}.before-{__version__}-from-{written_by or 'pre-2.0'}-{ts}"
        size = sum((self.out / (DB_NAME + x)).stat().st_size for x in ("", "-wal")
                   if (self.out / (DB_NAME + x)).exists())
        log.info("backing up the database (%.0f MiB) before opening it with version %s ...", size / 2**20,
                 __version__)
        try:
            dest_dir.mkdir(exist_ok=True)
            from .platform_utils import free_bytes
            if free_bytes(dest_dir) < size + 64 * 1024 * 1024:
                raise OSError(f"not enough free space for a {size / 2**20:.0f} MiB copy")
            tmp = dest.with_name(dest.name + ".tmp")
            dst = sqlite3.connect(str(tmp))
            try:
                self.db.backup(dst)
            finally:
                dst.close()
            safe_replace(tmp, dest)
        except (OSError, sqlite3.Error) as e:
            if required:
                raise RuntimeError(f"could not back up the database before upgrading it ({e}); nothing was "
                                   f"changed. Free some space on the archive drive and start again.") from None
            log.warning("could not back up the database (%s); continuing (no schema change is needed)", e)
            return None
        log.info("database backed up before the upgrade to %s: %s", __version__, dest)
        return dest
```

What to check: SQLite online backup to /db-backups/ before an upgrade; not enough space -> refuses with 'nothing was changed' when the backup is required.

`e6ai_archiver/control_panel.py` lines 901-936

```
def import_previous_install(src: str | os.PathLike, dest_ini: str | os.PathLike) -> dict:
    """Copy an earlier install's settings into dest_ini and point `out` at that install's archive (as an
    absolute path), so the new copy keeps archiving into the same folder and nothing is downloaded again.
    Nothing in the old install or its archive is changed or moved. An existing dest_ini is kept as
    <name>.before-import-<time>. Raises ValueError if src is not an e6ai-archiver install."""
    import shutil
    info = install_info(src)
    if not info:
        raise ValueError(f"{src}: no {INI_NAME} and no archive database there; choose the folder that has "
                         f"'{INI_NAME}' or the 'archive' folder in it (for example C:\\e6ai-archiver).")
    dest_ini = Path(dest_ini)
    if info["ini"] and Path(info["ini"]).resolve() == dest_ini.resolve():
        raise ValueError("That is this install's own settings file; nothing to import.")
    cpr = _parser()
    notes = []
    if info["ini"]:
        with open(info["ini"], encoding="utf-8-sig") as f:
            cpr.read_file(f)
        if cpr.get(SECTION, "api_key", fallback="").strip():
            notes.append("The old settings file contains an API key; it was copied as it is. Safer: delete it "
                         "from the file and use the E6AI_API_KEY environment variable.")
    if not cpr.has_section(SECTION):
        cpr.add_section(SECTION)
    cpr.set(SECTION, "out", info["out"])
    backup = None
    if dest_ini.is_file():
        backup = dest_ini.with_name(dest_ini.name + ".before-import-" + time.strftime("%Y%m%d-%H%M%S"))
        shutil.copy2(dest_ini, backup)
    dest_ini.parent.mkdir(parents=True, exist_ok=True)
    tmp = dest_ini.with_name(dest_ini.name + ".tmp")
    with open(tmp, "w", encoding="utf-8", newline="\n") as f:
        f.write(f"; imported from {info['dir']} by e6ai-archiver {__version__} on "
                f"{time.strftime('%Y-%m-%d %H:%M')}\n")
        cpr.write(f)
    os.replace(tmp, dest_ini)
    return dict(info, dest=str(dest_ini), backup=str(backup) if backup else None, notes=notes)
```

What to check: Copies the old ini, points `out` at the old archive (absolute), keeps an existing ini as .before-import-; never touches the old install.

`run.ps1` lines 135-144

```
$E6aiApp = $PSScriptRoot
$E6aiRoot = $PSScriptRoot
if ((Split-Path -Leaf $PSScriptRoot) -eq "app" -and (Test-Path -LiteralPath (Join-Path $PSScriptRoot
"e6ai_archiver"))) {
    $E6aiRoot = Split-Path -Parent $PSScriptRoot
}
if (-not $Out) { $Out = Join-Path $E6aiRoot "archive" }
Set-Location $E6aiRoot
# Python must import the archiver from app\ (via .venv), never an old copy left in the install folder by
# an upgrade "unzip over" (1.x had e6ai_archiver\ there). 3.11+: do not put the current folder on sys.path.
$env:PYTHONSAFEPATH = "1"
```

What to check: app\ vs install folder: settings, .venv and the default archive stay in the install folder (1.x paths); PYTHONSAFEPATH stops an old e6ai_archiver\ left by 'unzip over' from being imported.

`e6ai_archiver/exports.py` lines 97-103

```
def _log_no_checksum(entry: dict, fname: str) -> None:
    kind = _export_kind(entry, fname)
    if kind in CHECKSUM_IMPORTANT and kind not in NEVER_CHECKSUMMED:
        log.warning("export %s has no checksum in the listing; it cannot be verified", fname)
    else:
        log.info("export %s has no checksum in the listing (normal for this export); saved without "
                 "a checksum check", fname)
```

What to check: Only the posts export warns when the listing has no checksum; every other export (artists never has one) logs at INFO, so it is no longer the panel's 'Last problem'.

5. Test results

- `python -m unittest discover -s tests -v`: **337 tests, all pass** (about 85 s; Linux, Python 3.13, SQLite 3.53, Tk 9.0 under Xvfb), also under a socket guard that blocks every non-localhost connection (0 attempts). **21 new for 2.0** (`tests/test_v20.py`): the dashboard totals across the export merge (no double counting, ETA shown, "left" count), a stale snapshot, the archiver emitting totals before each queue; the artists checksum note at INFO and posts at WARNING; newer DB = exit 11 with nothing touched; backup and history only on a version change; a failed migration rolls back; no space for the backup; resume of real 1.6 / 1.7 / 1.8 / 1.9.1 partial archives (see 2a); importing settings from an old install; old ini keys; a running 1.9.1 = exit 5; the panel at minimum and default size (log lines, buttons inside, big Start, banner wraps); the import banner and button; `run.ps1` finding the install folder from `app\` (pwsh); the bundle files, README rules (ASCII, CRLF, required topics) and the zip manifest. Three earlier version asserts now accept 2.0. **14 new for 1.9.1** (`tests/test_v191.py`):
- static checks: no bare `py -3`, min version = pyproject, help / README / CHANGELOG, batch files pass the exit code, no PS7-only operators;
- pwsh behaviour tests with fake pythons: Store stub skipped, too-old skipped, `py -3` first, install folders newest first, marker-less output rejected, `check-python` exit 0 / 9, and the owner's no-Python case through `-Mode test` exits 9 without making `.venv`;
- a missing venv module is detected, and the scripts parse.

The pwsh tests skip when PowerShell is absent. test_v19's version check now accepts 1.9.x. **39 new for 1.9** (`tests/test_v19.py`): theme names, the Windows registry value (injected reader), resolve, both palettes complete with every text / log colour at least 4.5:1; `[gui]` theme / notify defaults, saving creates a CLI-valid ini, Save settings keeps the theme, an unparseable ini is left alone; `--on-low-disk` / `--failure-log-max-mb` on the command line, validation (exit 2), env vars, the run record; low disk: pause then continue when space is freed, Resume wakes at once, Stop while waiting keeps files pending, `stop` = exit 4, unknown mode = pause; trimming keeps still-failed rows and archives the rest, small files / 0 untouched; resume-offer rules and pid check; disk estimate, notification text, backends and PowerShell escaping; dashboard model and archive counts; the report's dark CSS; docs and version; under Xvfb: instant theme switch saved to the ini, start in the saved theme and follow the system, the buttons and notify checkbox, the resume banner, the disk-low event + Resume button, the pre-Start disk warning, the end-of-run notification with popup fallback, the idle dashboard. Earlier tests unchanged except the UA version string. **30 for 1.8** (`tests/test_v18.py`): `with_tags`, sync fetches + loads tags, listing without tags = note, missing tags fetched inside the daily window, `exports --download posts`, `ensure_tags` fetch / no refetch / stale / unavailable / site down / fetch off, `report` fetches tags with an anonymous client (key in env, never used) and counts real directors, `--no-fetch-tags`, report still written when

the site is down; failure CSV columns, HTTP status, tries per run, totals across runs, header once, recovered rows, events + summary + GuiModel counters, no secrets (user-info, query, key in path), the `attempts` migration; reason groups and unknown words, `--from-csv` skip rules; `retry-failed` downloads only failed files (a pending file stays untouched), `--reason`, still failing logged again, dry run offline, from CSV, bad reason = exit 2, interrupted retry restores status, nothing failed; panel mode / options / button under Xvfb, launchers and ini. Two earlier tests changed: the UA version string, and the 1.7 report helper passes `--no-fetch-tags`. **24 for 1.7** (`tests/test_v17.py`): every report bucket and reason on one fixture archive (old export seeded, newer export compared), filter and tier skips, extra / `.part` / quarantine files, directors from the tags export (excluded warning tags, posts not in the DB), `missing.csv` columns, URLs and the `missing.csv` copy, txt / html / `state.last_report` / the `report` event / `latest_report`, the report changes nothing in the DB or on disk, `--no-disk-check`, no export, the export date from the manifest, the `report` command offline (no client is created) and with `--json`, no export = exit 1, `full` writes the report after verify, `--no-report-after`, a failing report = exit 1 while a missing export = exit 0, `report_after` CLI / env / ini, failure reasons recorded (404, 403, 429, wrong size, md5) and cleared on success, `verify_zero_byte`, the stale-CDN retry (stale then fresh; stale twice = error, nothing kept; fresh at once = one request), panel mode / options / GuiModel event / the Open report button under Xvfb. One earlier test changed: the UA version string. **28 for 1.6** (`tests/test_v16.py`, with a second local server standing in for the static host): parallel run complete and really concurrent (peak 4-16 connections, never more than `--workers`), strict newest-first order with 1 worker, static files bypass a 1 req/s API limiter, every DB write on the calling thread, final name only when complete, crash `.part` leftovers, truncated transfers never placed, duplicate md5 fetched once, 20 consecutive 403s raise `ForbiddenError` (and CLI exit 3) after at most 52 requests, isolated 403s don't stop, 429/503 back-off then success or `failed`, Stop with 16 workers (DB and disk agree, resume completes), Pause holds new requests, verify delete / quarantine / requeue / redownload / resume / `--only-new`, same-size corruption caught inline with `--check-md5` and by `verify` without it, CLI verify offline by default, options / env / ini precedence, bad values exit 2, dry-run fields. **Earlier tests updated** (each reason in CHANGELOG): the rate-cap tests in test_offline / test_v14 / test_v15 (cap removed), test_offline's md5 test now sets `check_md5=True`, three tests that depended on strict download order (test_v12 x2, test_v13), `test_errors_disable_start` (uses "fast" as the invalid rate) and the panel round-trip sampler (flips a flag's default). - **Local throughput** (127.0.0.1, `build/bench16.py`): per-connection 2 MiB/s + 80 ms latency, 32 x 2 MiB: 1 worker 1.8 MiB/s (34.8 s), 16 workers 29.5 MiB/s (2.2 s). 200 x 200 KiB at 50 ms latency: 18.7 vs 182.7 files/s. Unthrottled loopback, 64 x 4 MiB: 555 vs 912 MiB/s. Verify hashing (page cache): 731 MiB/s with 1 thread, about 2.5 GiB/s with 4 on 8 cores; on a real disk the drive is the limit. - 1.5: **35 new for 1.5** (`tests/test_v15.py`): the key is never in argv (run and test-login commands), the command preview, the saved ini or the log file, and is masked in tracebacks / the excepthook / parsed results; keyring absent (Remember hidden), a fake keyring present (save, load, forget), a failing backend; Basic auth sent only to the API host; 401 not retried; `test-login` with mocked 200 (own account: exit 0, exactly 1 request; safe-mode warning; public profile only: exit 8), 401 (exit 8), 403 (exit 3, one request), no key (exit 2, no request), rate cap 2/s; re-queue: `no_url` detection, chunks of 100, deleted posts excluded except in `recheck`, the 24 h skip per account / set, `download` auto-retry and `--no-login-retry`, `stats.login_required`; the panel: sign-in on Basics not Advanced, masking, background Test sign-in with 401 / 403 (last exit 3, asks before retrying). **43 for 1.4** (`tests/test_v14.py`): an audit that every CLI option has a widget or a listed reason; every option round-trips through the real argparse parser with its value unchanged; unused options left out; API key only in the child environment; validation (rate cap, username, browser UA, numbers, templates, path warnings); settings save/load with the CLI reading the same ini and ignoring `[gui]`; `--no-config`; the meta `--max-items` fix; control-file

watcher and progress stream; **real child processes**: a stats run (exit 0), pause (no progress while paused), resume, Stop (exit 130 in under 5 s, then a second run gets the lock), Stop while paused, no double start, panel closing the pipe (exit 130), kill; and the Tk window under Xvfb (every option has a widget, state/preview round trip, greying, errors disable Start, Browse, Save, Start in stats mode, the Stop button). One existing test changed: the UA version string. 17 new for 1.3: ties and null score/favs, smallest-first/balanced, every filter, Spec validation and env, tiers (`--top`, `--limit-gb`), switching order mid-run (finished files kept, new order honoured), API JSON batching (1 request per 100), incremental re-seed with posts deleted/vanished between exports, truncated-export guard, header aliases/unknown/missing columns, sync gate/force/corrupt download/previous fallback/manifest, missing/stale export and listing errors, **403 during sync stops with no further request**, CLI (`sync-exports`, `preview`, `--dry-run`, `full --order`, invalid order/rating = exit 2), the GUI start panel (invalid then valid input, Cancel before Start = 130 with no request, `--autostart`). Earlier tests: 40 tests carried over from 1.1 (pagination, resume, 403 stop, rate cap, retries, md5, disk full, seeding, config, CLI) plus 46 new ones: sanitizer, source_name, templates and presets, layout/reorganize, file edge cases, database recovery, network outages/DNS/Retry-After, export and seed edge cases, Pause/Stop control, config/CLI 1.2, the GUI model, and a real Tk window closed mid-run (expects a clean stop, exit 130). - `pyflakes`: clean. - All tests use a local fake server. The real 2026-11-03 exports (posts, tags, pools) were used offline for the tier numbers, timings and the source_name statistics. Live requests for 1.3: 8 in total (see section 3).

6. Pre-run checklist

☐ Confirm the use is permitted by e6ai's Terms of Service (the site is closing at the end of November 2026).
☐ Set your **e6ai username** (`-Username`, `E6AI_USERNAME`, or `username=` in `e6ai-archiver.ini`). Never a browser UA.
☐ Keep the API rate at **1 req/s** (the e621-family API docs ask for about 2/s or less; there is no cap now).
☐ Decide on the number of parallel downloads (default 16). Because the "chief architect" authorization is only a Discord screenshot, consider confirming it with site staff, or start lower (e.g. `-Workers 4`) and watch for 403/429 in the log.
☐ Optional sign-in: create an API key on https://e6ai.net/users/home (not your password), paste it in the panel (Basics > Sign in to e6ai), press **Test sign-in** once. CLI: `E6AI_API_KEY` env var only, then `e6ai-archiver --username NAME test-login`. If it reports safe mode, q/e posts stay hidden.
☐ Windows: Python 3.11+ from python.org ("Add to PATH", "Disable path length limit"); unzip to a short local path.
☐ Archive on a local NTFS drive with **600 GB+ free**, not OneDrive/Dropbox/network share.
☐ Double-click `run_gui.bat` (control panel). Enter the username, Browse to the archive folder, Save settings.
☐ Mode "Test run", Start; then "Preview the download order". Or from PowerShell: `.\run.ps1 -Username NAME -Mode test`, then `-Mode preview -Order popular -Count 20` to check order, names and folders.
☐ Pick an order and tier, e.g. `.\run.ps1 -Username NAME -Mode full -Order popular -LimitGB 50 -Gui` (everything is 526.23 GiB: about 25 h at 50 Mbit/s, 12.5 h at 100). Stop/resume any time.
☐ Daily while it runs: `-Mode sync` (or just rerun `full`; it checks the exports at most once per 20 h).

☐ Afterwards: `-Mode retry-failed` (only the failed files; `-Reason network,md5`, `-DryRun` first to see counts; `failed-downloads.csv` in the archive folder lists each failure); `full` already verifies checksums when it finishes (or `-Mode verify`).
☐ Open the completeness report (`full` writes it at the end; panel **Open latest report**, or `-Mode report`): check "missing" and "failed" by reason and work through `missing.csv`.
☐ Low disk: the panel pauses and notifies (Resume when space is freed). For unattended runs use `-OnLowDisk stop` (the daily task does).
☐ On **exit code 3 (HTTP 403)**: stop. Don't change the UA or retry.

7. Known limits and risks

- **retry-failed and the failure log were only run offline** (local fake servers). Not verified against the live file host: which failures are transient there, and real failure volumes. The tags auto-fetch was verified live once (anonymous). Retrying 403s repeatedly can still trip the 20-consecutive-403 stop (exit 3); that is intended - don't loop retry-failed on 403s.
- Since 1.9 `failed-downloads.csv` / `.log` are trimmed past `--failure-log-max-mb` (default 10); the gzip archives next to them keep growing (small: about 10x compression).
- **2.0 was not run on real Windows.** The zip layout, the .bat files, `-ExecutionPolicy Bypass`, SmartScreen / Mark-of-the-Web behaviour, "unzip over" with Explorer, reusing a real 1.9.1 `.venv`, and the Import banner's Windows search folders (`C:\e6ai-archiver`, the user folders) were checked only on Linux (pwsh 7.6 parse + PSScriptAnalyzer, fake layouts). The upgrade tests use the real old Python code, not real old Windows installs. Screenshots are simulated data under Xvfb.
- **1.9.1's Python finder was not run on real Windows.** It was checked with PowerShell 7.6 on Linux, PSScriptAnalyzer's PS 5.1 compatibility rules and fake executables (including a stand-in for the Store shortcut that behaves like the real one: message on stderr, non-zero exit). Not verified:
- Windows PowerShell 5.1 running it;
- the real `WindowsApps` alias (that it never opens the Store when given arguments);
- real python.org / Store / winget installs;
- the end-to-end `.venv` creation and `pip install` on Windows.
- **1.9 was not run on Windows**: the registry read for Follow Windows, the toast / balloon (PowerShell), the dark title bar (DWM) and the `run.ps1` / `install_daily_task.ps1` changes are untested (no Windows / PowerShell here); each falls back silently (light theme, in-panel window). Screenshots are simulated data on Linux. The low-disk pause was only tested with a simulated free-space function.
- **The completeness report was run only on simulated archives.** It has not seen a real finished archive. Per-director numbers need the tags export; since 1.8 `report` fetches it if needed (or notes why it couldn't). "Expected" is what the export lists as not deleted with an md5, which includes 114 posts still pending approval; posts hidden from anonymous visitors (`login_only`) are counted as failed.
- The report is as fresh as the newest posts export in the archive (one a day, about 23:00 US Eastern).
- **Fast mode was never run against the live site.** Not verified: how `static1.e6ai.net` reacts to 16 parallel connections (rate limits, 403/429, connection caps, CDN behaviour), the real throughput, and whether 20 consecutive 403s is the right stop threshold there.

- **The authorization is unverified.** It rests on a Discord screenshot attributed to "DRLa", presented as e6ai's chief architect. Nothing confirms the identity or that e6ai allows unthrottled downloads; the e621-family API docs ask for about 2 req/s or less (that applies to the API, which still uses its limiter).
- **Not run on Windows** (1.6 included): threads + NTFS rename under antivirus, `run.ps1 -Workers` / `-Mode verify`, `run.bat` WORKERS argument.
- **Sign-in was not tested against the live site** (no signed-in requests by design). Verified offline with mocks: request shape, 200 / 401 / 403 handling, re-queueing. Not verified: that e6ai currently has the same hiding rules as e621ng master, which posts (if any) it hides from anonymous visitors, the exact fields `/users/<name>.json` returns there, whether its API keys can be limited in scope (a key without read access to the user page would fail the test), whether the static file server serves hidden posts without sign-in, and the Windows Credential Manager (keyring was simulated). Deleted posts stay hidden for normal accounts; account safe mode hides questionable / explicit posts.
- **The control panel (1.4 / 1.5) was not run on Windows.** Verified only on Linux (Xvfb): the window, Browse (dialog patched in tests), Start / Pause / Stop with real child processes. Not verified: `CREATE_NO_WINDOW` and the control file on NTFS, `pythonw.exe`, the Explorer folder dialog, DPI scaling on a high-DPI screen, `run_gui.bat` / `run.ps1 -Mode panel`, `os.startfile` for Open folder / Open log, and any panel run against the live site.
- **Not tested on real Windows**, and the GUI has not been run against the live site. Windows code paths (long paths, msvcrt lock, Ctrl-Break, keep-awake, DPI, `os.startfile`, PowerShell/batch launchers) were reviewed but only run on Linux. Run `-Mode test -Gui` first.
- File paths are fixed when a file is saved. Changing templates or tag edits on the site later do **not** rename existing files; use `reorganize` (dry run, then `--apply`).
- About 70 % of posts have no usable source, so most default names are `<id>.<ext>`.
- Size **526.23 GiB** (162,295 files in the 2026-11-03 export); time is bandwidth-bound in 1.6 (about 25 h at 50 Mbit/s).
- robots.txt disallows AI-training crawlers; the archive must not be used for AI training.
- The order uses the export's score/favs, which can be up to a day old; ranks change only when `sync-exports` merges a newer export (or the order/filters change). Not tested against a live 403 or a live schema change (both simulated offline). The GUI start panel was only run under Xvfb.
- `post_versions`, `wiki_pages`, `tag_aliases`, `tag_implications`, `post_replacements`, `bulk_update_requests` and `directors` are archived as files but not merged into the database.
- 31,477 deleted posts (2026-11-03 export) have no files; their metadata is kept. Anonymous-hidden posts are `no_url` (`recheck` with an API key).